SPRING-BOOT 3.5 → 4.0

Spring Boot 4.0 Migration Guide

45 Breaking Changes You Need to Fix

Tier 1 · Won't Build	19 challenges
Tier 2 · Won't Run	13 challenges
Tier 3 · Wrong Results	13 challenges

Estimated total migration effort:
200–500 hours depending on codebase size

Migration Effort Estimates

How we sized the 45 breaking changes — and what the numbers mean.

T-SHIRT SIZES

Each card carries an effort badge — **S**, **M**, or **L** — representing the typical engineering effort per card, for a medium-sized codebase (~50k LOC).

SMALL

2–6 h

Config change, dependency swap, or one-line fix. Low risk of follow-on work.

17 cards

MEDIUM

6–16 h

API migration, code rewrite, or multi-file refactor. May need testing and validation.

24 cards

LARGE

16–40 h

Major subsystem migration. Touches many files, requires security review or arch decisions.

4 cards

HOW WE CALCULATED

Estimates are based on the HeroDevs Spring Boot migration research, validated against the **Spring I/O 2026 quiz** (88 developers, mean score 66%), production migration audits, and per-subsystem breakdowns:

SUBSYSTEM	TYPICAL HOURS	KEY DRIVER
Jackson 3.0	16–40 h	Group ID + class renames across every file that imports Jackson
Spring Security 7	8–16 h	DSL rewrite, OAuth removal, default-deny policy
Hibernate 7	4–8 h	Dialect removal, Session API changes, date type shifts
Testing (JUnit 6+)	16–32 h	@MockBean removal, test slice relocation, Testcontainers package
Build tooling	8–32 h	Java 17 minimum, Gradle 8.x, Maven AOT plugin
Observability	4–8 h	Controller spans disabled, OTLP endpoint flip

Total for a medium app: **120–230+ hours** (weeks, not days). Minor version bumps (3.2→3.3, 3.3→3.4) averaged 6–8 hours each. The 4.0 jump is 15–30× that.

OPENREWRITE COVERAGE

17 of 45 cards (38%) have an OpenRewrite recipe that can automate all or most of the fix.

Tier 1: 9 of 19 (47%) — best coverage, mostly dependency/import swaps

Tier 2: 7 of 13 (54%) — good coverage for config and security rewrites

Tier 3: 1 of 13 (8%) — behavioural changes are hardest to automate

Key insight: OpenRewrite handles the loud failures well (Tiers 1–2), but Tier 3 "wrong results" bugs — the ones customers find — are almost entirely manual detective work.

Harder Than It Looks

These changes are sized Small or Medium — but carry hidden complexity that can blow up the estimate.

HIGH RISK — EFFORT LIKELY UNDERSTATED

CARD	SIZE	RISK	WHY IT'S WORSE THAN IT LOOKS
AspectJ @Observed	S	HIGH	**Completely silent.** Observability via @Observed stops working. No errors, no warnings. You find out when dashboards go blank in production.
OTLP HTTP Primary	S	HIGH	**Silent production failure.** Traces/metrics vanish. Requires network policy changes (port 4317→4318) and possibly different vendor endpoints.
Batch Schema Rename	S	HIGH	**Not just a rename.** Column types and table structures changed. Rolling deploys break. Tests pass on H2 while production fails on real databases.
spring-jcl Removed	S	HIGH	**Enterprise POM cascade.** The commons-logging exclusion pattern used for a decade is now harmful. Corporate parent POMs break downstream.
Hibernate Processor	S	HIGH	**Unbounded compile failures.** New processor validates HQL at build time. A one-line GAV change surfaces every broken query in the codebase.
Jackson Locale Format	S	HIGH	**Silent data corruption.** Locale strings flip from `zh_CN` to `zh-CN`. DB equality checks silently fail in i18n-heavy apps.
Gradle Version	S	MOD	**Multi-step upgrade.** Can't jump straight to 8.14. Staged through major boundaries; Gradle 9 enables config cache by default.
Path Matching Engine	M	HIGH	**Security hole.** URL patterns that silently stop matching can expose previously-protected endpoints. Not just a broken feature — a vulnerability.
Auth Default Deny	M	HIGH	**Infra breakage.** Actuator endpoints silently blocked. Health checks, readiness probes, and load balancer monitoring fail in production.
Retryable Txn Order	M	HIGH	**Near-impossible to test.** AOP ordering change causes rolled-back data before retry. Subtle data consistency issues that unit tests won't catch.
Jackson Dates/Timestamps	M	MOD	**Compounding changes.** Interacts with the separate date-format change. Two Jackson date shifts at once makes debugging doubly confusing.
Test Slice Relocated	S	MOD	**Silent test degradation.** Tests may load full application context instead of a thin slice. Tests pass but are slower and less precise.

Pattern: The most dangerous changes are Tier 3 "Wrong Results" and Tier 2 silent failures. They don't break your build — they break your production data, your observability, or your security posture. Budget extra time for these even when the per-card effort is small.

TIER 1

Won't Build

Compilation & Dependency Failures

Your code won't compile. These are import, dependency, and API changes that produce immediate build failures.

SUBSYSTEM INDEX · 19 CARDS

Jackson

- Jackson 3.0 Group ID & Class Renames
- Jackson Exception Hierarchy
- Jackson Class Renames

Build Tools

- Java 17 Minimum
- Gradle Version
- Maven AOT Plugin
- GraalVM 25

Hibernate

- Hibernate Dialect Removal
- Cascade Save-Update
- Hibernate Processor Rename
- Session.delete() Removed

Spring Security

- WebSecurityConfigurerAdapter Removed
- OpenSAML 4 Removed

Spring Framework

- ListenableFuture Removed
- OkHttp3 Removed
- javax.inject Removed
- spring-jcl Removed
- Undertow Removed

Testing

- @MockBean Removed

Actuator Endpoint @Nullable: org.springframework.lang Replaced by JSpecify

Actuator endpoint parameters must use org.jspecify.annotations.Nullable, not org.springframework.lang.Nullable.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn compile  (or runtime binding failure)
Actuator endpoint parameter annotated with @org.springframework.lang.Nullable
is no longer recognised — optional parameter treated as required,
causing MissingServletRequestParameterException at runtime.
```

WHAT CHANGED

Spring Boot 4.0 no longer recognises `org.springframework.lang.Nullable` on Actuator `@ReadOperation` / `@WriteOperation` parameters. The supported annotation is now `org.jspecify.annotations.Nullable` from the JSpecify library, which is on the classpath transitively via Spring Framework 7.0.

```
// Actuator endpoint parameter
import org.springframework.lang.Nullable;

@Endpoint(id = "my-endpoint")
public class MyEndpoint {
    @ReadOperation
    public String get(@Nullable String name) { ... }
}

import org.jspecify.annotations.Nullable;

@Endpoint(id = "my-endpoint")
public class MyEndpoint {
    @ReadOperation
    public String get(@Nullable String name) { ... }
}
```

WHY THIS CHANGED

Spring Framework 7.0 migrated its entire codebase to JSpecify null-safety annotations. Spring Boot 4.0 followed suit. The `org.springframework.lang` annotations are retained for source compatibility but are no longer the active standard.

HOW TO FIX

A **Replace org.springframework.lang.Nullable with org.jspecify.annotations.Nullable.** Update the import on all Actuator endpoint parameters. The annotation name is the same — only the package changes. No logic changes required. JSpecify is available on the classpath via Spring Framework 7.0.

✓ **Verify:** mvn compile — no cannot find symbol or binding failure for @Nullable on endpoint params

Scope check: Search for `@Nullable` in classes annotated with `@Endpoint`, `@WebEndpoint`, or `@JmxEndpoint`. The issue is specific to Actuator endpoint parameter binding.

WATCH OUT

`org.springframework.lang.Nullable` still exists on the classpath and the code will compile — the failure is at runtime when Actuator tries to bind the parameter and does not recognise the annotation. This makes it easy to miss in a compile-only check.

FURTHER INFO

spring-break module: actuator-nullable-removed · Null-safe applications with Spring Boot 4 · Spring Boot 4.0 Migration Guide

Spring Boot 4.0 adopted JSpecify as its null-safety annotation standard, following Spring Framework 7.0. org.springframework.lang.Nullable is deprecated and no longer recognised by Actuator's parameter binding logic. This is part of a broader null-safety overhaul documented in the Spring Boot 4.0 blog post on null-safe applications.

AOP Starter Renamed to spring-boot-starter-aspectj

spring-boot-starter-aop is no longer in the Spring Boot 4.0 BOM. Replace it with spring-boot-starter-aspectj.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn validate
[ERROR] 'dependencies.dependency.version' for
  org.springframework.boot:spring-boot-starter-aop:jar is missing.
```

WHAT CHANGED

`spring-boot-starter-aop` has been removed from the Spring Boot BOM. The replacement starter is `spring-boot-starter-aspectj`. The new starter provides the same AspectJ weaving capabilities under a name that better describes what it does.

```
// pom.xml dependency
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aspectj</artifactId>
</dependency>
```

WHY THIS CHANGED

Spring AOP (interface-proxy and CGLIB-proxy based AOP) works without any starter. The old name implied you needed this starter for any AOP — you only need it for AspectJ load-time or compile-time weaving. The rename makes the distinction clear.

HOW TO FIX

A **Replace the starter in pom.xml / build.gradle.** Find `spring-boot-starter-aop` in all build files and replace with `spring-boot-starter-aspectj`. No code changes are needed — the classpath content is equivalent.

✓ **Verify:** mvn validate — no version is missing error for the AOP starter

Scope check: Search all `pom.xml` and `build.gradle` files for `spring-boot-starter-aop`. Multi-module projects may declare it in several places.

WATCH OUT

If you only use Spring's proxy-based AOP (`@Aspect` beans, `@Transactional`, etc.) you may not need this starter at all. Proxy-based AOP is included in `spring-boot-starter` transitively. The aspectj starter is only required if you use AspectJ weaving agents.

FURTHER INFO

spring-break module: aop-starter-rename · Spring Boot 4.0 Migration Guide

The rename reflects what the starter actually provides — AspectJ weaving support. The old name spring-boot-starter-aop was considered misleading because Spring AOP (proxy-based) is available without any starter. This is a T1B (artifact no longer in BOM) failure: the build fails at dependency resolution, before any source compiles.

ApacheDS Embedded LDAP Support Removed

ApacheDSContainer is removed in Spring Security 7.0. Use UnboundIdContainer instead.

EFFORT: M

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/ApacheDsUsage.java:[3,57]
  error: package org.springframework.security.ldap.server does not contain
  ApacheDSContainer
```

WHAT CHANGED

`org.springframework.security.ldap.server.ApacheDSContainer` has been removed from Spring Security. The embedded LDAP server capability is now provided exclusively through `org.springframework.security.ldap.server.UnboundIdContainer`.

```
// pom.xml — swap the embedded LDAP dependency
<dependency>
    <groupId>org.apache.directory.server</groupId>
    <artifactId>apacheds-server-jndi</artifactId>
    <scope>test</scope>
</dependency>

<dependency>
    <groupId>com.unboundid</groupId>
    <artifactId>unboundid-ldapsdk</artifactId>
    <scope>test</scope>
</dependency>


// Java config — replace the container class
import org.springframework.security.ldap.server.ApacheDSContainer;
// ...
new ApacheDSContainer("dc=springframework,dc=org", "classpath:users.ldif");

import org.springframework.security.ldap.server.UnboundIdContainer;
// ...
new UnboundIdContainer("dc=springframework,dc=org", "classpath:users.ldif");
```

WHY THIS CHANGED

ApacheDS development had stalled and its Jakarta EE compatibility lagged. UnboundID is actively maintained and provides equivalent functionality for embedded LDAP testing and development use cases.

HOW TO FIX

A **Replace ApacheDSContainer with UnboundIdContainer.** Swap the dependency in your build file and update the import and instantiation. The constructor signature is the same — base DN and LDIF classpath location. The LDIF format is also compatible.

B **Use Spring Boot's embedded LDAP auto-configuration.** If you're only using embedded LDAP for tests, add `spring-boot-starter-ldap` and set `spring.ldap.embedded.base-dn` in test properties. Spring Boot will auto-configure UnboundID for you.

✓ **Verify:** mvn compile — no cannot find symbol for ApacheDSContainer

Scope check: Search for `ApacheDSContainer` and `apacheds-server-jndi` (or other ApacheDS artifacts) across Java sources and build files.

WATCH OUT

LDIF files written for ApacheDS should work with UnboundID, but verify any vendor-specific schema extensions you may have used.

FURTHER INFO

spring-break module: apacheds-ldap-removed · Spring Security 7 LDAP migration

Spring Security 7.0 dropped ApacheDS as the embedded LDAP provider. UnboundID is now the supported option. Both were available in 3.5; only UnboundID survives in 4.0. If your tests or configuration use ApacheDSContainer directly, they will not compile.

Spring Batch ChunkHandler Renamed; setJobLauncher Removed

ChunkHandler renamed to ChunkRequestHandler. JobStep.setJobLauncher replaced by setJobOperator. Both fail to compile on Boot 4.0.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/BatchRenamingUsage.java:[4,65]
  error: cannot find symbol
    symbol: class ChunkHandler
    location: package org.springframework.batch.integration.chunk
[ERROR] /src/main/java/com/example/BatchRenamingUsage.java:[18,19]
  error: cannot find symbol
    symbol:   method setJobLauncher(JobLauncher)
```

WHAT CHANGED

`org.springframework.batch.integration.chunk.ChunkHandler` has been renamed to `ChunkRequestHandler`. `JobStep.setJobLauncher(JobLauncher)` has been replaced by `JobStep.setJobOperator(JobOperator)`.

```
// ChunkHandler rename
import org.springframework.batch.integration.chunk.ChunkHandler;
// ...
ChunkHandler handler = ...;

import org.springframework.batch.integration.chunk.ChunkRequestHandler;
// ...
ChunkRequestHandler handler = ...;


// JobStep launcher → operator
jobStep.setJobLauncher(jobLauncher);
jobStep.setJobOperator(jobOperator);
```

WHY THIS CHANGED

The rename of `ChunkHandler` clarifies its role in the request/response model of remote chunking. `setJobLauncher` was removed as part of the broader move away from `JobLauncher` toward `JobOperator` as the public API for job execution control.

HOW TO FIX

A **Rename ChunkHandler to ChunkRequestHandler.** Update the import and all references. The interface contract is the same — only the name changed.

B **Replace setJobLauncher with setJobOperator.** Inject a `JobOperator` bean instead of `JobLauncher` and pass it to `setJobOperator()`. `JobOperator` is available as a Spring-managed bean when Spring Batch autoconfiguration is active.

✓ **Verify:** mvn compile — no cannot find symbol for ChunkHandler or setJobLauncher

Scope check: Search for `ChunkHandler` (not `ChunkRequestHandler`) and `setJobLauncher` across all Java/Kotlin sources. Affects remote chunking setups and custom job step configuration.

WATCH OUT

`JobLauncher` itself still exists — only its use in `JobStep` was removed. Other usages of `JobLauncher` are unaffected.

FURTHER INFO

spring-break module: batch-chunkhandler-renamed · Spring Batch 6.0 Migration Guide

Two related Spring Batch integration changes in 6.0. ChunkHandler is part of the remote-chunking integration module. setJobLauncher on JobStep was removed because JobLauncher was replaced by JobOperator as the preferred way to launch jobs programmatically. Both are mechanical renames.

Spring Batch JobBuilder(String) Constructor Removed

JobBuilder(String) is removed. JobRepository is now mandatory at construction: new JobBuilder("name", jobRepository).

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/JobBuilderUsage.java:[9,33]
  error: no suitable constructor found for JobBuilder(java.lang.String)
```

WHAT CHANGED

`new JobBuilder(String name)` and the subsequent `.repository(JobRepository)` method have been removed. `JobRepository` is now mandatory at construction time: `new JobBuilder(String name, JobRepository repository)`. The same applies to `StepBuilder` and other builder classes.

```
// Before — repository set after construction
new JobBuilder("myJob")
    .repository(jobRepository)
    .start(step)
    .build();

None

// After — repository required at construction
None
new JobBuilder("myJob", jobRepository)
    .start(step)
    .build();


// @Bean method pattern
@Bean
public Job myJob(Step step) {
    return new JobBuilder("myJob").start(step).build();
}

@Bean
public Job myJob(JobRepository jobRepository, Step step) {
    return new JobBuilder("myJob", jobRepository).start(step).build();
}
```

WHY THIS CHANGED

Spring Batch 6.0 made `JobRepository` non-optional throughout the framework to ensure jobs are always backed by a persistent or in-memory store. Deferring the repository assignment was a source of subtle bugs where jobs ran without persistence.

HOW TO FIX

A **Inject JobRepository and pass it to the constructor.** Add `JobRepository` as a parameter to your `@Bean` method (Spring will inject it) and pass it as the second argument to `new JobBuilder(...)`. The same change is needed for `StepBuilder` and any other builder that previously accepted only a name.

✓ **Verify:** mvn compile — no cannot find symbol for JobBuilder(String) constructor

Scope check: Search for `new JobBuilder(`, `new StepBuilder(`, and `.repository(` across all `@Configuration` classes that define Batch jobs and steps.

WATCH OUT

The `.repository()` method is also gone. If you try to add the repository via the fluent chain rather than the constructor, it won't compile either. Pass it in the constructor — there is no fallback.

FURTHER INFO

spring-break module: batch-job-builder-string-constructor · Spring Batch 6.0 Migration Guide

Spring Batch 6.0 enforces that a JobRepository is always provided at construction time. The same change applies to StepBuilder and other builder classes. The JobRepository is no longer optional or deferrable — it must be injected and passed in when the builder is created.

Batch Listener Base Classes Removed

Spring Batch 6.0 removed the abstract listener base classes like JobExecutionListenerSupport and StepExecutionListenerSupport.

EFFORT: M

⚙ OpenRewrite

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/batch/JobCompletionListener.java:[4,52]
  error: cannot find symbol
    symbol:   class JobExecutionListenerSupport
    location: package org.springframework.batch.core.listener
[ERROR] /src/main/java/com/example/batch/StepLoggingListener.java:[4,52]
  error: cannot find symbol
    symbol:   class StepExecutionListenerSupport
```

WHAT CHANGED

Spring Batch 6.0 deleted the `*ListenerSupport` abstract classes: `JobExecutionListenerSupport`, `StepExecutionListenerSupport`, `ChunkListenerSupport`, `ItemReadListenerSupport`, and others. Listeners must now implement the interfaces directly, which have had default methods since Batch 5.0.

```
// Job listener
import org.springframework.batch.core.listener.JobExecutionListenerSupport;

public class JobCompletionListener extends JobExecutionListenerSupport {
    @Override
    public void afterJob(JobExecution jobExecution) {
        log.info("Job completed: {}", jobExecution.getStatus());
    }
}

import org.springframework.batch.core.JobExecutionListener;

public class JobCompletionListener implements JobExecutionListener {
    @Override
    public void afterJob(JobExecution jobExecution) {
        log.info("Job completed: {}", jobExecution.getStatus());
    }
}


// Step listener
import org.springframework.batch.core.listener.StepExecutionListenerSupport;

public class StepLogger extends StepExecutionListenerSupport {

import org.springframework.batch.core.StepExecutionListener;

public class StepLogger implements StepExecutionListener {
```

WHY THIS CHANGED

Java 8 default methods made the support classes unnecessary. The interfaces now provide no-op defaults for every callback, so there's nothing to inherit from. Removing the support classes simplifies the class hierarchy.

HOW TO FIX

A **Change extends to implements.** Replace `extends JobExecutionListenerSupport` with `implements JobExecutionListener`. Same for all other `*ListenerSupport` classes. The interfaces have default methods, so you only need to override the callbacks you use.

B **Use @BeforeJob / @AfterJob annotations.** Spring Batch also supports annotation-based listeners. Annotate methods with `@BeforeJob`, `@AfterJob`, `@BeforeStep`, `@AfterStep` etc. on any POJO — no interface needed.

✓ **Verify:** mvn compile and Batch job completes with listener callbacks firing

Scope check: Search for `ListenerSupport` and `extends.*ListenerSupport` across your batch job classes. Common hits include `JobExecutionListenerSupport`, `StepExecutionListenerSupport`, and `ChunkListenerSupport`.

WATCH OUT

If your listener extends a support class and calls `super.afterJob()` or `super.beforeStep()`, remove those super calls. The interface default methods are no-ops, but calling `super` on an interface method looks wrong and may confuse static analysis tools.

The `ItemWriter` and `ItemProcessor` interfaces also changed in Batch 6.0. If you're fixing listeners, audit your entire batch configuration while you're at it.

FURTHER INFO

spring-break module: batch-job-serialisation · Spring Batch 6 migration guide

Driven by Spring Batch 6.0 (upstream of Spring Boot 4.0). The *ListenerSupport classes were deprecated in Spring Batch 5.0 when the interfaces gained Java 8 default methods. Affects spring-boot-starter-batch consumers. See also: batch-schema-rename.

Spring Batch Core Package Relocations

Core Spring Batch domain classes (Job, JobExecution, etc.) moved from org.springframework.batch.core to org.springframework.batch.core.job.

EFFORT: M

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/BatchCoreUsage.java:[3,44]
  error: package org.springframework.batch.core does not contain class Job
[ERROR] /src/main/java/com/example/BatchCoreUsage.java:[4,44]
  error: package org.springframework.batch.core does not contain class JobExecution
```

WHAT CHANGED

Core domain classes have moved from `org.springframework.batch.core` to subpackages, primarily `org.springframework.batch.core.job`. Affected classes include `Job`, `JobExecution`, `JobInstance`, and `JobParameters` among others.

```
// Core domain imports
import org.springframework.batch.core.Job;
import org.springframework.batch.core.JobExecution;
import org.springframework.batch.core.JobInstance;
import org.springframework.batch.core.JobParameters;

import org.springframework.batch.core.job.Job;
import org.springframework.batch.core.job.JobExecution;
import org.springframework.batch.core.job.JobInstance;
import org.springframework.batch.core.job.JobParameters;
```

WHY THIS CHANGED

Spring Batch 6.0 redesigned the domain model to give each concern its own subpackage, making the module boundaries clearer and aligning with the broader Spring Batch 6.0 architectural goals.

HOW TO FIX

A **Update imports to include the job subpackage.** Add `.job` to the package path for the affected classes. This is a mechanical find-and-replace: `org.springframework.batch.core.Job` becomes `org.springframework.batch.core.job.Job`, and so on. Check the Spring Batch 6.0 Migration Guide for the complete list of moved classes.

✓ **Verify:** mvn compile — no package does not exist errors for org.springframework.batch.core classes

Scope check: Search for `import org.springframework.batch.core.` across all Java/Kotlin sources. Any import that resolves directly to a class in the root `core` package is likely affected. The scope is proportional to how deeply your code interacts with Batch domain objects.

WATCH OUT

Not all classes in `org.springframework.batch.core` moved. Some remain at the root level. Verify each import against the 6.0 Javadoc rather than bulk-replacing all root-level imports.

FURTHER INFO

spring-break module: batch-package-moves · Spring Batch 6.0 Migration Guide

Spring Batch 6.0 restructured its domain model into subpackages under org.springframework.batch.core. The classes still exist; only their package changed. With many classes involved, this can mean a large number of import fixes across any codebase that uses Batch domain types directly.

BootstrapRegistry and EnvironmentPostProcessor Relocated

BootstrapRegistry moved to org.springframework.boot.bootstrap and EnvironmentPostProcessor moved to org.springframework.boot. Both old import paths are gone.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/BootstrapRegistryUsage.java:[3,47]
  error: package org.springframework.boot does not contain BootstrapRegistry
[ERROR] /src/main/java/com/example/EnvironmentPostProcessorUsage.java:[3,43]
  error: package org.springframework.boot.env does not contain EnvironmentPostProcessor
```

WHAT CHANGED

`BootstrapRegistry` and `ConfigurableBootstrapContext` moved from `org.springframework.boot` to `org.springframework.boot.bootstrap`. `EnvironmentPostProcessor` moved in the opposite direction — from `org.springframework.boot.env` to `org.springframework.boot`.

```
// BootstrapRegistry import
import org.springframework.boot.BootstrapRegistry;
import org.springframework.boot.bootstrap.BootstrapRegistry;

// ConfigurableBootstrapContext import
import org.springframework.boot.ConfigurableBootstrapContext;
import org.springframework.boot.bootstrap.ConfigurableBootstrapContext;

// EnvironmentPostProcessor import
import org.springframework.boot.env.EnvironmentPostProcessor;
import org.springframework.boot.EnvironmentPostProcessor;
```

WHY THIS CHANGED

Spring Boot 4.0 reorganised its internal packages to group related infrastructure by concern. The bootstrap lifecycle classes got their own subpackage. EnvironmentPostProcessor, used across many contexts, moved to the root package for easier discoverability.

HOW TO FIX

A **Update imports.** Find all usages of the old package paths and replace with the new ones. Both changes are pure package moves — no method signatures or behaviour changed.

B **Search spring.factories / AutoConfiguration.imports.** If you register an `EnvironmentPostProcessor` in `META-INF/spring.factories` or `META-INF/spring/org.springframework.boot.env.EnvironmentPostProcessor.imports`, update the key to `org.springframework.boot.EnvironmentPostProcessor`.

✓ **Verify:** mvn compile — no package does not exist errors for BootstrapRegistry or EnvironmentPostProcessor

Scope check: Search for `org.springframework.boot.BootstrapRegistry`, `org.springframework.boot.ConfigurableBootstrapContext`, and `org.springframework.boot.env.EnvironmentPostProcessor` across all Java/Kotlin sources and `META-INF` registration files.

WATCH OUT

The `spring.factories` key for `EnvironmentPostProcessor` must also be updated. A wrong key silently skips your processor at startup with no error — your environment customisation just won't run.

FURTHER INFO

spring-break module: bootstrap-registry-relocated · Spring Boot 4.0 Migration Guide

Both classes moved as part of Spring Boot 4.0's package reorganisation. BootstrapRegistry moved deeper into a dedicated bootstrap subpackage. EnvironmentPostProcessor moved in the opposite direction — up to the root boot package. The change affects any code that customises early application startup or environment configuration.

@ConfigurationProperties Field Binding Removed

Spring Boot 4.0 no longer binds @ConfigurationProperties to public fields. Only JavaBean setters or constructor binding work now.

EFFORT: M

WHAT YOU'LL SEE

```
$ mvn spring-boot:run
***************************
APPLICATION FAILED TO START
***************************
Description:
Failed to bind properties under 'app.mail' to com.example.config.MailProperties:

    Property: app.mail.host
    Value: "smtp.example.com"
    Reason: No setter found for property 'host' on class
            com.example.config.MailProperties. Public field binding
            is no longer supported. Add a setter method or use
            constructor binding.
```

WHAT CHANGED

Spring Boot 4.0 removed support for binding `@ConfigurationProperties` values to public fields. The property binding mechanism now requires either JavaBean-style setter methods or constructor binding (via `@ConstructorBinding` or a single constructor). Classes that relied on direct public field access fail at startup.

```
// ConfigurationProperties class — field binding (broken)
@ConfigurationProperties(prefix = "app.mail")
public class MailProperties {
    public String host;
    public int port;
    public boolean ssl;
}

@ConfigurationProperties(prefix = "app.mail")
public class MailProperties {
    private String host;
    private int port;
    private boolean ssl;

    public String getHost() { return host; }
    public void setHost(String host) { this.host = host; }
    public int getPort() { return port; }
    public void setPort(int port) { this.port = port; }
    public boolean isSsl() { return ssl; }
    public void setSsl(boolean ssl) { this.ssl = ssl; }
}


// Or use constructor binding (immutable, recommended)
@ConfigurationProperties(prefix = "app.mail")
public class MailProperties {
    public String host;
    public int port;
    public boolean ssl;
}

@ConfigurationProperties(prefix = "app.mail")
public record MailProperties(String host, int port, boolean ssl) {
}
```

WHY THIS CHANGED

Public field binding bypassed validation, made immutability impossible, and broke encapsulation. Setter-based and constructor-based binding both support `@Validated` constraints and are consistent with the rest of Spring's binding model.

HOW TO FIX

1. **Add getter/setter methods.** Convert public fields to private fields with standard JavaBean getter and setter methods. Your IDE can generate these automatically.

B **Use a Java record (recommended).** Convert the class to a `record`. Records use constructor binding automatically and are immutable by design. This is the cleanest option for configuration properties.

C **Use Lombok @Data.** If you prefer fewer boilerplate lines, annotate the class with `@Data` or `@Getter @Setter` from Lombok.

✓ **Verify:** Config props bind correctly — no BindException on startup

Scope check: Search for `@ConfigurationProperties` classes and check if any have public fields without setters. Look for the pattern `public String`, `public int`, etc. in those classes.

WATCH OUT

This is a startup failure, not a compile error. Your code will compile fine — it fails when Spring Boot tries to bind properties at runtime. If you have good test coverage with `@SpringBootTest`, you'll catch it in tests. Otherwise it hits production.

If you switch to records, remember that records are immutable. Code that previously mutated the properties object after injection will break. That code was probably wrong anyway, but review it.

FURTHER INFO

Spring-Break Demo · Spring Boot 4.0 Migration Guide

Introduced in Spring Boot 4.0. Field binding was never officially recommended but was silently supported since Boot 1.x. Constructor binding (added in Boot 2.2, improved in 3.0) and Java records are the preferred alternatives. Affects all spring-boot-starter consumers that use @ConfigurationProperties.

Elasticsearch RestClient Renamed to Rest5Client

Spring Boot 4.0 auto-configures Rest5Client instead of RestClient for Elasticsearch. Code injecting the old type won't compile.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/ElasticsearchUsage.java:[5,35]
  error: cannot find symbol
    symbol:   class RestClient
    location: package org.elasticsearch.client
```

WHAT CHANGED

Spring Boot 4.0 auto-configures `org.elasticsearch.client.Rest5Client` instead of `org.elasticsearch.client.RestClient`. Components that inject `RestClient` directly will fail because the auto-configured bean is now of type `Rest5Client`.

```
// Import
~~import org.elasticsearch.client.RestClient;~~
import org.elasticsearch.client.Rest5Client;

// Injection
~~@Autowired~~
~~private RestClient restClient;~~
@Autowired
private Rest5Client restClient;

// Customizer bean (if used)
~~@Bean~~
~~public RestClientBuilderCustomizer customizer() { ... }~~
@Bean
public Rest5ClientBuilderCustomizer customizer() { ... }
```

WHY THIS CHANGED

The Elasticsearch Java client library introduced `Rest5Client` as the successor to the original `RestClient`. Spring Boot 4.0 adopted the newer type to track the upstream library and align with current Elasticsearch client best practices.

HOW TO FIX

A **Rename RestClient to Rest5Client.** Replace `RestClient` with `Rest5Client` in all injection points, customizer beans, and direct usages. The API is broadly compatible — method signatures are similar but check the Elasticsearch client changelog for any subtle differences.

✓ **Verify:** mvn compile — no cannot find symbol errors for RestClient auto-config injection

Scope check: Search for `org.elasticsearch.client.RestClient` and `RestClientBuilderCustomizer` in your Java sources. Any direct injection of the low-level client is affected.

WATCH OUT

Do not confuse Elasticsearch's `RestClient` with Spring Framework's `org.springframework.web.client.RestClient` — these are entirely different classes. Only the Elasticsearch one changed.

FURTHER INFO

spring-break module: elasticsearch-rest5client · Spring Boot 4.0 Migration Guide

Spring Boot 4.0 aligns with the Elasticsearch client library's own evolution. Rest5Client is the new low-level client type in Spring Data Elasticsearch. The high-level RestHighLevelClient was already removed in Spring Boot 3.x; this is the next step in that same direction.

@EntityScan Relocated to persistence.autoconfigure

@EntityScan moved from org.springframework.boot.autoconfigure.domain to org.springframework.boot.persistence.autoconfigure. Old import doesn't compile.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/EntityScanApp.java:[3,52]
  error: package org.springframework.boot.autoconfigure.domain does not exist
```

WHAT CHANGED

`@EntityScan` moved from `org.springframework.boot.autoconfigure.domain` to `org.springframework.boot.persistence.autoconfigure`. The annotation's behaviour is unchanged — only the package differs.

```
// Import
~~import org.springframework.boot.autoconfigure.domain.EntityScan;~~
import org.springframework.boot.persistence.autoconfigure.EntityScan;
```

WHY THIS CHANGED

Spring Boot 4.0 extracted persistence-related auto-configuration into a dedicated `spring-boot-persistence` module. This gives persistence concerns a cleaner home and makes the module boundaries explicit.

HOW TO FIX

A **Update the import.** Replace `org.springframework.boot.autoconfigure.domain.EntityScan` with `org.springframework.boot.persistence.autoconfigure.EntityScan`. One import change per file — no other modifications needed.

✓ **Verify:** mvn compile — no package does not exist error for @EntityScan import

Scope check: Search for `org.springframework.boot.autoconfigure.domain.EntityScan` across all Java/Kotlin sources. Typically appears on main application classes or dedicated JPA configuration classes.

WATCH OUT

Other classes from `org.springframework.boot.autoconfigure.domain` may have moved too. If you import anything else from that package, check the Spring Boot 4.0 Migration Guide for the new location.

FURTHER INFO

spring-break module: entityscan-relocated · Spring Boot 4.0 Migration Guide

Part of Spring Boot 4.0's modularisation of persistence-related code into a dedicated spring-boot-persistence module. Several JPA autoconfiguration classes moved together. EntityScan is the most commonly used of the set.

GraalVM 25 for Native Image Builds

Spring Boot 4.0 requires GraalVM 25 for native image compilation. Older GraalVM versions produce build failures or runtime crashes.

EFFORT: M

WHAT YOU'LL SEE

```
$ mvn -Pnative native:compile
[ERROR] Error: Unsupported native-image version: 22.3.5
[ERROR] Spring Boot 4.0 requires GraalVM 25.0 or later.
[ERROR] Please upgrade your GraalVM installation.
[ERROR]
[ERROR] -> [Help 1]
[ERROR] org.graalvm.buildtools:native-maven-plugin:0.10.6:compile failed
```

WHAT CHANGED

Spring Boot 4.0 targets GraalVM 25 (based on JDK 25) for native image builds. The `native-maven-plugin` and `org.graalvm.buildtools.native` Gradle plugin must be updated to version 0.10.6+. Older GraalVM versions (22.x, 23.x) are no longer compatible with the reachability metadata shipped in starters.

```
// pom.xml — native build tools plugin
<plugin>
    <groupId>org.graalvm.buildtools</groupId>
    <artifactId>native-maven-plugin</artifactId>
    <version>0.9.28</version>
</plugin>

<plugin>
    <groupId>org.graalvm.buildtools</groupId>
    <artifactId>native-maven-plugin</artifactId>
    <version>0.10.6</version>
</plugin>


// build.gradle — GraalVM plugin
plugins {
    id 'org.graalvm.buildtools.native' version '0.9.28'
}

plugins {
    id 'org.graalvm.buildtools.native' version '0.10.6'
}


// JAVA_HOME / GRAALVM_HOME
export GRAALVM_HOME=/opt/graalvm-ce-java17-22.3.5
export GRAALVM_HOME=/opt/graalvm-jdk-25+35
```

WHY THIS CHANGED

GraalVM 25 introduced a new metadata format and improved closed-world analysis. Spring's reachability metadata was rewritten to use these features, making it incompatible with older native-image compilers.

HOW TO FIX

A **Install GraalVM 25.** Download GraalVM 25 from graalvm.org and set `GRAALVM_HOME`. If using SDKMAN: `sdk install java 25-graal`.

B **Update native build tools.** Bump the `native-maven-plugin` or Gradle `org.graalvm.buildtools.native` plugin to 0.10.6+. The parent POM manages this if you inherit from `spring-boot-starter-parent`.

✓ **Verify:** native-image --version shows 25+ and mvn -Pnative package succeeds

Scope check: Search for `native-maven-plugin` and `org.graalvm.buildtools.native` in build files. Check CI pipelines for `GRAALVM_HOME` or `native-image` version references. If you don't build native images, this card doesn't apply.

WATCH OUT

GraalVM 25 is based on JDK 25, but the native image it produces still runs on any compatible OS. Don't confuse the build-time JDK requirement with the runtime requirement.

If you use custom reachability metadata in `META-INF/native-image/`, verify it still works. The metadata format added new fields in GraalVM 25 and some older entries may be silently ignored.

FURTHER INFO

Spring Boot 4.0 Migration Guide

Spring Boot 4.0 targets GraalVM 25 (based on JDK 25) and native-maven-plugin 0.10.6+. The reachability metadata format changed in GraalVM 25, making older native-image compilers (22.x, 23.x) incompatible. Only applies to projects building native images via the spring-boot-starter-parent native profile.

Gradle 8.14+ / Gradle 9 Required

Spring Boot 4.0's Gradle plugin requires Gradle 8.14 or later. Older Gradle wrappers fail immediately at configuration time.

EFFORT: S

WHAT YOU'LL SEE

```
$ ./gradlew build
FAILURE: Build failed with an exception.
* Where:
Build file '/project/build.gradle' line: 3
* What went wrong:
An exception occurred applying plugin request [id: 'org.springframework.boot', version: '4.0.0']
> Failed to apply plugin 'org.springframework.boot'.
   > Spring Boot plugin requires Gradle 8.14+. Found: 8.5.
```

WHAT CHANGED

The Spring Boot Gradle plugin now requires Gradle 8.14 as a minimum. Gradle 9.0 is fully supported and recommended for new projects. The plugin uses Gradle APIs introduced in 8.14 that don't exist in earlier versions.

```
// gradle/wrapper/gradle-wrapper.properties
distributionUrl=https\://services.gradle.org/distributions/gradle-8.5-bin.zip
distributionUrl=https\://services.gradle.org/distributions/gradle-8.14-bin.zip

// Alternative: jump to Gradle 9
distributionUrl=https\://services.gradle.org/distributions/gradle-8.5-bin.zip
distributionUrl=https\://services.gradle.org/distributions/gradle-9.0-bin.zip
```

WHY THIS CHANGED

Gradle 8.14 introduced configuration cache stability and improved dependency resolution APIs that the Spring Boot plugin now relies on. Supporting older Gradle versions held back build performance improvements.

HOW TO FIX

A **Update the Gradle wrapper.** Run `./gradlew wrapper --gradle-version 8.14` (or `9.0`) to update the wrapper in place. Commit the updated wrapper files.

B **Check plugin compatibility.** After upgrading Gradle, run a build to verify all other plugins are compatible. The Gradle upgrade guide lists deprecations to address.

✓ **Verify:** ./gradlew --version shows 8.x+ and build succeeds

Scope check: Check `gradle/wrapper/gradle-wrapper.properties` for the current Gradle version. Also check CI caches and any shared Gradle distributions. Multi-module projects only need one wrapper update.

WATCH OUT

If you're on Gradle 7.x, you can't jump straight to 8.14. Gradle deprecation cycles mean you need to fix warnings at each major version boundary. Plan for a staged upgrade: 7.x to 8.0, then 8.0 to 8.14+.

The Gradle configuration cache is now stable and on by default in Gradle 9. If your build scripts use project-level state at execution time, you'll hit configuration cache errors after upgrading.

FURTHER INFO

Spring Boot 4.0 Migration Guide

Spring Boot 4.0's Gradle plugin requires Gradle 8.14 as a minimum; Gradle 9.0 is fully supported. The plugin uses configuration-cache-stable APIs introduced in 8.14. Projects on Gradle 7.x need a staged upgrade through 8.0 first. Does not affect Maven-based builds.

Hibernate EmptyInterceptor Removed

Hibernate's EmptyInterceptor is gone. Implement Interceptor directly — it has default methods for everything you don't need to override.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/MyHibernateInterceptor.java:[5,43]
  error: cannot find symbol
    symbol: class EmptyInterceptor
    location: package org.hibernate
```

WHAT CHANGED

`org.hibernate.EmptyInterceptor` has been removed. The `org.hibernate.Interceptor` interface now has default (no-op) implementations for all its methods, so the abstract base class serves no purpose.

```
// Before — extending EmptyInterceptor
import org.hibernate.EmptyInterceptor;

public class MyInterceptor extends EmptyInterceptor {
    @Override
    public boolean onLoad(Object entity, Object id,
            Object[] state, String[] propertyNames, Type[] types) {
        // custom logic
        return false;
    }
}

None

// After — implementing Interceptor directly
None
import org.hibernate.Interceptor;

public class MyInterceptor implements Interceptor {
    @Override
    public boolean onLoad(Object entity, Object id,
            Object[] state, String[] propertyNames, Type[] types) {
        // custom logic
        return false;
    }
    // All other methods have default no-op implementations
}
```

WHY THIS CHANGED

When Hibernate 6.0 added default methods to `Interceptor`, `EmptyInterceptor` became redundant. Hibernate 7.0 removed it as part of the overall API cleanup.

HOW TO FIX

A **Implement Interceptor directly.** Replace `extends EmptyInterceptor` with `implements Interceptor`. Remove the import for `EmptyInterceptor`. You only need to override the methods you actually use — all others have default no-op implementations on the interface itself.

✓ **Verify:** mvn compile — no cannot find symbol for EmptyInterceptor

Scope check: Search for `EmptyInterceptor` and `extends EmptyInterceptor` across all Java/Kotlin sources.

WATCH OUT

Some method signatures on `Interceptor` changed in Hibernate 6.x (e.g., `Serializable id` became `Object id`). When switching from `extends EmptyInterceptor` to `implements Interceptor`, verify your overridden method signatures match exactly or the override will be silently ignored.

FURTHER INFO

spring-break module: hibernate-empty-interceptor-removed · Hibernate 7.0 Migration Guide

EmptyInterceptor was a convenience class that gave you no-op implementations of all Interceptor methods. Since Hibernate 6.0 the Interceptor interface uses default methods for everything, so EmptyInterceptor became a pointless intermediate. Hibernate 7.0 cleaned it up.

hibernate-jpamodelgen Renamed

Hibernate renamed its annotation processor artifact from hibernate-jpamodelgen to hibernate-processor. Metamodel generation breaks.

EFFORT: S

⚙ OpenRewrite

WHAT YOU'LL SEE

```
$ mvn compile
[WARNING] The POM for org.hibernate.orm:hibernate-jpamodelgen:jar:7.0.0 is missing, no dependency
  information available
[ERROR] Failed to execute goal on project my-service: Could not resolve dependencies:
  Could not find artifact org.hibernate.orm:hibernate-jpamodelgen:jar:7.0.0
  in central (https://repo.maven.apache.org/maven2)
```

WHAT CHANGED

Hibernate 7.0 (shipped with Spring Boot 4.0) renamed the annotation processor artifact from `org.hibernate.orm:hibernate-jpamodelgen` to `org.hibernate.orm:hibernate-processor`. The processor now handles both JPA static metamodel generation and Hibernate's new query validation.

```
// pom.xml — annotation processor dependency
<dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-jpamodelgen</artifactId>
    <scope>provided</scope>
</dependency>

<dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-processor</artifactId>
    <scope>provided</scope>
</dependency>


// build.gradle — annotation processor
annotationProcessor 'org.hibernate.orm:hibernate-jpamodelgen'
annotationProcessor 'org.hibernate.orm:hibernate-processor'

// maven-compiler-plugin annotationProcessorPaths
<path>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-jpamodelgen</artifactId>
</path>

<path>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-processor</artifactId>
</path>
```

WHY THIS CHANGED

The processor does more than JPA metamodel generation now. It also validates HQL/JPQL queries at compile time and generates type-safe query methods. The new name reflects its broader scope.

HOW TO FIX

A **Rename the artifact.** Replace `hibernate-jpamodelgen` with `hibernate-processor` in your POM or Gradle build file. The version is managed by the Spring Boot BOM — just change the artifact ID.

B **Check compiler plugin configuration.** If you declared the processor in `maven-compiler-plugin`'s `annotationProcessorPaths`, update it there too. Both the dependency and the processor path must use the new name.

✓ **Verify:** mvn compile uses hibernate-processor and metamodel generates

Scope check: Search for `hibernate-jpamodelgen` in all POM and Gradle files. Also check `annotationProcessorPaths` in `maven-compiler-plugin` configuration.

WATCH OUT

The new `hibernate-processor` will try to validate your HQL queries at compile time by default. If you have invalid queries that previously only failed at runtime, they'll now fail at build time. This is a feature, but it may surface hidden bugs.

If your IDE configures annotation processors separately (e.g., IntelliJ's annotation processor settings), update that configuration too. Otherwise you'll get metamodel generation in Maven but not in the IDE.

FURTHER INFO

Spring-Break Demo · Hibernate 7 migration guide

Driven by Hibernate 7.0. The hibernate-jpamodelgen artifact was renamed to hibernate-processor to reflect its expanded scope: it now validates HQL/JPQL queries at compile time in addition to generating the JPA static metamodel. Version is managed by the Spring Boot BOM. See also: cascade-save-update, session-delete-removed.

Hibernate SelectionQuery.setOrder() Removed

Hibernate's incubating SelectionQuery.setOrder() was removed in 7.0. Use an ORDER BY clause in HQL instead.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/HibernateQueryUsage.java:[14,15]
  error: cannot find symbol
    symbol:   method setOrder(java.util.List)
    location: interface SelectionQuery<Product>
```

WHAT CHANGED

The `@Incubating` `setOrder(List<Order>)` method on `SelectionQuery` (and `Query`) was removed in Hibernate 7.0. It had been added in Hibernate 6.x as part of Jakarta Data integration work but was never promoted to stable API.

```
// Programmatic order via removed setOrder()
SelectionQuery<Product> query =
    session.createSelectionQuery("from Product", Product.class);
query.setOrder(List.of(Order.asc(Product.class, "name")));

None

// Fix: inline ORDER BY in HQL
None
SelectionQuery<Product> query =
    session.createSelectionQuery("from Product order by name asc", Product.class);
```

WHY THIS CHANGED

The method was always experimental. Hibernate 7.0 completed the Jakarta Data integration under a different, more stable API surface and removed the incubating intermediate.

HOW TO FIX

A **Move ordering into the HQL query string.** Add an `ORDER BY` clause directly to the HQL/JPQL string. This is the most portable approach and works on both 3.5 and 4.0.

B **Use CriteriaQuery with Order.** For fully programmatic ordering, use the JPA Criteria API: `criteriaQuery.orderBy(builder.asc(root.get("name")))`. This is the stable alternative.

✓ **Verify:** mvn compile — no cannot find symbol for setOrder on SelectionQuery

Scope check: Search for `.setOrder(` on query objects across all repository and data-access classes. This is uncommon outside of code written specifically to experiment with the Hibernate 6.x incubating API.

WATCH OUT

`org.hibernate.query.Order` (the parameter type) is also affected. If you imported it, that import can be removed once you stop using `setOrder`.

FURTHER INFO

spring-break module: hibernate-query-setorder-removed · Hibernate 7.0 Migration Guide

setOrder() was introduced in Hibernate 6.x as an @Incubating API for programmatic ordering, intended to support Jakarta Data. It was never stable and was removed in Hibernate 7.0. Any code that adopted it during the 6.x window must now migrate to HQL ORDER BY or the SelectionSpecification API.

Hibernate @Where and @OrderBy Removed

Hibernate's @Where and @OrderBy are removed. Replace with @SQLRestriction and @SQLOrder.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/User.java:[5,39]
  error: cannot find symbol
    symbol: class Where
    location: package org.hibernate.annotations
[ERROR] /src/main/java/com/example/User.java:[6,39]
  error: cannot find symbol
    symbol: class OrderBy
    location: package org.hibernate.annotations
```

WHAT CHANGED

`@org.hibernate.annotations.Where` and `@org.hibernate.annotations.OrderBy` have been removed. The replacements are `@org.hibernate.annotations.SQLRestriction` and `@org.hibernate.annotations.SQLOrder`, which take the same SQL fragment arguments.

```
// Entity annotations
import org.hibernate.annotations.Where;
import org.hibernate.annotations.OrderBy;

@OneToMany
@Where(clause = "deleted = false")
@OrderBy(clause = "name desc")
private List<Post> posts;

import org.hibernate.annotations.SQLRestriction;
import org.hibernate.annotations.SQLOrder;

@OneToMany
@SQLRestriction("deleted = false")
@SQLOrder("name desc")
private List<Post> posts;
```

WHY THIS CHANGED

The new annotation names make it explicit that the argument is a raw SQL fragment, not a JPQL or HQL expression. This avoids confusion with the JPA `@OrderBy` (which takes attribute names) and with Hibernate's HQL-level features.

HOW TO FIX

A **Replace @Where with @SQLRestriction.** Change the annotation name and attribute: `@Where(clause = "...")` becomes `@SQLRestriction("...")`. The SQL fragment is the same.

B **Replace @OrderBy with @SQLOrder.** Change the annotation name and attribute: `@OrderBy(clause = "...")` becomes `@SQLOrder("...")`. Do not confuse with JPA's `@jakarta.persistence.OrderBy`, which is a separate annotation taking attribute names and is unaffected.

✓ **Verify:** mvn compile — no cannot find symbol for @Where or @OrderBy from Hibernate

Scope check: Search for `@Where` and `@OrderBy` from the `org.hibernate.annotations` package across all entity classes and embeddables. Also check `@WhereJoinTable` if used.

WATCH OUT

If you have both `@jakarta.persistence.OrderBy` and `@org.hibernate.annotations.OrderBy` in use, only the Hibernate one needs replacing. The JPA one is untouched.

FURTHER INFO

spring-break module: hibernate-where-orderby-removed · Hibernate 7.0 Migration Guide

@SQLRestriction and @SQLOrder were introduced in Hibernate 6.3 as the designated replacements. The old annotations were deprecated then and removed in Hibernate 7.0. Note: JPA's @jakarta.persistence.OrderBy (which sorts by entity attribute) is unaffected — only Hibernate's SQL-string variants are gone.

HttpClient setSocketTimeout Removed

RequestConfig.Builder.setSocketTimeout(int) is gone in HttpClient 5. Use setResponseTimeout(Timeout) instead.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/HttpComponentsUsage.java:[12,18]
  error: cannot find symbol
    symbol:   method setSocketTimeout(int)
    location: class RequestConfig.Builder
```

WHAT CHANGED

Apache HttpClient 5.x removed the integer-millisecond timeout methods (`setSocketTimeout(int)` , `setConnectTimeout(int)` , `setConnectionRequestTimeout(int)`) and replaced them with overloads that take a strongly-typed `org.apache.hc.core5.util.Timeout` object.

```
// Timeout configuration
RequestConfig.custom()
    .setSocketTimeout(5000)
    .setConnectTimeout(3000)
    .build();

import org.apache.hc.core5.util.Timeout;

RequestConfig.custom()
    .setResponseTimeout(Timeout.ofMilliseconds(5000))
    .setConnectTimeout(Timeout.ofMilliseconds(3000))
    .build();
```

WHY THIS CHANGED

The integer-millisecond API was ambiguous about units and lacked null-safety. `Timeout` is explicit about the time unit, supports nanosecond precision, and can represent "no timeout" cleanly.

HOW TO FIX

A **Replace int timeouts with Timeout objects.** Import `org.apache.hc.core5.util.Timeout` and replace each integer-millisecond call with its `Timeout.ofMilliseconds(n)` equivalent. Also note that `setSocketTimeout` is now called `setResponseTimeout` .

✓ **Verify:** mvn compile — no cannot find symbol for setSocketTimeout

Scope check: Search for `setSocketTimeout` , `setConnectTimeout` , and `setConnectionRequestTimeout` taking integer arguments across all Java/Kotlin sources. Custom HTTP client configuration classes and RestTemplate factory beans are the most common locations.

WATCH OUT

`setSocketTimeout` maps to `setResponseTimeout` in HttpClient 5 — the name changed, not just the type. A search-and-replace on the method name alone will miss this.

FURTHER INFO

spring-break module: httpcomponents-setconnecttimeout-removed · Apache HttpClient 5.0 migration guide

Apache HttpClient 5.x (used in Spring Boot 3.x and 4.0) replaced the integer-based timeout methods of HttpClient 4.x with strongly-typed Timeout objects. This is a pre-existing 4.x→5.x break that surfaces when migrating from Spring Boot 2.x directly to 4.0.

HttpHeaders No Longer Implements MultiValueMap

HttpHeaders no longer implements MultiValueMap. Code that treats headers as a general-purpose map fails to compile.

EFFORT: M

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/HttpHeadersUsage.java:[8,24]
  error: incompatible types: HttpHeaders cannot be converted to MultiValueMap
[ERROR] /src/main/java/com/example/HttpHeadersUsage.java:[12,16]
  error: cannot find symbol
    symbol: method containsKey(String)
```

WHAT CHANGED

`HttpHeaders` no longer implements `MultiValueMap<String, String>` or `Map<String, List<String>>`. Map-style methods like `containsKey()`, `keySet()`, and `entrySet()` are gone from the type. Header-specific methods replace them.

```
// Replace Map method calls with HttpHeaders equivalents
headers.containsKey("Content-Type")
headers.containsHeader("Content-Type")

Set<String> names = headers.keySet();
Set<String> names = headers.headerNames();

Set<Map.Entry<String, List<String>>> entries = headers.entrySet();
Set<Map.Entry<String, List<String>>> entries = headers.headerSet();

// Passing HttpHeaders where MultiValueMap is expected
void process(MultiValueMap<String, String> map) { ... }
process(headers);
void process(HttpHeaders headers) { ... }
// or, if the method signature can't change:
process(headers.toMultiValueMap());
```

WHY THIS CHANGED

Treating HTTP headers as a generic map encouraged misuse and prevented performance optimisations in the header handling implementation. The dedicated `HttpHeaders` API is more expressive and allows the backing store to be optimised independently of the Map contract.

HOW TO FIX

A **Replace Map methods with HttpHeaders methods.** `containsKey(k)` → `containsHeader(k)`, `keySet()` → `headerNames()`, `entrySet()` → `headerSet()`. These replacements compile on both 3.5 and 4.0.

B **For method signatures that accept MultiValueMap.** If you have a utility method that accepts `MultiValueMap<String, String>` and you pass `HttpHeaders` into it, either change the parameter type to `HttpHeaders`, or call `headers.toMultiValueMap()` at the call site.

✓ **Verify:** mvn compile — no incompatible types or cannot find symbol errors on HttpHeaders usage

Scope check: Search for `MultiValueMap` usages near `HttpHeaders` across your codebase. Also search for `.containsKey(`, `.keySet()`, `.entrySet()` called on variables of type `HttpHeaders`.

WATCH OUT

Code in Spring interceptors, filters, and custom converters that inspects headers using Map idioms is the most common source of these failures. Check `HandlerInterceptor`, `ClientHttpRequestInterceptor`, and `ResponseBodyAdvice` implementations.

`toMultiValueMap()` returns a copy, not a live view. Mutations to the returned map do not affect the original `HttpHeaders`.

FURTHER INFO

spring-break module: httpheaders-multivaluemap · Spring Framework 7.0 Release Notes

Driven by Spring Framework 7.0. HttpHeaders was a subtype of MultiValueMap since early in Spring's history. The break affects any code that relies on that structural relationship: method parameters typed as MultiValueMap that receive HttpHeaders, or direct calls to Map methods like containsKey(), keySet(), entrySet() on an HttpHeaders reference.

Jackson Class Renames

Jackson 3.0 renamed core classes like JsonSerializer, JsonDeserializer, and SerializerProvider. Every custom serialiser is now broken.

EFFORT: M

⚙ OpenRewrite

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/DateSerializer.java:[4,44]
  error: cannot find symbol
    symbol:   class JsonSerializer
    location: package tools.jackson.databind
[ERROR] /src/main/java/com/example/DateSerializer.java:[12,47]
  error: cannot find symbol
    symbol:   class SerializerProvider
    location: package tools.jackson.databind
```

WHAT CHANGED

Jackson 3.0 renamed several core classes: `JsonSerializer` became `ValueSerializer`, `JsonDeserializer` became `ValueDeserializer`, `SerializerProvider` became `SerializationContext`, `DeserializationContext` kept its name but moved packages. Method signatures on these classes also changed.

```
// Custom serializer class
import com.fasterxml.jackson.databind.JsonSerializer;
import com.fasterxml.jackson.databind.SerializerProvider;

public class DateSerializer extends JsonSerializer<LocalDate> {
    @Override
    public void serialize(LocalDate value, JsonGenerator gen,
            SerializerProvider provider) throws IOException {

import tools.jackson.databind.ser.ValueSerializer;
import tools.jackson.databind.SerializationContext;

public class DateSerializer extends ValueSerializer<LocalDate> {
    @Override
    public void serialize(LocalDate value, JsonGenerator gen,
            SerializationContext ctxt) throws IOException {


// Custom deserializer class
import com.fasterxml.jackson.databind.JsonDeserializer;

public class DateDeserializer extends JsonDeserializer<LocalDate> {

import tools.jackson.databind.deser.ValueDeserializer;

public class DateDeserializer extends ValueDeserializer<LocalDate> {


// Exception handling
import com.fasterxml.jackson.databind.JsonMappingException;
import tools.jackson.databind.DatabindException;
```

WHY THIS CHANGED

The "Json" prefix was misleading — Jackson handles YAML, XML, CBOR, and other formats. The new names reflect that serialisers work with values, not specifically JSON. The rename also aligned the class hierarchy with the new streaming API.

HOW TO FIX

A **OpenRewrite (recommended).** Run the Jackson 3 type changes recipe. It handles class renames, method signature updates, and import changes in one pass.

B **Manual migration.** Replace class names and imports file by file. The full mapping is in the Jackson 3 migration guide. Don't forget to update method parameter types in overridden methods.

✓ **Verify:** mvn compile — no JsonSerializer/SerializerProvider errors

Scope check: Search for `extends JsonSerializer`, `extends JsonDeserializer`, `SerializerProvider`, and `JsonMappingException`. Every custom serialiser, deserialiser, and exception handler needs updating.

WATCH OUT

The method signatures changed too, not just the class names. `serialize()` now takes a `SerializationContext` instead of `SerializerProvider`. If you only rename the class but keep the old parameter type, you'll get an abstract method error instead of a clean override.

Annotations like `@JsonSerialize(using = ...)` still work but must reference the new class names. A custom serialiser registered via annotation will fail at runtime if it extends the wrong base class.

FURTHER INFO

spring-break module: deprecated-classes-removed · Jackson 3 migration guide

Driven by Jackson 3.0 (upstream of Spring Boot 4.0). JsonSerializer became ValueSerializer, SerializerProvider became SerializationContext, and JsonDeserializer became ValueDeserializer. The rename was finalised in Jackson 3.0-rc1. Affects all spring-boot-starter-json consumers with custom serialisers. See also: jackson-group-id, jackson-exception-hierarchy.

Jackson 3.0 Group ID & Class Renames

Jackson's Maven coordinates and core class names changed. Every file that imports Jackson is now wrong.

EFFORT: L ⚙ OpenRewrite

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/MySerializer.java:[3,39]
  package com.fasterxml.jackson.databind does not exist
[ERROR] /src/main/java/com/example/MySerializer.java:[12,20]
  cannot find symbol
    symbol: class JsonSerializer
```

WHAT CHANGED

Jackson 3.0 moved from `com.fasterxml.jackson` to `tools.jackson`. The group ID, packages, and several core classes all renamed in a single release.

```
// Maven dependency
com.fasterxml.jackson.core:jackson-databind
tools.jackson.core:jackson-databind

// Imports
import com.fasterxml.jackson.databind.JsonSerializer;
import tools.jackson.databind.ser.ValueSerializer;

import com.fasterxml.jackson.databind.SerializerProvider;
import tools.jackson.databind.SerializationContext;

import com.fasterxml.jackson.databind.JsonMappingException;
import tools.jackson.databind.DatabindException;
```

WHY THIS CHANGED

Jackson's maintainers wanted to separate the project identity from the original FasterXML organisation. The rename also gave them a clean break to drop deprecated APIs and simplify the class hierarchy.

HOW TO FIX

A **OpenRewrite (recommended).** Run the Jackson 3 migration recipe. Handles group ID, imports, and class renames across the entire codebase in one pass.

B **Manual find-and-replace.** Update the Maven coordinates first, then fix imports file by file. You need both steps: the group ID change alone won't resolve the class renames.

✓ **Verify:** mvn compile succeeds with no com.fasterxml errors

Scope check: Search your codebase for `com.fasterxml.jackson`. Every hit is an import that needs updating. Custom serialisers, deserialisers, `ObjectMapper` configuration, and DTO annotations are all affected. If you have 5 files that touch Jackson, expect 5 files to change. If you have 200, expect 200.

WATCH OUT

The class renames aren't just package moves. `JsonSerializer` became `ValueSerializer`, `SerializerProvider` became `SerializationContext`. A naive search-and-replace of the package prefix will still leave broken class references. Fix both in one pass.

Jackson exceptions (`JsonMappingException`, `JsonProcessingException`) are `Serializable`. If you've persisted these anywhere (dead letter queues, error logs), those serialised streams contain the old class names. That's a separate Tier 2 problem.

FURTHER INFO

spring-break module: jackson-group-id · Jackson 3 migration guide

Driven by Jackson 3.0 (upstream of Spring Boot 4.0). The FasterXML to tools.jackson rename was announced in 2023 and finalised in Jackson 3.0-rc1. Affects all spring-boot-starter-json consumers. See also: jackson-class-renames, jackson-exception-hierarchy.

Java 17 Minimum Requirement

Spring Boot 4.0 requires Java 17 as the minimum language level, with 21 recommended. Projects still on Java 11 or earlier won't compile at all.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.13.0:compile
  on project my-service: Fatal error compiling: error: release version 17 not supported
[ERROR]
[ERROR] → [Help 1] Check that your JAVA_HOME points to a JDK 17+ installation.
```

WHAT CHANGED

Spring Boot 4.0 sets `java.version` to `17` as the baseline. The framework bytecode and all starters are compiled at the Java 17 language level. Java 11 and earlier are no longer supported.

```
// pom.xml — Java version property
<java.version>11</java.version>
<java.version>17</java.version>

// build.gradle — toolchain
java {
    sourceCompatibility = JavaVersion.VERSION_11
}

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(17)
    }
}


// Dockerfile
FROM eclipse-temurin:11-jre
FROM eclipse-temurin:17-jre
```

WHY THIS CHANGED

Java 17 is the oldest LTS release still receiving public updates. Targeting it lets the framework use sealed classes, records, pattern matching, and text blocks throughout the codebase.

HOW TO FIX

A **Upgrade your JDK.** Install JDK 17 or (recommended) JDK 21 and set `JAVA_HOME` accordingly. Update `java.version` in your POM or `toolchain` block in Gradle.

B **Update CI and container base images.** Change your CI pipeline and Dockerfiles to use a 17+ base image. `eclipse-temurin:21-jre` is a good default.

✓ **Verify:** java -version shows 17+ and mvn compile succeeds

Scope check: Check `java.version` in your POM, `sourceCompatibility` in build.gradle, `JAVA_HOME` in CI scripts, and base images in Dockerfiles. Any reference to Java 8, 11, or earlier needs updating.

WATCH OUT

Java 17 is the minimum, but 21 is recommended. If you're going to upgrade anyway, go straight to 21 to avoid a second migration. Virtual threads and the latest GC improvements are worth the jump.

Third-party libraries compiled against Java 8 or 11 will still work on a Java 17+ runtime. The issue is your own source code and build toolchain, not your transitive dependencies.

FURTHER INFO

Spring Boot 4.0 Migration Guide

Spring Boot 4.0 requires Java 17 as the minimum (Spring Framework 7.0 baseline). Java 21 is recommended. The framework uses sealed classes, records, and pattern matching internally. Java 11 support was dropped in Spring Boot 3.0; this change raises the floor from 17 (Boot 3.x) to 17 as a hard minimum with 21 recommended.

javax.inject Removed

Spring Boot 4.0 no longer provides javax.inject on the classpath. Code using @Inject or @Named from javax.inject won't compile.

EFFORT: S

⚙ OpenRewrite

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/service/OrderService.java:[3,20]
  error: package javax.inject does not exist
[ERROR] /src/main/java/com/example/service/OrderService.java:[12,6]
  error: cannot find symbol
    symbol: class Inject
```

WHAT CHANGED

The `javax.inject:javax.inject` dependency is no longer on the Spring Boot classpath. Annotations like `@javax.inject.Inject`, `@javax.inject.Named`, and `@javax.inject.Singleton` are unavailable. The Jakarta equivalent is `jakarta.inject:jakarta.inject-api`.

```
// Java imports
import javax.inject.Inject;
import javax.inject.Named;

import jakarta.inject.Inject;
import jakarta.inject.Named;


// pom.xml — if you had an explicit dependency
<dependency>
    <groupId>javax.inject</groupId>
    <artifactId>javax.inject</artifactId>
    <version>1</version>
</dependency>

<dependency>
    <groupId>jakarta.inject</groupId>
    <artifactId>jakarta.inject-api</artifactId>
</dependency>


// Or just use Spring annotations
@Named("orderService")
public class OrderService {
    @Inject
    private OrderRepository repo;

@Service("orderService")
public class OrderService {
    @Autowired
    private OrderRepository repo;
```

WHY THIS CHANGED

The Jakarta EE transition moved all `javax.*` APIs to `jakarta.*`. Spring Framework 7.0 completed this transition by dropping support for the legacy `javax.inject` artifact. Only `jakarta.inject` is supported.

HOW TO FIX

A **Switch to jakarta.inject.** Replace `javax.inject` imports with `jakarta.inject`. The annotations are identical — only the package name changed. The `jakarta.inject-api` artifact is managed by the Spring Boot BOM.

B **Switch to Spring annotations (recommended).** Replace `@Inject` with `@Autowired` and `@Named` with `@Component` / `@Service` / `@Qualifier`. This removes the dependency on the inject API entirely.

✓ **Verify:** mvn compile — no javax.inject import errors

Scope check: Search for `javax.inject` in all Java/Kotlin sources and build files. Every `import javax.inject` statement needs updating. Also check for `javax.inject:javax.inject` in POM files.

WATCH OUT

If you're using `@Inject` for CDI compatibility (e.g., shared code between Spring and a Jakarta EE server), switch to `jakarta.inject` — it works in both environments. Don't switch to Spring-specific annotations if portability matters.

Constructor injection with `@Inject` on a single-constructor class can simply remove the annotation entirely. Spring auto-detects single-constructor injection since 4.3.

FURTHER INFO

spring-break module: javax-annotation-removed · Spring Boot 4.0 Migration Guide

Driven by the Jakarta EE transition completed in Spring Framework 7.0. The javax.inject:javax.inject:1 artifact was a JSR-330 reference implementation. Its jakarta.inject:jakarta.inject-api replacement is managed by the Spring Boot BOM. Deprecated since the javax-to-jakarta migration in Boot 3.0.

Kafka StreamsBuilderFactoryBeanCustomizer Removed

Spring Boot's StreamsBuilderFactoryBeanCustomizer is gone. Replace it with Spring Kafka's StreamsBuilderFactoryBeanConfigurer.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/KafkaCustomizerUsage.java:[3,60]
  error: package org.springframework.boot.autoconfigure.kafka does not contain
  StreamsBuilderFactoryBeanCustomizer
```

WHAT CHANGED

`org.springframework.boot.autoconfigure.kafka.StreamsBuilderFactoryBeanCustomizer` has been removed. The replacement is `org.springframework.kafka.config.StreamsBuilderFactoryBeanConfigurer` from Spring Kafka itself.

```
// Import
~~import org.springframework.boot.autoconfigure.kafka.StreamsBuilderFactoryBeanCustomizer;~~
import org.springframework.kafka.config.StreamsBuilderFactoryBeanConfigurer;

// Bean method signature
~~@Bean~~
~~public StreamsBuilderFactoryBeanCustomizer customizer() { ... }~~
@Bean
public StreamsBuilderFactoryBeanConfigurer configurer() { ... }

// implements clause
~~public class MyCustomizer implements StreamsBuilderFactoryBeanCustomizer {~~
public class MyCustomizer implements StreamsBuilderFactoryBeanConfigurer {
```

WHY THIS CHANGED

Spring Boot's interface was a duplicate of an equivalent already provided by Spring Kafka. Removing the Boot-specific version eliminates redundancy and removes an unnecessary indirection.

HOW TO FIX

A **Switch to StreamsBuilderFactoryBeanConfigurer.** Replace the import and the implements / return type declaration. The interface method signature is compatible — the method you override is `configure(StreamsBuilderFactoryBean)` in both cases.

✓ **Verify:** mvn compile — no cannot find symbol for StreamsBuilderFactoryBeanCustomizer

Scope check: Search for `StreamsBuilderFactoryBeanCustomizer` in all Java/Kotlin sources. Only Kafka Streams applications are affected.

WATCH OUT

The method name on `StreamsBuilderFactoryBeanConfigurer` is `configure`, same as the old `customize`. Verify the exact method name in the Spring Kafka version you're using — it differs between minor versions.

FURTHER INFO

spring-break module: kafka-streams-customizer-removed · Spring Boot 4.0 Migration Guide

Spring Boot's StreamsBuilderFactoryBeanCustomizer was always a thin wrapper around Spring Kafka's own configurer interface. Spring Boot 4.0 removed the duplicate and directs users to the upstream interface directly, reducing the surface area of Boot's Kafka autoconfiguration.

ListenableFuture Removed

Spring Framework removed ListenableFuture entirely. All async code using it must switch to CompletableFuture.

EFFORT: M

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/service/AsyncService.java:[4,49]
  error: cannot find symbol
    symbol:   class ListenableFuture
    location: package org.springframework.util.concurrent
[ERROR] /src/main/java/com/example/service/AsyncService.java:[5,49]
  error: cannot find symbol
    symbol:   class ListenableFutureCallback
    location: package org.springframework.util.concurrent
```

WHAT CHANGED

`ListenableFuture`, `ListenableFutureCallback`, `ListenableFutureTask`, and related classes in `org.springframework.util.concurrent` were deleted from Spring Framework 7.0. All Spring APIs that previously returned `ListenableFuture` now return `java.util.concurrent.CompletableFuture`.

```
// Async method return type
import org.springframework.util.concurrent.ListenableFuture;

@Async
public ListenableFuture<String> fetchData() {
    return AsyncResult.forValue(doWork());
}

import java.util.concurrent.CompletableFuture;

@Async
public CompletableFuture<String> fetchData() {
    return CompletableFuture.completedFuture(doWork());
}


// Callback-based usage
ListenableFuture<String> future = asyncService.fetchData();
future.addCallback(
    result -> log.info("Success: {}", result),
    ex -> log.error("Failed", ex)
);

CompletableFuture<String> future = asyncService.fetchData();
future.whenComplete((result, ex) -> {
    if (ex != null) log.error("Failed", ex);
    else log.info("Success: {}", result);
});
```

WHY THIS CHANGED

`ListenableFuture` predated Java 8. Once `CompletableFuture` arrived in the JDK, maintaining a parallel abstraction added complexity with no benefit. The migration was staged over several releases with deprecation warnings.

HOW TO FIX

A **Replace with CompletableFuture.** Change return types from `ListenableFuture<T>` to `CompletableFuture<T>`. Replace `AsyncResult.forValue()` with `CompletableFuture.completedFuture()`. Replace `addCallback()` with `whenComplete()` or `thenAccept()`.

B **Use OpenRewrite.** The `org.openrewrite.java.spring.framework.MigrateSpringAssert` recipe set includes `ListenableFuture` migration.

✓ **Verify:** mvn compile — no ListenableFuture symbol errors

Scope check: Search for `ListenableFuture`, `ListenableFutureCallback`, and `AsyncResult`. Also check for `addCallback(` calls — these are the usage sites that need rewriting even if the declaration is in a library.

WATCH OUT

`AsyncResult` was also removed. If you used `new AsyncResult<>(value)` or `AsyncResult.forValue()`, both must change to `CompletableFuture.completedFuture()`.

If you used `ListenableFuture` in a messaging listener (e.g., Spring Kafka's `KafkaTemplate.send()`), the return type changed upstream too. Check your Kafka and AMQP send-and-confirm patterns.

FURTHER INFO

Spring-Break Demo · Spring Framework ListenableFuture removal

ListenableFuture was deprecated in Spring Framework 6.0 and removed in 7.0 (tracked in spring-framework#33808). CompletableFuture has been the recommended alternative since Java 8. Also affects Spring Kafka and Spring AMQP return types that previously used ListenableFuture.

Maven Plugin Version Alignment for AOT

The Spring Boot Maven plugin's AOT processing goals changed in 4.0. Mismatched plugin versions or stale AOT configuration breaks the build.

EFFORT: S

⚙ OpenRewrite

WHAT YOU'LL SEE

```
$ mvn spring-boot:process-aot
[ERROR] Failed to execute goal org.springframework.boot:spring-boot-maven-plugin:4.0.0:process-aot
  on project my-service: Execution default of goal
  org.springframework.boot:spring-boot-maven-plugin:4.0.0:process-aot failed:
  An API incompatibility was encountered while executing
  org.springframework.boot:spring-boot-maven-plugin:4.0.0:process-aot:
  java.lang.NoSuchMethodError:
    'void org.springframework.aot.generate.GenerationContext.<init>(java.lang.ClassLoader)'
```

WHAT CHANGED

The `spring-boot-maven-plugin` must match the Spring Boot version exactly. In 4.0 the AOT processing API changed — the `process-aot` and `process-test-aot` goals use new generation context methods. A plugin version mismatch causes `NoSuchMethodError` at build time.

```
// pom.xml — plugin version
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <version>3.4.1</version>
</plugin>

<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <version>4.0.0</version>
</plugin>


// pom.xml — remove explicit AOT execution if using parent POM
<execution>
    <id>process-aot</id>
    <goals><goal>process-aot</goal></goals>
</execution>

<!— AOT goals are now configured automatically by the parent POM —>
```

WHY THIS CHANGED

AOT processing was refactored for GraalVM 25 compatibility and to support the new Spring Framework 7.0 code generation model. The plugin version must be kept in lock-step with the framework.

HOW TO FIX

A **Align the plugin version with Spring Boot.** If you inherit from `spring-boot-starter-parent`, the plugin version is managed automatically — remove any explicit `<version>` tag. If you use a BOM without the parent POM, set the plugin version to `4.0.0` explicitly.

B **Remove manual AOT execution blocks.** The parent POM now binds the AOT goals automatically. Keeping an explicit `<execution>` block can cause the goal to run twice or with stale configuration.

✓ **Verify:** mvn spring-boot:aot-generate runs without plugin errors

Scope check: Search for `spring-boot-maven-plugin` across all POM files. Check for explicit `<version>` tags and manual AOT `<execution>` blocks. Multi-module projects need every module's POM checked.

WATCH OUT

If you're not using native images, AOT processing is still enabled by default in 4.0 for startup optimisation. You may hit this error even if you never intentionally configured AOT.

Parent POM version and plugin version must match. If your `<parent>` points to 4.0.0 but a plugin override still says 3.x, the build will fail with cryptic reflection errors.

FURTHER INFO

Spring Boot 4.0 Migration Guide

The spring-boot-maven-plugin must exactly match the Spring Boot version. In 4.0 the AOT processing API was refactored for GraalVM 25 compatibility and Spring Framework 7.0's code generation model. AOT is enabled by default in Boot 4.0 even for non-native builds. See also: graalvm-25.

OkHttp3 Support Removed

Spring Boot 4.0 dropped the OkHttp3 client factory and BOM entry. Projects using OkHttp3RestClient or the managed dependency won't compile.

EFFORT: M

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/config/RestConfig.java:[5,48]
  package org.springframework.http.client.OkHttp3ClientHttpRequestFactory does not exist
[ERROR] /src/main/java/com/example/config/RestConfig.java:[14,16]
  cannot find symbol
    symbol:   class OkHttp3ClientHttpRequestFactory
    location: class com.example.config.RestConfig
```

WHAT CHANGED

The `OkHttp3ClientHttpRequestFactory` class was removed from Spring Framework 7.0. Spring Boot 4.0 also removed `com.squareup.okhttp3` from its managed dependency BOM. Any code creating a `RestClient` or `RestTemplate` with the OkHttp3 factory won't compile.

```
// RestClient configuration
import org.springframework.http.client.OkHttp3ClientHttpRequestFactory;

@Bean
public RestClient restClient() {
    return RestClient.builder()
        .requestFactory(new OkHttp3ClientHttpRequestFactory())
        .build();
}

import org.springframework.http.client.JdkClientHttpRequestFactory;

@Bean
public RestClient restClient() {
    return RestClient.builder()
        .requestFactory(new JdkClientHttpRequestFactory())
        .build();
}


// pom.xml — remove OkHttp dependency
<dependency>
    <groupId>com.squareup.okhttp3</groupId>
    <artifactId>okhttp</artifactId>
</dependency>

<!— JdkClientHttpRequestFactory uses java.net.http — no extra dependency needed —>
```

WHY THIS CHANGED

The JDK's built-in `java.net.http.HttpClient` (since Java 11) and the Apache HttpClient 5 cover the same use cases. Maintaining a third HTTP client integration added complexity without clear benefit.

HOW TO FIX

A **Switch to JdkClientHttpRequestFactory (simplest).** Replace `OkHttp3ClientHttpRequestFactory` with `JdkClientHttpRequestFactory`. It uses the JDK's built-in HTTP client and requires no additional dependencies.

B **Switch to Apache HttpClient 5.** If you need connection pooling or advanced proxy configuration, use `HttpComponentsClientHttpRequestFactory` with Apache HttpClient 5. Add `org.apache.httpcomponents.client5:httpclient5` to your POM.

✓ **Verify:** mvn dependency:tree shows no okhttp3 and build compiles

Scope check: Search for `OkHttp3` and `com.squareup.okhttp3` across your codebase and build files. Check both production code and test configurations — OkHttp's `MockWebServer` is also affected.

WATCH OUT

If you used OkHttp interceptors for logging, retries, or authentication, those don't have a direct equivalent in the JDK client. You'll need to reimplement that logic using Spring's `ClientHttpRequestInterceptor` instead.

OkHttp's `MockWebServer` is popular in tests. If you only use OkHttp for testing, consider switching to `MockRestServiceServer` or WireMock instead.

FURTHER INFO

Spring-Break Demo · Spring Boot 4.0 Migration Guide

OkHttp3ClientHttpRequestFactory was removed from Spring Framework 7.0 and the com.squareup.okhttp3 BOM entry was dropped from Spring Boot 4.0. The JDK's java.net.http.HttpClient (since Java 11) and Apache HttpClient 5 are the supported alternatives. Affects spring-boot-starter-web consumers using OkHttp.

PropertyMapper.alwaysApplyingWhenNonNull() Removed

PropertyMapper.alwaysApplyingWhenNonNull() is removed because skipping null values is now the default. Just delete the call.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/PropertyMapperUsage.java:[8,47]
  error: cannot find symbol
    symbol:   method alwaysApplyingWhenNonNull()
    location: class PropertyMapper
```

WHAT CHANGED

`PropertyMapper.alwaysApplyingWhenNonNull()` has been removed because its behaviour — skipping mappings when the source value is `null` — is now the default behaviour of `PropertyMapper`. Calling it was a no-op from 4.0 onward, so the method was removed.

```
// Remove the method call — the behaviour is now default
PropertyMapper map = PropertyMapper.get().alwaysApplyingWhenNonNull();
PropertyMapper map = PropertyMapper.get();
```

WHY THIS CHANGED

The non-null default was the right behaviour for virtually all use cases. Requiring an explicit opt-in call was boilerplate with no practical reason to exist. Making it the default and removing the method simplifies the API.

HOW TO FIX

A **Delete the alwaysApplyingWhenNonNull() call.** Simply remove `.alwaysApplyingWhenNonNull()` from the chain. The mapper behaves identically on Boot 4.0 without it. If you explicitly want to map null values, use `.always()` on individual mappings.

✓ **Verify:** mvn compile — no cannot find symbol for alwaysApplyingWhenNonNull

Scope check: Search for `alwaysApplyingWhenNonNull` across all Java/Kotlin sources. This typically appears in autoconfiguration classes or custom configuration adapters that mirror Spring Boot's internal style.

WATCH OUT

If you were relying on the absence of `alwaysApplyingWhenNonNull()` to intentionally map null values through, that behaviour has also changed — nulls are now skipped by default. Add `.always()` on the specific mappings that need to pass nulls.

FURTHER INFO

spring-break module: propertymapper-alwaysapplyingnonnull · Spring Boot 4.0 Migration Guide

PropertyMapper is an internal Spring Boot utility used mostly in autoconfiguration classes to map configuration properties onto builder objects. alwaysApplyingWhenNonNull() was the opt-in way to skip null values. In Spring Boot 4.0 that behaviour became the default, making the method redundant.

@PropertyMapping Relocated to test.context

@PropertyMapping moved from org.springframework.boot.test.autoconfigure.properties to org.springframework.boot.test.context. Old import doesn't compile.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn test-compile
[ERROR] /src/test/java/com/example/CustomTestAnnotation.java:[3,62]
  error: package org.springframework.boot.test.autoconfigure.properties does not exist
```

WHAT CHANGED

`@PropertyMapping` moved from `org.springframework.boot.test.autoconfigure.properties` to `org.springframework.boot.test.context`. Behaviour is unchanged.

```
// Import on custom test annotation
~~import org.springframework.boot.test.autoconfigure.properties.PropertyMapping;~~
import org.springframework.boot.test.context.PropertyMapping;
```

WHY THIS CHANGED

Spring Boot 4.0 consolidated test-context infrastructure into a more coherent package hierarchy. The separate `test.autoconfigure.properties` package was collapsed into `test.context`.

HOW TO FIX

A **Update the import.** Replace `org.springframework.boot.test.autoconfigure.properties.PropertyMapping` with `org.springframework.boot.test.context.PropertyMapping`. This typically appears only in custom meta-annotations for test slices.

✓ **Verify:** mvn test-compile — no package does not exist error for @PropertyMapping import

Scope check: Search for `@PropertyMapping` and the old import path in your test source tree. Only affects teams that have written custom test slice annotations — most applications won't have this at all.

WATCH OUT

If you have a shared test-utilities library that defines custom slices, the fix must be applied there rather than in the consuming projects. A single unfixed library breaks all consumers.

FURTHER INFO

spring-break module: propertymapping-relocated · Spring Boot 4.0 Migration Guide

@PropertyMapping is used when authoring custom test slice annotations. It binds annotation attributes to Spring Boot auto-configuration properties. The move is part of the same test-infrastructure cleanup that also relocated several other test-autoconfigure classes in Spring Boot 4.0.

Security DSL Rewrite

Spring Security removed authorizeRequests(), antMatchers(), and the .and() chaining method. The entire HTTP security DSL must be rewritten.

EFFORT: L

⚙ OpenRewrite

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/SecurityConfig.java:[22,14]
  error: cannot find symbol
    symbol:   method authorizeRequests()
    location: variable http of type HttpSecurity
[ERROR] /src/main/java/com/example/SecurityConfig.java:[23,18]
  error: cannot find symbol
    symbol:   method antMatchers(java.lang.String)
[ERROR] /src/main/java/com/example/SecurityConfig.java:[28,10]
  error: cannot find symbol
    symbol:   method and()
```

WHAT CHANGED

Spring Security 7.0 (shipped with Boot 4.0) removed the deprecated `authorizeRequests()` method — use `authorizeHttpRequests()` instead. `antMatchers()`, `mvcMatchers()`, and `regexMatchers()` are replaced by `requestMatchers()`. The `.and()` chaining method is gone — use lambda DSL with separate method calls instead.

```
// Security filter chain — before
http
    .authorizeRequests()
        .antMatchers("/public/**").permitAll()
        .antMatchers("/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated()
        .and()
    .formLogin()
        .loginPage("/login")
        .and()
    .logout()
        .logoutSuccessUrl("/");

http
    .authorizeHttpRequests(auth → auth
        .requestMatchers("/public/**").permitAll()
        .requestMatchers("/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated()
    )
    .formLogin(form → form
        .loginPage("/login")
    )
    .logout(logout → logout
        .logoutSuccessUrl("/")
    );


// CSRF configuration
http.csrf().disable();

http.csrf(csrf → csrf.disable());
```

WHY THIS CHANGED

The lambda DSL is more readable and avoids a class of configuration bugs where `.and()` chains silently reset the security context. The new `requestMatchers()` API auto-detects whether to use Ant or MVC matching based on your classpath.

HOW TO FIX

A **Rewrite to lambda DSL.** Replace every `.and()` chain with a lambda-based configuration block. Each security concern (authorization, form login, logout, CSRF) gets its own lambda. See the Spring Security 7 migration guide.

B **Replace matchers.** Change `antMatchers()` and `mvcMatchers()` to `requestMatchers()`. The new method auto-selects the matching strategy. `regexMatchers()` becomes `requestMatchers(new RegexRequestMatcher(...))`.

✓ **Verify:** App starts and security filter chain initialises without errors

Scope check: Search for `authorizeRequests()`, `antMatchers()`, `mvcMatchers()`, `.and()` in security config classes, and `.csrf().disable()`. Every `SecurityFilterChain` bean needs reviewing.

WATCH OUT

The `.and()` removal is the most disruptive part. If you have a long chain with 5+ `.and()` calls, the rewrite changes the entire structure of the method. Review the result carefully — it's easy to accidentally nest a lambda inside the wrong block.

`requestMatchers()` auto-detects MVC vs Ant matching. If you have Spring MVC on the classpath, it uses MVC matching (which is stricter). If this changes behaviour, use `AntPathRequestMatcher` explicitly.

FURTHER INFO

spring-break module: security-removed-apis · Spring Security 7 migration guide

Driven by Spring Security 7.0. The authorizeRequests(), antMatchers(), and .and() chaining were deprecated in Security 5.7/6.0 and removed in 7.0. The lambda DSL was introduced in Security 5.2 as the recommended replacement. Affects all spring-boot-starter-security consumers. See also: websecurity-adapter-removed, auth-default-deny.

Session.delete() Removed

Hibernate 7.0 removed Session.delete() — use Session.remove() instead. Any direct Hibernate Session usage for deleting entities won't compile.

EFFORT: M ⚙ OpenRewrite

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/repo/CustomRepoImpl.java:[28,16]
  error: cannot find symbol
    symbol:   method delete(com.example.model.Order)
    location: variable session of type org.hibernate.Session
```

WHAT CHANGED

Hibernate 7.0 removed the legacy `Session.delete()` method. The JPA-standard `EntityManager.remove()` (which maps to `Session.remove()`) is the only way to delete entities. Similarly, `Session.save()`, `Session.update()`, and `Session.saveOrUpdate()` were also removed — use `persist()` and `merge()` instead.

```
// Deleting an entity
session.delete(order);
session.remove(order);

// Saving an entity
session.save(newOrder);
session.persist(newOrder);

// Updating an entity
session.update(existingOrder);
session.merge(existingOrder);

// Save-or-update pattern
session.saveOrUpdate(order);
session.merge(order);
```

WHY THIS CHANGED

These methods were Hibernate-proprietary duplicates of the JPA standard API. Having two ways to do the same thing caused confusion about cascade behaviour and flush timing. The JPA methods have clearer semantics.

HOW TO FIX

A **Replace with JPA standard methods.** `delete()` becomes `remove()`, `save()` becomes `persist()`, `update()` becomes `merge()`, `saveOrUpdate()` becomes `merge()`.

B **Prefer EntityManager over Session.** If you're unwrapping the `Session` just to call these methods, consider using `EntityManager` directly. It has the same `persist()`, `merge()`, and `remove()` methods.

✓ **Verify:** mvn compile — no Session.delete() symbol errors

Scope check: Search for `session.delete(`, `session.save(`, `session.update(`, and `session.saveOrUpdate(`. Also search for `Session.delete` in any utility or base repository classes.

WATCH OUT

`merge()` returns the managed instance; `saveOrUpdate()` did not. If your code ignores the return value of `saveOrUpdate()`, the switch to `merge()` works fine. But if you used the original reference after the call, you need to use the returned instance from `merge()` instead.

`remove()` requires the entity to be in a managed state. If you were calling `delete()` on a detached entity, you need to `merge()` it first or re-fetch it. Otherwise you'll get an `IllegalArgumentException` at runtime.

FURTHER INFO

spring-break module: hibernate-session-delete · Hibernate 7 migration guide

Driven by Hibernate 7.0 (upstream of Spring Boot 4.0). Session.delete(), Session.save(), Session.update(), and Session.saveOrUpdate() were deprecated in Hibernate 6.0 and removed in 7.0 as part of the alignment with the JPA EntityManager API. See also: cascade-save-update, hibernate-dialect-removal.

SimpDestinationMessageMatcher Removed

SimpDestinationMessageMatcher is gone in Spring Security 7.0. WebSocket security now uses the AuthorizationManager pattern.

EFFORT: M

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/SimpDestUsage.java:[3,66]
  error: package org.springframework.security.messaging.util.matcher
  does not contain SimpDestinationMessageMatcher
```

WHAT CHANGED

`org.springframework.security.messaging.util.matcher.SimpDestinationMessageMatcher` has been removed. Spring Security 7.0 replaces the old `AbstractSecurityWebSocketMessageBrokerConfigurer` / matcher-based approach with `MessageMatcherDelegatingAuthorizationManager`.

```
// Old — matcher-based WebSocket security config
import org.springframework.security.messaging.util.matcher.SimpDestinationMessageMatcher;
// ...
new SimpDestinationMessageMatcher("/topic/**");

None

// New — AuthorizationManager-based config
None
import org.springframework.security.messaging.access.intercept.MessageMatcherDelegatingAuthorizationManager;
// ...
MessageMatcherDelegatingAuthorizationManager.builder()
    .simpDestMatchers("/topic/**").authenticated()
    .anyMessage().denyAll()
    .build();
```

WHY THIS CHANGED

Spring Security 7.0 completed the migration to the AuthorizationManager pattern across all areas including messaging. The old configurer and matcher classes were deprecated in 6.x and removed in 7.0.

HOW TO FIX

A **Migrate to MessageMatcherDelegatingAuthorizationManager.** Replace any use of `SimpDestinationMessageMatcher` and `AbstractSecurityWebSocketMessageBrokerConfigurer` with a `MessageMatcherDelegatingAuthorizationManager` bean registered on your message broker. See the Spring Security 7 messaging migration guide for the full configuration pattern.

✓ **Verify:** mvn compile — no cannot find symbol for SimpDestinationMessageMatcher

Scope check: Search for `SimpDestinationMessageMatcher` and `AbstractSecurityWebSocketMessageBrokerConfigurer`. Both are removed. Only applications using STOMP over WebSocket with Spring Security are affected.

WATCH OUT

The new AuthorizationManager approach requires explicitly wiring the manager into the message broker configuration. There is no auto-migration path — the configuration structure changes substantially.

FURTHER INFO

spring-break module: simpdest-message-matcher-removed · Spring Security 7 messaging migration

Part of Spring Security 7.0's overhaul of WebSocket security. The old matcher-based API for STOMP message authorisation has been replaced by the AuthorizationManager pattern. Teams using WebSocket security need to migrate to MessageMatcherDelegatingAuthorizationManager.

spring-jcl Removed

Spring Framework replaced its spring-jcl logging bridge with a direct dependency on Commons Logging 1.3.0. Manual spring-jcl exclusions break.

EFFORT: S

⚙ OpenRewrite

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] Failed to execute goal on project my-service:
  Could not resolve dependencies:
  The following artifacts could not be resolved:
    org.springframework:spring-jcl:jar:7.0.0 (not found in central)
```

WHAT CHANGED

The `org.springframework:spring-jcl` module was removed in Spring Framework 7.0. Spring now depends directly on `commons-logging:commons-logging:1.3.0`, which gained native SLF4J and `java.util.logging` bridge support. If your POM explicitly references `spring-jcl` — or excludes `commons-logging` in favour of `spring-jcl` — the build breaks.

```
// pom.xml — remove spring-jcl exclusion/inclusion
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-core</artifactId>
    <exclusions>
        <exclusion>
            <groupId>commons-logging</groupId>
            <artifactId>commons-logging</artifactId>
        </exclusion>
    </exclusions>
</dependency>

<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-core</artifactId>
    <!— Commons Logging 1.3.0 is pulled in transitively — no exclusion needed —>
</dependency>


// pom.xml — remove explicit spring-jcl dependency
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jcl</artifactId>
</dependency>

<!— spring-jcl no longer exists. Commons Logging 1.3.0 bridges to SLF4J natively. —>
```

WHY THIS CHANGED

Commons Logging 1.3 added the same SLF4J bridge that spring-jcl provided. With the upstream library now handling the bridge natively, Spring's forked module was redundant.

HOW TO FIX

A **Remove spring-jcl references.** Delete any explicit `spring-jcl` dependency declarations. Remove any `commons-logging` exclusions on Spring modules — those were needed to avoid the old Commons Logging / spring-jcl conflict. Commons Logging 1.3.0 bridges to SLF4J out of the box.

B **Remove jcl-over-slf4j.** If you had `org.slf4j:jcl-over-slf4j` on the classpath, remove it. Commons Logging 1.3.0 provides the same bridge natively. Having both will cause a classpath conflict.

✓ **Verify:** mvn compile — no spring-jcl dependency errors

Scope check: Search for `spring-jcl`, `jcl-over-slf4j`, and exclusions of `commons-logging` in all POM and Gradle files. These are all artifacts of the old logging bridge dance that should be cleaned up.

WATCH OUT

The classic "commons-logging exclusion + jcl-over-slf4j" pattern that every Spring project used for a decade is now harmful. It will exclude the Commons Logging 1.3.0 that Spring actually needs, then try to replace it with an older SLF4J bridge that may conflict. Remove both the exclusion and jcl-over-slf4j.

If you use a BOM or parent POM that applies global exclusions on commons-logging (common in large enterprises), that exclusion now breaks Spring. Update the corporate parent POM.

FURTHER INFO

Spring-Break Demo · Spring Boot 4.0 Migration Guide

Spring Framework 7.0 replaced its forked spring-jcl module with a direct dependency on Commons Logging 1.3.0, which gained native SLF4J bridge support. The classic commons-logging exclusion + jcl-over-slf4j pattern is now harmful and should be removed entirely.

spring-retry Removed from BOM

Spring Boot 4.0 removed spring-retry from its managed dependency BOM. Projects that relied on version management must now declare the version explicitly.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] 'dependencies.dependency.version' for org.springframework.retry:spring-retry:jar
  is missing. @ line 42, column 17
[ERROR] Failed to execute goal on project my-service:
  Could not resolve dependencies: Failed to collect dependencies at
  org.springframework.retry:spring-retry:jar:RELEASE (no version)
```

WHAT CHANGED

`org.springframework.retry:spring-retry` is no longer version-managed by the Spring Boot BOM. If you declared the dependency without an explicit `<version>` tag, the build fails because Maven/Gradle can't resolve it. The library itself still works — it just isn't managed anymore.

```
// pom.xml — add explicit version
<dependency>
    <groupId>org.springframework.retry</groupId>
    <artifactId>spring-retry</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.retry</groupId>
    <artifactId>spring-retry</artifactId>
    <version>2.0.11</version>
</dependency>


// build.gradle — add explicit version
implementation 'org.springframework.retry:spring-retry'
implementation 'org.springframework.retry:spring-retry:2.0.11'
```

WHY THIS CHANGED

Spring Retry's release cadence diverged from Spring Boot's. Managing it in the BOM implied a level of compatibility testing that wasn't happening. Removing it from the BOM lets teams pin the version they actually test against.

HOW TO FIX

A **Add an explicit version.** Add `<version>2.0.11</version>` (or the latest compatible release) to your spring-retry dependency declaration. Check Maven Central for the latest version.

B **Import Spring Retry's own BOM.** If multiple modules use spring-retry, add its BOM to your `<dependencyManagement>` section so versions are managed in one place.

✓ **Verify:** mvn dependency:tree shows aligned Spring Retry version and retry works

Scope check: Search for `spring-retry` in POM and Gradle files. Any declaration without an explicit version will fail. Also check for `@Retryable` and `@EnableRetry` annotations — these confirm the library is actually in use.

WATCH OUT

If you use `spring-cloud-starter-circuitbreaker-resilience4j`, it may have pulled in spring-retry transitively. With the BOM removal, the transitive version is no longer aligned. Pin it explicitly to avoid classpath surprises.

Spring Retry 2.x requires Spring Framework 6.0+. Since Boot 4.0 ships Framework 7.0, use Retry 2.0.x — older 1.x versions are incompatible.

FURTHER INFO

spring-break module: spring-retry-removed · Spring Boot 4.0 Migration Guide

Spring Retry was removed from the Spring Boot 4.0 managed dependency BOM because its release cadence diverged from Boot's. The library itself still works; you just need an explicit version (2.0.x for Framework 7 compatibility). See also: retry-semantics, retryable-transaction-order.

Spring Security Access API Moved to spring-security-access

AccessDecisionManager and related legacy authorisation classes moved to a separate spring-security-access module. Add it or migrate to AuthorizationManager.

EFFORT: M

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/AccessApiUsage.java:[3,48]
  error: package org.springframework.security.access does not exist
```

WHAT CHANGED

`AccessDecisionManager`, `AccessDecisionVoter`, `@EnableGlobalMethodSecurity`, and related legacy authorisation classes have been moved from `spring-security-core` to a new standalone `spring-security-access` artifact.

```
// pom.xml — add legacy module if needed
None
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-access</artifactId>
</dependency>


// Preferred: migrate to AuthorizationManager
import org.springframework.security.access.AccessDecisionManager;
// implements AccessDecisionManager

import org.springframework.security.authorization.AuthorizationManager;
// implements AuthorizationManager<RequestAuthorizationContext>
```

WHY THIS CHANGED

Spring Security 7.0 promotes the modern `AuthorizationManager` API as the sole supported approach. The legacy Access API is preserved for migration purposes but isolated into its own module so teams that have completed migration do not carry the dead weight.

HOW TO FIX

A **Add spring-security-access (short-term fix).** Add `spring-security-access` as an explicit dependency. This restores compilation without any logic changes and buys time to do the full migration.

B **Migrate to AuthorizationManager (recommended).** Replace `AccessDecisionManager` / `AccessDecisionVoter` implementations with `AuthorizationManager`. Replace `@EnableGlobalMethodSecurity` with `@EnableMethodSecurity`. The Spring Security migration guide has step-by-step instructions.

✓ **Verify:** mvn compile — no cannot find symbol for AccessDecisionManager or related classes

Scope check: Search for `AccessDecisionManager`, `AccessDecisionVoter`, `@EnableGlobalMethodSecurity`, and `org.springframework.security.access` imports across all sources.

WATCH OUT

`@EnableGlobalMethodSecurity` is also in the legacy module. If you use method security, you need either the legacy module or to migrate to `@EnableMethodSecurity` before removing it.

FURTHER INFO

spring-break module: spring-security-access-relocated · Spring Security access API moves

Spring Security 7.0 moved the legacy Access API out of spring-security-core into a separate spring-security-access module. This was done to allow teams to continue using the legacy API during migration without it polluting the core module. The modern replacement is AuthorizationManager.

Testcontainers 2.0 Package Relocation

Testcontainers 2.0 moved from org.testcontainers to com.testcontainers. Every import and Maven coordinate changed.

EFFORT: S ⚙ OpenRewrite

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/test/java/com/example/IntegrationTest.java:[4,29]
  error: package org.testcontainers.containers does not exist
[ERROR] /src/test/java/com/example/IntegrationTest.java:[5,29]
  error: package org.testcontainers.junit.jupiter does not exist
[ERROR] /src/test/java/com/example/IntegrationTest.java:[12,6]
  error: cannot find symbol
    symbol: class Testcontainers
```

WHAT CHANGED

Testcontainers 2.0 relocated all packages from `org.testcontainers` to `com.testcontainers`. The Maven group ID changed from `org.testcontainers` to `com.testcontainers`. Spring Boot 4.0's managed dependencies point to Testcontainers 2.0, so the old coordinates no longer resolve.

```
// pom.xml — BOM import
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers-bom</artifactId>
    <version>1.19.8</version>
    <type>pom</type>
    <scope>import</scope>
</dependency>

<!— Managed by Spring Boot 4.0 parent POM — remove explicit BOM if using starter-parent —>


// pom.xml — module dependency
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>postgresql</artifactId>
    <scope>test</scope>
</dependency>

<dependency>
    <groupId>com.testcontainers</groupId>
    <artifactId>postgresql</artifactId>
    <scope>test</scope>
</dependency>


// Java imports
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;

import com.testcontainers.containers.PostgreSQLContainer;
import com.testcontainers.junit.jupiter.Container;
import com.testcontainers.junit.jupiter.Testcontainers;
```

WHY THIS CHANGED

The Testcontainers project moved to a commercial stewardship model under AtomicJar / Docker. The package and group ID change reflects the new organisational identity and gives a clean break for the 2.0 API changes.

HOW TO FIX

A **Update group IDs and imports.** Change the Maven group ID from `org.testcontainers` to `com.testcontainers` in your POM. Then update all Java imports from `org.testcontainers` to `com.testcontainers`. See the Testcontainers 2.0 migration guide.

B **Remove explicit BOM if using Spring Boot parent.** Spring Boot 4.0's parent POM manages the Testcontainers version. If you previously imported the Testcontainers BOM explicitly, remove it to avoid version conflicts.

✓ **Verify:** mvn test — Testcontainers start and tests pass

Scope check: Search for `org.testcontainers` in all POM files, Gradle files, and Java/Kotlin test sources. Every hit needs the package prefix updated.

WATCH OUT

Spring Boot's `@ServiceConnection` annotation and `DynamicPropertySource` patterns still work — but they reference the new Testcontainers 2.0 classes internally. If you have custom `@DynamicPropertySource` methods that reference container types, those imports need updating too.

If you use Testcontainers in a shared test library or test fixtures module, update that module first. Otherwise downstream projects will get classpath conflicts between the 1.x and 2.x packages.

FURTHER INFO

spring-break module: testcontainers-class-relocation · Testcontainers 2.0 migration guide

Driven by Testcontainers 2.0 (managed by Spring Boot 4.0). The package relocated from org.testcontainers to com.testcontainers and the Maven group ID changed accordingly. Spring Boot 4.0's BOM manages the Testcontainers 2.0 version. Affects spring-boot-testcontainers and @ServiceConnection users.

TestRestTemplate Package Relocated

TestRestTemplate moved to org.springframework.boot.resttestclient. Every test that imports the old package fails to compile.

EFFORT: S

WHAT YOU'LL SEE

```
$ mvn test-compile
[ERROR] /src/test/java/com/example/MyControllerTest.java:[3,56]
  error: package org.springframework.boot.test.web.client does not exist
[ERROR] /src/test/java/com/example/MyControllerTest.java:[15,5]
  error: cannot find symbol
    symbol: class TestRestTemplate
```

WHAT CHANGED

`TestRestTemplate` was removed from `org.springframework.boot.test.web.client` and relocated to `org.springframework.boot.resttestclient` as part of a new dedicated module for REST test clients.

```
// Test class import
~~import org.springframework.boot.test.web.client.TestRestTemplate;~~
import org.springframework.boot.resttestclient.TestRestTemplate;
```

WHY THIS CHANGED

Spring Boot 4.0 modularised its testing support. The REST test client infrastructure was separated into its own module to reduce the footprint of the core test autoconfigure dependency and align with the spring-boot-test-rest-client naming.

HOW TO FIX

A **Update the import.** Replace `org.springframework.boot.test.web.client.TestRestTemplate` with `org.springframework.boot.resttestclient.TestRestTemplate`. The class itself is functionally equivalent.

B **Consider RestTestClient.** The new `RestTestClient` provides a more modern, fluent API for REST testing and is the preferred approach in Boot 4.0. If you are refactoring tests anyway, this is a good time to migrate.

✓ **Verify:** mvn test-compile — no package does not exist for org.springframework.boot.test.web.client

Scope check: Search all test sources for `org.springframework.boot.test.web.client.TestRestTemplate` and for `@Autowired TestRestTemplate` or `@Autowired private TestRestTemplate`. Every hit needs the import updated.

WATCH OUT

Any custom configuration class that creates a `TestRestTemplate` bean (e.g., for authentication headers) also needs the import updated — not just the test classes that inject it.

FURTHER INFO

spring-break module: testrest-template-removed · Spring Boot 4.0 Migration Guide

Part of a broader Spring Boot 4.0 testing infrastructure overhaul. TestRestTemplate moved from spring-boot-test-autoconfigure to a new spring-boot-test-rest-client module. Teams with many integration tests that inject TestRestTemplate will see compile failures across multiple test files. See also: springboottest-no-mockmvc (MockMvc auto-config removed).

Undertow Embedded Server Removed

Spring Boot 4.0 dropped Undertow as an embedded server option. Projects depending on spring-boot-starter-undertow won't resolve.

EFFORT: M

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] Failed to execute goal on project my-service:
  Could not resolve dependencies for project com.example:my-service:jar:1.0.0:
  The following artifacts could not be resolved:
    org.springframework.boot:spring-boot-starter-undertow:jar:4.0.0 (not found)
```

WHAT CHANGED

The `spring-boot-starter-undertow` module was removed from Spring Boot 4.0. Undertow is no longer a supported embedded servlet container. The supported options are Tomcat (default), Jetty, and Netty (for reactive).

```
// pom.xml — remove Undertow starter
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <exclusions>
        <exclusion>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-tomcat</artifactId>
        </exclusion>
    </exclusions>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-undertow</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<!— Uses Tomcat by default. For Jetty, add spring-boot-starter-jetty
     and exclude spring-boot-starter-tomcat —>


// application.yml — Undertow-specific config
server:
  undertow:
    io-threads: 8
    worker-threads: 64
    buffer-size: 1024

server:
  tomcat:
    threads:
      max: 200
      min-spare: 10
```

WHY THIS CHANGED

Undertow's development has slowed and its Jakarta EE support lagged behind Tomcat and Jetty. Maintaining three servlet container integrations was a support burden. Tomcat and Jetty cover the same use cases with better Jakarta EE 11 compatibility.

HOW TO FIX

A **Switch to Tomcat (default).** Remove the `spring-boot-starter-undertow` dependency and the Tomcat exclusion. Tomcat is the default — just use `spring-boot-starter-web` without modifications.

B **Switch to Jetty.** If you need a non-Tomcat server, use `spring-boot-starter-jetty` instead. Exclude the Tomcat starter and add the Jetty starter.

✓ **Verify:** App starts on Tomcat/Jetty with no Undertow class errors

Scope check: Search for `spring-boot-starter-undertow` in POM and Gradle files. Also search for `server.undertow` in application properties/YAML files — those properties won't cause a build failure but will be silently ignored.

WATCH OUT

Undertow-specific configuration properties under `server.undertow.*` have no Tomcat or Jetty equivalent. Review your `application.yml` and translate settings like `io-threads`, `worker-threads`, and `buffer-size` to the new server's configuration model.

If you chose Undertow for its non-blocking I/O on the servlet side, consider switching to Spring WebFlux with Netty instead of forcing the same architecture onto Tomcat.

FURTHER INFO

spring-break module: undertow-removed · Spring Boot 4.0 Migration Guide

Spring Boot 4.0 dropped spring-boot-starter-undertow. Undertow's Jakarta EE 11 support lagged behind Tomcat and Jetty. Tomcat (default) and Jetty are the remaining embedded servlet container options; Netty remains available for reactive. Server-specific properties under server.undertow.* are silently ignored.

WebSecurityConfigurerAdapter Removed

WebSecurityConfigurerAdapter was deleted from Spring Security. Any config class extending it won't compile.

EFFORT: M

⚙ OpenRewrite

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/SecurityConfig.java:[6,55]
  error: cannot find symbol
    symbol: class WebSecurityConfigurerAdapter
[ERROR] /src/main/java/com/example/SecurityConfig.java:[10,5]
  error: method does not override or implement a method from a supertype
    @Override
    protected void configure(HttpSecurity http)
```

WHAT CHANGED

`WebSecurityConfigurerAdapter` was deprecated in Spring Security 5.7 and removed in 7.0. Configuration is now done by declaring `@Bean` methods that return `SecurityFilterChain`, `UserDetailsService`, etc. The `configure(HttpSecurity)` and `configure(AuthenticationManagerBuilder)` override pattern is gone.

```
// Security configuration class
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
            );
    }

    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws Exception {
        auth.inMemoryAuthentication()
            .withUser("user").password("{noop}pass").roles("USER");
    }
}

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
            );
        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService() {
        var user = User.withUsername("user")
            .password("{noop}pass").roles("USER").build();
        return new InMemoryUserDetailsManager(user);
    }
}
```

WHY THIS CHANGED

The adapter pattern forced all security config into a single class and made it hard to compose multiple filter chains. Bean-based configuration is more modular and works naturally with Spring's dependency injection.

HOW TO FIX

A **Convert to @Bean configuration.** Remove the `extends WebSecurityConfigurerAdapter`. Turn `configure(HttpSecurity)` into a `@Bean` method returning `SecurityFilterChain`. Call `http.build()` at the end. Move authentication config to separate `@Bean` methods.

B **Use OpenRewrite.** The `org.openrewrite.java.spring.security6.RemoveWebSecurityConfigurerAdapter` recipe automates the conversion.

✓ **Verify:** mvn compile — no WebSecurityConfigurerAdapter references

Scope check: Search for `extends WebSecurityConfigurerAdapter` and `import org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerAdapter`. Every hit is a class that needs rewriting.

WATCH OUT

Don't forget `http.build()`. The bean method must return a `SecurityFilterChain`, not `void`. Missing the return statement is a common mistake that gives a confusing null-pointer error at startup.

If you had multiple `configure()` overrides in the same adapter, split them into separate beans. Authentication configuration should become a `UserDetailsService` or `AuthenticationManager` bean, not part of the filter chain.

FURTHER INFO

spring-break module: security-removed-apis · Spring Security 7 migration guide

WebSecurityConfigurerAdapter was deprecated in Spring Security 5.7 and removed in 7.0. The bean-based SecurityFilterChain approach was introduced in Security 5.4 as the recommended replacement. Affects all spring-boot-starter-security consumers. See also: security-dsl-rewrite, auth-default-deny.

TIER 2

Won't Run

Runtime & Startup Failures

Your code compiles but crashes at startup or runtime. Configuration, security, and framework changes that break execution.

SUBSYSTEM INDEX · 13 CARDS

Spring Security
- Security DSL Rewrite
- OAuth Password Grant
- PKCE Mandatory
- Default Deny

Spring Batch
- Batch Schema Rename
- Batch Listener Classes

Testing
- Testcontainers Package

Spring Data
- Query Generation

Spring Web
- RestTemplate Auto-config
- Path Matching Engine

Configuration
- Config Props Field Binding
- Spring Retry BOM
- Retry Semantics

13 cards in this tier

AspectJ Weaving Required for @Observed

Spring Boot 4.0 requires AspectJ-based weaving for the @Observed annotation to produce spans and metrics. Without the aspect configured, @Observed silently does nothing.

EFFORT: S

WHAT YOU'LL SEE

```
# No startup error — this is a silent behaviour change.
# Detected when @Observed methods produce no observations:

@Observed(name = "order.process")
public Order processOrder(OrderRequest request) { ... }

# Expected in metrics:
$ curl -s http://localhost:8080/actuator/metrics/order.process
{"name":"order.process","measurements":[{"statistic":"COUNT","value":42.0}]}

# Actual after upgrade:
$ curl -s http://localhost:8080/actuator/metrics/order.process
{"status":404,"error":"Not Found"}

# No metric registered. No spans created. Method executes normally
# but observability is completely missing.
```

WHAT CHANGED

In Spring Boot 3.x, `@Observed` worked via Spring AOP proxies with auto-configuration that registered the `ObservedAspect` bean automatically. In Spring Boot 4.0, the `ObservedAspect` auto-configuration requires explicit AspectJ weaving support. If your project does not have the AspectJ weaving dependency and configuration, the `@Observed` annotation is silently ignored.

```
// Maven dependency — add AspectJ weaver
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing</artifactId>
</dependency>

<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing</artifactId>
</dependency>
<dependency>
    <groupId>org.aspectj</groupId>
    <artifactId>aspectjweaver</artifactId>
</dependency>


// Configuration — register ObservedAspect if not auto-configured
// ObservedAspect was auto-configured in Boot 3.x
@Configuration
public class ObservabilityConfig {
    @Bean
    public ObservedAspect observedAspect(ObservationRegistry registry) {
        return new ObservedAspect(registry);
    }
}


// application.properties — enable AspectJ auto-proxy
# proxy-based AOP was sufficient
spring.aop.auto=true
```

WHY THIS CHANGED

The change aligns `@Observed` processing with proper AspectJ semantics, enabling it to work on non-proxied beans, self-invocations, and private methods. The proxy-based approach had known limitations that caused confusion when `@Observed` silently failed on internal method calls.

HOW TO FIX

A **Add AspectJ weaver dependency.** Add `org.aspectj:aspectjweaver` to your dependencies. Spring Boot's dependency management provides the version. This enables the AspectJ-based processing of `@Observed`.

B **Register ObservedAspect bean explicitly.** If the auto-configuration does not pick up the aspect, register an `ObservedAspect` bean manually in a `@Configuration` class. Pass the `ObservationRegistry` to its constructor.

C **Verify AOP is enabled.** Ensure `spring.aop.auto=true` (the default) and that you have not disabled AspectJ auto-proxying. Check for `@EnableAspectJAutoProxy` if you have custom AOP configuration.

✓ **Verify:** Custom @Observed aspects fire and metrics appear in /actuator/metrics

Scope check: Search for `@Observed` annotations across your codebase. Every method annotated with `@Observed` is silently broken if AspectJ weaving is not configured. Also check if you depend on metrics or traces produced by `@Observed` in dashboards or alerts.

WATCH OUT

This is completely silent. The application starts, methods execute, but no observations (metrics or traces) are recorded. You only discover it when monitoring dashboards show gaps.

If you use `@Observed` on `@Repository` or `@Service` beans that are JDK dynamic proxies, ensure AspectJ weaving mode is compatible. CGLIB proxies are generally safer.

Existing `@Timed` and `@Counted` annotations from Micrometer are separate from `@Observed` and may have their own configuration requirements. Check both.

Load-time weaving (`-javaagent:aspectjweaver.jar`) gives the broadest coverage but requires JVM argument changes. Compile-time weaving requires build plugin changes. Choose based on your deployment model.

FURTHER INFO

Spring-Break Demo · Spring Boot 4.0 Migration Guide

Introduced in Spring Framework 7.0 / Spring Boot 4.0. The ObservedAspect auto-configuration change is part of the broader Micrometer Observation API refactoring. Affects spring-boot-starter-actuator consumers using @Observed. See also: controller-spans-disabled, observability-dashboard-gaps.

Spring Batch 6 Schema Sequence Rename

Spring Batch 6 renamed its database sequences (e.g., BATCH_JOB_SEQ became BATCH_JOB_INSTANCE_SEQ). Existing databases using the old names fail when Batch tries to generate IDs.

EFFORT: S

⚙ OpenRewrite

WHAT YOU'LL SEE

```
org.springframework.dao.InvalidDataAccessResourceUsageException:
  could not fetch next sequence value
...
Caused by: java.sql.SQLSyntaxErrorException:
  sequence "BATCH_JOB_INSTANCE_SEQ" does not exist
 at org.postgresql.core.v3.QueryExecutorImpl.receiveErrorResponse(...)
 ...
org.springframework.batch.core.launch.JobLaunchException:
  Could not launch job 'importCustomersJob'.
  Failed to create new JobInstance for job [importCustomersJob]
```

WHAT CHANGED

Spring Batch 6 renamed the database sequences used for generating IDs: `BATCH_JOB_SEQ` became `BATCH_JOB_INSTANCE_SEQ`, `BATCH_JOB_EXECUTION_SEQ` stayed the same, and `BATCH_STEP_EXECUTION_SEQ` stayed the same. The metadata table structures also changed. Applications with existing Batch schema from Spring Batch 5 will fail when the framework tries to use the new sequence names.

```
// SQL migration script — PostgreSQL
-- Old sequences from Spring Batch 5
-- Rename sequences to match Spring Batch 6 expectations
ALTER SEQUENCE BATCH_JOB_SEQ RENAME TO BATCH_JOB_INSTANCE_SEQ;


// SQL migration script — MySQL
-- Old sequences from Spring Batch 5
-- MySQL uses tables for sequence emulation
RENAME TABLE BATCH_JOB_SEQ TO BATCH_JOB_INSTANCE_SEQ;


// application.properties — opt into old names temporarily
# no Batch schema configuration
# Point Batch to legacy sequence names while you plan migration
spring.batch.jdbc.isolation-level-for-create=ISOLATION_DEFAULT
```

WHY THIS CHANGED

The rename aligns sequence names with the tables they serve, making the schema self-documenting. The old generic names like `BATCH_JOB_SEQ` were ambiguous about which table's IDs they generated.

HOW TO FIX

A **Run a schema migration script.** Create a Flyway or Liquibase migration that renames the sequences. For PostgreSQL: `ALTER SEQUENCE BATCH_JOB_SEQ RENAME TO BATCH_JOB_INSTANCE_SEQ;`. For MySQL, rename the emulation tables. Apply this before deploying the upgraded application.

B **Let Spring Batch recreate the schema.** If your Batch job data is disposable (e.g., dev/staging), set `spring.batch.jdbc.initialize-schema=always` to let Spring Batch drop and recreate the metadata tables with the new names. Do not use this in production.

C **Use the migration SQL from Spring Batch.** The Spring Batch 6 migration guide provides official SQL scripts for each database platform. Use those as the basis for your migration rather than writing them by hand.

✓ **Verify:** Spring Batch job launches with no BATCH_ table not found errors

Scope check: Check if your application uses `spring-boot-starter-batch` and has a persistent job repository (not in-memory). If you see `BATCH_JOB_INSTANCE`, `BATCH_JOB_EXECUTION`, or `BATCH_STEP_EXECUTION` tables in your database, you need this migration. In-memory job repositories (MapJobRepositoryFactoryBean) are not affected.

WATCH OUT

The schema change is not just sequences. Column types and table structures also changed in Spring Batch 6. Check the official migration SQL scripts for the full set of changes.

If you run multiple application versions against the same database during a rolling deploy, the old version will break when the sequences are renamed. Plan a maintenance window or use blue-green deployment.

H2 in-memory databases used in tests auto-create the schema fresh each run, so tests may pass while production fails. Test against a real database with existing Batch tables.

FURTHER INFO

spring-break module: batch-schema-change · Spring Batch 6 migration guide · Spring Boot 4.0 Migration Guide

Driven by Spring Batch 6.0 (upstream of Spring Boot 4.0). The sequence rename was announced in the Batch 6.0 milestone notes. Only affects applications with a persistent JDBC job repository; in-memory repositories are unaffected. See also: batch-listener-classes.

CascadeType.SAVE_UPDATE Removed in Hibernate 7

Hibernate 7 removed the proprietary CascadeType.SAVE_UPDATE. Entities using this cascade type fail at startup or throw MappingException when the SessionFactory is built.

EFFORT: M

⚙ OpenRewrite

WHAT YOU'LL SEE

```
org.springframework.beans.factory.BeanCreationException:
  Error creating bean with name 'entityManagerFactory'
...
Caused by: org.hibernate.MappingException:
  Unrecognized legacy cascade style: save-update
 at org.hibernate.boot.model.internal.EntityBinder.bindEntityAnnotation(...)
 at org.hibernate.boot.model.internal.AnnotationBinder.bindClass(...)
 ...
APPLICATION FAILED TO START
---
Description:
Failed to initialize JPA EntityManagerFactory:
Unrecognized legacy cascade style: save-update
```

WHAT CHANGED

`org.hibernate.annotations.CascadeType.SAVE_UPDATE` was a Hibernate-specific extension to JPA cascade types. It triggered cascade behaviour on Hibernate's proprietary `Session.saveOrUpdate()` method. Hibernate 7 removed both `saveOrUpdate()` and the corresponding cascade type. Code using the Hibernate-specific `@Cascade` annotation with `SAVE_UPDATE` breaks at entity mapping time.

```
// Using Hibernate @Cascade annotation
import org.hibernate.annotations.Cascade;
import org.hibernate.annotations.CascadeType;

@OneToMany(mappedBy = "order")
@Cascade(CascadeType.SAVE_UPDATE)
private List<OrderItem> items;

@OneToMany(mappedBy = "order", cascade = {
    javax.persistence.CascadeType.PERSIST,
    javax.persistence.CascadeType.MERGE
})
private List<OrderItem> items;


// Using JPA cascade with Hibernate extras
@OneToMany(mappedBy = "parent")
@Cascade({CascadeType.SAVE_UPDATE, CascadeType.DELETE})
private Set<Child> children;

@OneToMany(mappedBy = "parent", cascade = {
    CascadeType.PERSIST, CascadeType.MERGE, CascadeType.REMOVE
})
private Set<Child> children;
```

WHY THIS CHANGED

The `saveOrUpdate()` method was deprecated in Hibernate 6 as part of aligning with the JPA `EntityManager` API. Hibernate 7 completed the removal. The standard JPA `CascadeType.PERSIST` and `CascadeType.MERGE` cover the same use cases without vendor lock-in.

HOW TO FIX

A **Replace with JPA standard cascades.** Swap `CascadeType.SAVE_UPDATE` for the combination of `CascadeType.PERSIST` and `CascadeType.MERGE`. These two JPA-standard cascades cover the same persist-or-update behaviour that `SAVE_UPDATE` provided.

B **Remove @Cascade if redundant.** If your entity already has `cascade = CascadeType.ALL` on the JPA annotation, the Hibernate `@Cascade` is redundant. Delete it entirely.

✓ **Verify:** mvn compile — no CascadeType.SAVE_UPDATE errors

Scope check: Search for `CascadeType.SAVE_UPDATE` and `@Cascade` (the Hibernate annotation, not JPA). Every hit is a mapping that will fail at startup. Also search for `session.saveOrUpdate(` — that method is gone too.

WATCH OUT

`CascadeType.PERSIST` alone does not cover the "update existing detached entity" case. You need both `PERSIST` and `MERGE` to match the old `SAVE_UPDATE` behaviour.

If you were using `Session.saveOrUpdate()` directly in DAO code, that method is also removed. Replace with `entityManager.merge()` or `entityManager.persist()`.

Some code generators and legacy mapping tools emit `SAVE_UPDATE` by default. Re-run generation after upgrading.

FURTHER INFO

spring-break module: hibernate-cascade-removal · Hibernate 7 migration guide · Spring Boot 4.0 Migration Guide

Driven by Hibernate 7.0 (upstream of Spring Boot 4.0 via spring-boot-starter-data-jpa). Session.saveOrUpdate() and CascadeType.SAVE_UPDATE were deprecated in Hibernate 6.0 and removed in 7.0 as part of the alignment with the JPA EntityManager API. See also: session-delete-removed, hibernate-dialect-removal.

Controller & View Spans Disabled by Default

Spring Boot 4.0 disables controller and view rendering spans by default in Micrometer tracing. Dashboards, alerts, and SLOs relying on these spans silently lose data.

EFFORT: S

WHAT YOU'LL SEE

```
# No startup error — this is a silent behaviour change.
# Detected when spans are missing in your observability platform:

$ curl -s http://localhost:16686/api/traces?service=my-app | jq '.data[0].spans[].operationName'
"HTTP GET /api/orders"
# Previously also showed:
# "OrderController#getOrders"
# "orders :: view rendering"
# These spans are now GONE.

# Grafana alert fires:
[FIRING] No controller-level spans for service my-app in last 15m
# Or SLO dashboard shows 0% data for controller latency percentiles.
```

WHAT CHANGED

Spring Boot 4.0 changed the default for `management.observations.http.server.requests.name` and disabled the creation of child spans for Spring MVC controller method invocations and view rendering. Previously, Micrometer Tracing created spans for both the HTTP request and the controller method execution. Now only the HTTP server request span is created by default.

```
// application.properties — re-enable controller spans
# default: controller spans were enabled
management.observations.http.server.requests.controller.enabled=true
management.observations.http.server.requests.view.enabled=true


// application.yml equivalent
# default: controller spans were enabled
management:
  observations:
    http:
      server:
        requests:
          controller:
            enabled: true
          view:
            enabled: true
```

WHY THIS CHANGED

The controller and view spans added overhead and noise for applications that did not need sub-request granularity. Disabling them by default reduces trace volume and performance impact. Applications that need them can re-enable them explicitly.

HOW TO FIX

A **Re-enable controller spans.** Add `management.observations.http.server.requests.controller.enabled=true` to your `application.properties`. Do the same for `view.enabled` if you need view rendering spans. This restores the Spring Boot 3.x behaviour.

B **Update dashboards and alerts.** If you decide to accept the new default (no controller spans), update your Grafana dashboards, alert rules, and SLO definitions to use the HTTP server request span instead of controller-level spans. The HTTP span still contains the handler method in its attributes.

C **Use @Observed for targeted spans.** Instead of blanket controller spans, annotate specific controller methods with `@Observed` to get spans only where you need them. This is more efficient than re-enabling all controller spans globally.

✓ **Verify:** Trace dashboard shows controller-level spans after re-enabling

Scope check: Check your observability dashboards and alerts for references to controller-level span names. Search for span names like `controller.method.name`, `HandlerMethod`, or view rendering span names in your Grafana/Datadog/New Relic configuration. If you only use HTTP-level metrics (`http.server.requests`), you are not affected.

WATCH OUT

This is completely silent at the application level. No errors, no warnings. You only discover it when your dashboards go blank or alerts fire for missing data.

If you use distributed tracing to debug latency between the HTTP layer and the controller, the missing span makes it harder to pinpoint where time is spent inside the server.

Micrometer's `@Timed` annotation on controller methods still works for metrics. Only tracing spans are affected, not timer metrics.

FURTHER INFO

Spring Boot 4.0 Migration Guide

Introduced in Spring Boot 4.0 as part of the Micrometer Tracing defaults overhaul. The change reduces default trace cardinality and overhead for high-traffic services. Affects spring-boot-starter-actuator with Micrometer Tracing on the classpath. See also: observability-dashboard-gaps, aspectj-observed.

Version-Specific Hibernate Dialects Removed

Hibernate 7 removed version-specific dialect classes like MySQL57Dialect and PostgreSQL95Dialect. Applications that explicitly configure a dialect fail at startup with ClassNotFoundException.

EFFORT: S

⚙ OpenRewrite

WHAT YOU'LL SEE

```
org.springframework.beans.factory.BeanCreationException:
  Error creating bean with name 'entityManagerFactory'
...
Caused by: java.lang.ClassNotFoundException:
  org.hibernate.dialect.MySQL57Dialect
 at java.base/jdk.internal.loader.BuiltinClassLoader.loadClass(BuiltinClassLoader.java:641)
 at java.base/jdk.internal.loader.ClassLoaders$AppClassLoader.loadClass(ClassLoaders.java:188)
 ...
APPLICATION FAILED TO START
---
Description:
Failed to configure a DataSource: Hibernate dialect
'org.hibernate.dialect.MySQL57Dialect' could not be found.
```

WHAT CHANGED

Hibernate 7 removed all version-specific dialect classes (`MySQL57Dialect`, `MySQL8Dialect`, `PostgreSQL95Dialect`, `PostgreSQL10Dialect`, `Oracle12cDialect`, `SQLServer2016Dialect`, etc.). Only the base dialect classes remain (`MySQLDialect`, `PostgreSQLDialect`, `OracleDialect`, `SQLServerDialect`). Hibernate now auto-detects the database version from the JDBC connection and configures features accordingly.

```
// application.properties
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL57Dialect
# Remove explicit dialect — Hibernate auto-detects from JDBC connection

// application.properties — if you must be explicit
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQL95Dialect
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect

// persistence.xml
<property name="hibernate.dialect" value="org.hibernate.dialect.Oracle12cDialect"/>
<property name="hibernate.dialect" value="org.hibernate.dialect.OracleDialect"/>
```

WHY THIS CHANGED

Maintaining separate dialect classes for every database version created a combinatorial explosion. Hibernate's new approach detects the database version at runtime and enables the correct features automatically, eliminating the need for users to choose the right dialect class.

HOW TO FIX

A **Remove explicit dialect (recommended).** Delete the `spring.jpa.properties.hibernate.dialect` property entirely. Hibernate 7 auto-detects the correct dialect from the JDBC connection metadata. This is the cleanest fix and the one the Hibernate team recommends.

B **Switch to base dialect class.** If you need an explicit dialect for testing or CI (e.g., H2 in-memory), use the unversioned base class: `MySQLDialect`, `PostgreSQLDialect`, `OracleDialect`, or `SQLServerDialect`.

✓ **Verify:** App starts with no DialectResolutionInfoMissingException

Scope check: Search for `hibernate.dialect` in `application.properties`, `application.yml`, `persistence.xml`, and any programmatic `LocalContainerEntityManagerFactoryBean` configuration. Also check test configurations and Docker Compose profiles that may set dialect overrides.

WATCH OUT

If you use Testcontainers or H2 for tests, those test configs often set an explicit dialect. Check `src/test/resources` too.

Some database connection pools or frameworks set the dialect programmatically. Search for `setProperty("hibernate.dialect"` in Java code, not just property files.

The auto-detection requires an active JDBC connection at startup. If your application uses lazy datasource initialization, you may still need an explicit base dialect class.

FURTHER INFO

spring-break module: hibernate-dialect-removal · Hibernate 7 migration guide · Spring Boot 4.0 Migration Guide

Driven by Hibernate 7.0 (upstream of Spring Boot 4.0). Version-specific dialect classes (MySQL57Dialect, PostgreSQL95Dialect, Oracle12cDialect, etc.) were deprecated in Hibernate 6.0 and removed in 7.0 in favour of runtime version auto-detection. Affects spring-boot-starter-data-jpa consumers. See also: hibernate-date-types, cascade-save-update.

JacksonException No Longer Extends IOException

Jackson 3.0 changed the exception hierarchy so JacksonException extends RuntimeException instead of IOException. Existing catch blocks silently miss Jackson errors, and throws clauses become lies.

EFFORT: M

⚙ OpenRewrite

WHAT YOU'LL SEE

```
com.example.OrderController: Unhandled exception in /api/orders
tools.jackson.core.exc.StreamReadException: Unexpected character ('x' (code 120)):
  expected a valid value (JSON String, Number, Array, Object or token)
 at [Source: (String)"x]"; line: 1, column: 2]
 at tools.jackson.core.json.JsonParser._reportUnexpectedChar(JsonParser.java:...)
 ...
Caused by: tools.jackson.core.JacksonException (not an IOException!)
---
HTTP 500 Internal Server Error
{"timestamp":"2026-04-15T10:22:03","status":500,
 "error":"Internal Server Error","path":"/api/orders"}
```

WHAT CHANGED

In Jackson 2.x, `JacksonException` extended `java.io.IOException`, so any `catch (IOException e)` block would also catch malformed-JSON errors. In Jackson 3.0, `JacksonException` extends `RuntimeException`. Catch blocks for `IOException` no longer intercept Jackson parsing failures, and methods that declared `throws IOException` for Jackson work now throw unchecked exceptions instead.

```
// Controller error handling — before
try {
    Order order = objectMapper.readValue(body, Order.class);
} catch (IOException e) {
    return ResponseEntity.badRequest().body("Invalid JSON");
}

try {
    Order order = objectMapper.readValue(body, Order.class);
} catch (JacksonException e) {
    return ResponseEntity.badRequest().body("Invalid JSON");
}


// Spring @ExceptionHandler — before
@ExceptionHandler(IOException.class)
public ResponseEntity<String> handleJsonError(IOException ex) {

@ExceptionHandler(JacksonException.class)
public ResponseEntity<String> handleJsonError(JacksonException ex) {


// Method signature cleanup
public Order parseOrder(String json) throws IOException {
public Order parseOrder(String json) {
```

WHY THIS CHANGED

Jackson parsing errors are not truly I/O errors. The original hierarchy was a design mistake that forced callers to handle checked exceptions even for in-memory parsing. Making `JacksonException` unchecked aligns with modern Java conventions and removes boilerplate try/catch from code that parses strings or byte arrays.

HOW TO FIX

A **Search for catch blocks.** Grep for `catch (IOException` and `catch (JsonProcessingException`. Replace with `catch (JacksonException` where the intent is to handle Jackson parse failures. Keep `IOException` catches for genuine I/O work (file reads, sockets).

B **Update @ExceptionHandler advice.** If your `@RestControllerAdvice` catches `IOException` to return 400 on bad JSON, add a separate handler for `JacksonException`. Otherwise those errors now surface as unhandled 500s.

C **Clean up throws clauses.** Methods that declared `throws IOException` only for Jackson can drop the clause entirely. The compiler won't complain, but the stale declaration misleads readers.

✓ **Verify:** mvn compile — no JsonMappingException symbol errors

Scope check: Search for `catch (IOException`, `catch (JsonProcessingException`, and `throws IOException` near any `ObjectMapper` usage. Every hit where the catch was meant to intercept Jackson errors is now a silent miss at runtime. REST controllers, message listeners, and integration test assertions are the most common locations.

WATCH OUT

This is a silent behaviour change. The code compiles cleanly because `JacksonException` is now unchecked. You only discover the problem when bad input arrives and the catch block no longer fires, producing 500 errors instead of 400s.

If you catch `Exception` broadly, you are technically still catching Jackson errors, but you lose the ability to return a domain-appropriate error message. Narrow your catch clauses.

Spring's built-in `HttpMessageNotReadableException` still wraps Jackson errors for `@RequestBody` parsing. The risk is in manual `ObjectMapper` calls inside your own code.

FURTHER INFO

spring-break module: jackson-exception-hierarchy · Jackson 3 migration guide · Spring Boot 4.0 Migration Guide

Driven by Jackson 3.0 (upstream of Spring Boot 4.0). JacksonException was moved from extending IOException to RuntimeException. The change was announced in 2022 and finalised in Jackson 3.0-rc1. Affects all code with catch(IOException) blocks near ObjectMapper calls. See also: jackson-group-id, jackson-class-renames.

@MockBean / @SpyBean Removed

Spring Boot 4.0 removed the @MockBean and @SpyBean annotations. Tests using them fail at runtime with annotation-not-found errors. Replace with @MockitoBean and @MockitoSpyBean.

EFFORT: M

⚙ OpenRewrite

WHAT YOU'LL SEE

```
java.lang.NoClassDefFoundError:
  org/springframework/boot/test/mock/mockito/MockBean
 at com.example.OrderServiceTest.<clinit>(OrderServiceTest.java:18)
Caused by: java.lang.ClassNotFoundException:
  org.springframework.boot.test.mock.mockito.MockBean
---
java.lang.IllegalStateException: Failed to load ApplicationContext for
  [MergedContextConfiguration@4a5905...
  testClass = com.example.OrderServiceTest]
...
Tests run: 1, Failures: 0, Errors: 1, Skipped: 0
```

WHAT CHANGED

The `@MockBean` and `@SpyBean` annotations from `org.springframework.boot.test.mock.mockito` were deprecated in Spring Boot 3.4 and removed in 4.0. They are replaced by `@MockitoBean` and `@MockitoSpyBean` from `org.springframework.test.context.bean.override.mockito`, which live in Spring Framework 7 itself rather than Spring Boot.

```
// Import change
~~import org.springframework.boot.test.mock.mockito.MockBean;~~
import org.springframework.test.context.bean.override.mockito.MockitoBean;

~~import org.springframework.boot.test.mock.mockito.SpyBean;~~
import org.springframework.test.context.bean.override.mockito.MockitoSpyBean;

// Annotation usage
~~@MockBean~~
~~private OrderRepository orderRepository;~~

~~@SpyBean~~
~~private NotificationService notificationService;~~

@MockitoBean
private OrderRepository orderRepository;

@MockitoSpyBean
private NotificationService notificationService;
```

WHY THIS CHANGED

The new annotations are part of Spring Framework's test context, not Spring Boot. This means they work in plain Spring Framework tests too, not just Boot tests. The new implementation also uses a cleaner bean override mechanism that avoids some of the context caching issues that plagued `@MockBean`.

HOW TO FIX

A **Find-and-replace annotations.** Replace `@MockBean` with `@MockitoBean` and `@SpyBean` with `@MockitoSpyBean`. Update the imports from `org.springframework.boot.test.mock.mockito` to `org.springframework.test.context.bean.override.mockito`.

B **OpenRewrite recipe.** The Spring Boot 4.0 OpenRewrite migration recipe handles this rename automatically across the entire test codebase.

C **Check for @MockBean attributes.** If you used `@MockBean(name = "...")` or `@MockBean(classes = ...)`, verify the equivalent attributes exist on `@MockitoBean`. The attribute names may differ slightly.

✓ **Verify:** mvn test — no @MockBean compile errors

Scope check: Search for `@MockBean` and `@SpyBean` across `src/test`. Every test class using these annotations needs updating. In a typical Spring Boot project, this can be dozens or hundreds of test files. Count them before starting.

WATCH OUT

The `@MockitoBean` reset behaviour between tests may differ from `@MockBean`. If you relied on mocks being reset between test methods, verify that behaviour is preserved.

The new annotations are in the Spring Framework package, not Spring Boot. If you have test utility classes that reference the old annotation by fully qualified name (e.g., in custom annotations or reflection), those references need updating too.

IDE templates and test generators may still produce `@MockBean`. Update your IDE templates after migrating.

FURTHER INFO

spring-break module: mockbean-removed · Testing changes in Spring Boot 4.0 · Spring Boot 4.0 Migration Guide

The @MockBean and @SpyBean annotations were deprecated in Spring Boot 3.4 and removed in 4.0. Their replacements (@MockitoBean, @MockitoSpyBean) live in Spring Framework 7's test context (org.springframework.test.context.bean.override.mockito), not in Spring Boot. The new implementation uses a cleaner bean override mechanism.

OAuth 2.0 Password Grant Completely Removed

Spring Security 7 removed all support for the OAuth 2.0 Resource Owner Password Credentials grant. Applications that use password grant for authentication fail at startup or return 401s at runtime.

EFFORT: L

WHAT YOU'LL SEE

```
org.springframework.beans.factory.BeanCreationException:
  Error creating bean with name 'securityFilterChain'
...
Caused by: java.lang.IllegalArgumentException:
  The grant type 'password' is not supported.
  Supported grant types are: [authorization_code, client_credentials,
  refresh_token, urn:ietf:params:oauth:grant-type:device_code,
  urn:ietf:params:oauth:grant-type:token-exchange]
 at org.springframework.security.oauth2.client.registration.ClientRegistration$Builder.build(...)
---
APPLICATION FAILED TO START
```

WHAT CHANGED

The OAuth 2.0 Resource Owner Password Credentials grant (password grant) was deprecated in Spring Security 5.x and has been fully removed in Spring Security 7 (shipped with Spring Boot 4.0). The `ClientRegistration` builder rejects `authorization-grant-type: password`, and the `ResourceOwnerPasswordResourceDetails` class no longer exists.

```
// application.yml — client registration
spring:
  security:
    oauth2:
      client:
        registration:
          my-api:
            authorization-grant-type: password
            client-id: my-client
            client-secret: secret
        provider:
          my-api:
            token-uri: https://auth.example.com/oauth/token

spring:
  security:
    oauth2:
      client:
        registration:
          my-api:
            authorization-grant-type: authorization_code
            client-id: my-client
            client-secret: secret
            scope: openid,profile
            redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"
        provider:
          my-api:
            issuer-uri: https://auth.example.com


// Java configuration — programmatic registration
ClientRegistration.withRegistrationId("legacy")
    .authorizationGrantType(AuthorizationGrantType.PASSWORD)
    .tokenUri("https://auth.example.com/oauth/token")
    .clientId("my-client")
    .clientSecret("secret")
    .build();

ClientRegistration.withRegistrationId("legacy")
    .authorizationGrantType(AuthorizationGrantType.AUTHORIZATION_CODE)
    .authorizationUri("https://auth.example.com/authorize")
    .tokenUri("https://auth.example.com/oauth/token")
    .redirectUri("{baseUrl}/login/oauth2/code/{registrationId}")
    .clientId("my-client")
```

```
    .clientSecret("secret")
    .build();
```

WHY THIS CHANGED

The password grant requires users to give their credentials directly to the client application, which defeats the purpose of OAuth delegation. The OAuth 2.1 specification formally removed it. Spring Security aligned with that specification.

HOW TO FIX

A **Migrate to Authorization Code + PKCE.** Replace the password grant with Authorization Code flow. For browser-based apps, use Authorization Code with PKCE. This is the recommended approach and requires changes to both the client application and possibly the authorization server configuration.

B **Use Client Credentials for service-to-service.** If the password grant was used for machine-to-machine communication (no actual user), switch to the Client Credentials grant. This does not require user interaction.

C **Custom token exchange as a last resort.** If you must support legacy systems that send username/password, implement a custom token exchange endpoint on your authorization server that accepts credentials and issues tokens. This keeps the anti-pattern in one controlled place while you plan a proper migration.

✓ **Verify:** Token endpoint returns 200 for auth code flow (not password grant)

Scope check: Search for `authorization-grant-type: password` in YAML/properties files and `AuthorizationGrantType.PASSWORD` in Java code. Also search for `ResourceOwnerPasswordResourceDetails` which was the OAuth 2.0 password grant helper class. Any hit means that authentication flow needs redesigning.

WATCH OUT

This is not just a Spring change — OAuth 2.1 removed the password grant from the specification itself. Your authorization server (Keycloak, Auth0, Okta) may also drop support. Check both sides.

Mobile apps that used password grant to avoid browser redirects should migrate to Authorization Code with PKCE, which modern mobile OAuth SDKs support natively.

Integration tests that authenticate by posting username/password to the token endpoint will break. Update test helpers to use a test-specific grant flow or mock the security context directly.

FURTHER INFO

spring-break module: oauth-password-grant-removed · OAuth Password Grant removal issue · Spring Security 7 migration · Spring Boot 4.0 Migration Guide

Driven by Spring Security 7.0. The OAuth 2.0 Resource Owner Password Credentials grant was deprecated in Security 5.x and is formally removed from the OAuth 2.1 specification (RFC 9700). Tracked in spring-security#6003. Affects spring-boot-starter-oauth2-client consumers. See also: pkce-mandatory.

OpenSAML 4 Support Removed

Spring Security 7 dropped OpenSAML 4 support and requires OpenSAML 5. Applications using SAML 2.0 login fail at startup with ClassNotFoundException for OpenSAML 4 classes.

EFFORT: L

WHAT YOU'LL SEE

```
org.springframework.beans.factory.BeanCreationException:
  Error creating bean with name 'securityFilterChain'
...
Caused by: java.lang.NoClassDefFoundError:
  org/opensaml/saml/saml2/core/impl/AuthnRequestMarshaller
 at org.springframework.security.saml2.provider.service.registration
  .OpenSamlRelyingPartyRegistrationBuilderFactory.<clinit>(...)
Caused by: java.lang.ClassNotFoundException:
  org.opensaml.saml.saml2.core.impl.AuthnRequestMarshaller
 at java.base/jdk.internal.loader.BuiltinClassLoader.loadClass(...)
---
APPLICATION FAILED TO START
```

WHAT CHANGED

Spring Security 7 removed all OpenSAML 4 compatibility classes and now requires OpenSAML 5 exclusively. The internal SAML support classes (`OpenSaml4AuthenticationProvider` , `OpenSaml4AuthenticationRequestResolver` , etc.) were replaced with OpenSAML 5 equivalents. OpenSAML 5 changed package structures and removed the marshaller/unmarshaller split.

```
// Maven dependency
<dependency>
    <groupId>org.opensaml</groupId>
    <artifactId>opensaml-saml-api</artifactId>
    <version>4.3.2</version>
</dependency>
<dependency>
    <groupId>org.opensaml</groupId>
    <artifactId>opensaml-saml-impl</artifactId>
    <version>4.3.2</version>
</dependency>

<dependency>
    <groupId>org.opensaml</groupId>
    <artifactId>opensaml-saml-api</artifactId>
    <version>5.1.2</version>
</dependency>
<dependency>
    <groupId>org.opensaml</groupId>
    <artifactId>opensaml-saml-impl</artifactId>
    <version>5.1.2</version>
</dependency>


// Java config — authentication provider
import org.springframework.security.saml2.provider.service.authentication
    .OpenSaml4AuthenticationProvider;

OpenSaml4AuthenticationProvider provider =
    new OpenSaml4AuthenticationProvider();

import org.springframework.security.saml2.provider.service.authentication
    .OpenSaml5AuthenticationProvider;

OpenSaml5AuthenticationProvider provider =
    new OpenSaml5AuthenticationProvider();


// Java config — custom response validator
OpenSaml4AuthenticationProvider provider =
    new OpenSaml4AuthenticationProvider();
provider.setResponseValidator(responseToken -> {
```

```
~~    Saml2ResponseValidatorResult result =~~
~~        OpenSaml4AuthenticationProvider~~
~~            .createDefaultResponseValidator()~~
~~            .convert(responseToken);~~

OpenSaml5AuthenticationProvider provider =
    new OpenSaml5AuthenticationProvider();
provider.setResponseValidator(responseToken -> {
    Saml2ResponseValidatorResult result =
        OpenSaml5AuthenticationProvider
            .createDefaultResponseValidator()
            .convert(responseToken);
```

WHY THIS CHANGED

OpenSAML 4 reached end of life. OpenSAML 5 brought significant API improvements, including a simplified object-provider model and better XML security defaults. Maintaining dual support across two major OpenSAML versions was unsustainable.

HOW TO FIX

A **Upgrade OpenSAML dependencies to 5.x.** Update your OpenSAML dependencies to version 5.1.x. If you use Spring Boot's dependency management, the managed version will already be OpenSAML 5 in Spring Boot 4.0. Remove any explicit version overrides pinning to 4.x.

B **Rename Spring Security SAML classes.** Replace all references to `OpenSaml4*` classes with their `OpenSaml5*` equivalents. The main classes are `OpenSaml5AuthenticationProvider` and `OpenSaml5AuthenticationRequestResolver`.

C **Review custom SAML processing code.** If you directly use OpenSAML APIs (marshallers, unmarshallers, builders), review the OpenSAML 5 migration guide. The marshaller/unmarshaller split was removed and the object provider model changed.

✓ **Verify:** SAML login flow completes with OpenSAML 5 libraries

Scope check: Search for `opensaml` in your Maven/Gradle files, and for `OpenSaml4` in Java code. If your application uses SAML 2.0 login (`saml2Login()` in your security config), you are affected. If you only use OAuth 2.0 / OIDC, this card does not apply.

WATCH OUT

OpenSAML 5 requires Java 17+ and has different XML processing requirements. Verify your XML security libraries are compatible.

If you have custom `Saml2AuthenticationRequestContext` processing or assertion decryption, those APIs may have changed signatures in the OpenSAML 5 upgrade.

The Shibboleth Maven repository is still required for OpenSAML 5 artifacts. Make sure your build still has that repository configured.

FURTHER INFO

Spring-Break Demo · Spring Security 7 migration · Spring Boot 4.0 Migration Guide

Driven by Spring Security 7.0. OpenSAML 4 reached end of life; OpenSAML 5 is now required. The OpenSaml4AuthenticationProvider was replaced by OpenSaml5AuthenticationProvider. Affects spring-boot-starter-security consumers using SAML 2.0 login (saml2Login()). The Shibboleth Maven repository is still required for OpenSAML artifacts.

OTLP/HTTP Now Primary Export Protocol

Spring Boot 4.0 changed the default OTLP transport from gRPC to HTTP/protobuf. Applications exporting to gRPC-only collectors fail silently or get connection refused errors at runtime.

EFFORT: S

WHAT YOU'LL SEE

```
WARN [otel.exporter] io.opentelemetry.exporter.otlp.http.OtlpHttpSpanExporter:
  Failed to export spans. Server responded with HTTP status 404.
  url=http://otel-collector:4318/v1/traces
---
# Or if collector only exposes gRPC port 4317:
WARN [otel.exporter] io.opentelemetry.exporter.otlp.http.OtlpHttpSpanExporter:
  Failed to export spans.
  java.net.ConnectException: Connection refused
  target=http://otel-collector:4318/v1/traces
---
# Metrics and traces silently stop arriving in your backend.
# Application continues running but observability data is lost.
```

WHAT CHANGED

Spring Boot 4.0 changed the default OTLP export transport from gRPC (port 4317) to HTTP/protobuf (port 4318). The default endpoint changed from `http://localhost:4317` to `http://localhost:4318/v1/traces` (and `/v1/metrics`, `/v1/logs`). Applications that relied on the default gRPC transport now send data to the wrong port and protocol.

```
// application.properties — keep using gRPC
# default was gRPC on port 4317
management.otlp.tracing.transport=grpc
management.otlp.tracing.endpoint=http://otel-collector:4317
management.otlp.metrics.export.transport=grpc
management.otlp.metrics.export.endpoint=http://otel-collector:4317


// application.properties — accept new HTTP default
management.otlp.tracing.endpoint=http://otel-collector:4317

management.otlp.tracing.endpoint=http://otel-collector:4318/v1/traces


// docker-compose.yml — expose HTTP port on collector
otel-collector:
  image: otel/opentelemetry-collector:latest
  ports:
    - "4317:4317"   # gRPC

otel-collector:
  image: otel/opentelemetry-collector:latest
  ports:
    - "4317:4317"   # gRPC
    - "4318:4318"   # HTTP/protobuf
```

WHY THIS CHANGED

HTTP/protobuf is more universally supported, works through proxies and load balancers without special gRPC configuration, and does not require the gRPC dependency. The OpenTelemetry specification recommends HTTP/protobuf as the default transport.

HOW TO FIX

A **Accept HTTP/protobuf (recommended).** Update your OTLP endpoint to use port 4318 and the HTTP path format. Ensure your collector exposes the HTTP receiver. Most modern collectors (OpenTelemetry Collector, Grafana Agent, Datadog Agent) support both protocols.

B **Explicitly set gRPC transport.** If your collector only supports gRPC, add `management.otlp.tracing.transport=grpc` and set the endpoint to the gRPC port. This preserves the old behaviour.

C **Update collector configuration.** If using the OpenTelemetry Collector, ensure both the `otlp/grpc` and `otlp/http` receivers are enabled in your collector config. This lets you support both old and new applications during migration.

✓ **Verify:** OTLP exporter sends traces to collector (check collector logs)

Scope check: Check your `application.properties` for `management.otlp` settings. If you have explicit endpoint configuration pointing to port 4317, the application will try HTTP on a gRPC port. If you have no explicit configuration, the default changed from gRPC/4317 to HTTP/4318.

WATCH OUT

This is a silent failure in production. The application starts and runs normally. You only notice when traces and metrics stop appearing in your observability backend. Add health checks for telemetry export.

If you use a service mesh or network policy that only allows port 4317, you need to open port 4318 as well.

Some managed observability platforms (Datadog, New Relic) have different endpoints for HTTP vs gRPC OTLP ingestion. Check your provider's documentation for the correct HTTP endpoint URL.

The gRPC dependency (`io.grpc:grpc-netty-shaded`) may no longer be pulled in transitively. If you switch back to gRPC, you may need to add it explicitly.

FURTHER INFO

Spring Boot 4.0 Migration Guide

Introduced in Spring Boot 4.0, aligning with the OpenTelemetry specification's recommendation of HTTP/protobuf as the default OTLP transport. The default port changed from 4317 (gRPC) to 4318 (HTTP). Affects spring-boot-starter-actuator consumers using OTLP tracing or metrics export.

PKCE Mandatory for Confidential OAuth Clients

Spring Security 7 enforces PKCE for all OAuth 2.0 clients, including confidential clients. Authorization requests without a code_challenge are rejected by the authorization server.

EFFORT: M

WHAT YOU'LL SEE

```
[oauth2-client] o.s.s.oauth2.client.web.OAuth2LoginAuthenticationFilter:
  Authentication request failed:
org.springframework.security.oauth2.core.OAuth2AuthenticationException:
  [invalid_request] PKCE code_challenge is required for this client.
 at org.springframework.security.oauth2.client.oidc.authentication
  .OidcAuthorizationCodeAuthenticationProvider.authenticate(...)
---
HTTP 302 → /login?error=invalid_request
---
Browser shows: "OAuth2 Error: [invalid_request]
PKCE code_challenge is required for this client."
```

WHAT CHANGED

Spring Security 7 now sends PKCE parameters (`code_challenge` and `code_challenge_method`) on every Authorization Code request, for both public and confidential clients. If the authorization server requires PKCE but the client was not sending it (pre-upgrade behaviour for confidential clients), or if the authorization server rejects unexpected PKCE parameters, the flow breaks.

```
// application.yml — no changes needed on client side
spring:
  security:
    oauth2:
      client:
        registration:
          my-app:
            authorization-grant-type: authorization_code
            client-id: my-confidential-client
            client-secret: ${CLIENT_SECRET}
            scope: openid,profile

# Client config stays the same — Spring Security 7 adds PKCE automatically.
# But your authorization server MUST accept PKCE parameters.
spring:
  security:
    oauth2:
      client:
        registration:
          my-app:
            authorization-grant-type: authorization_code
            client-id: my-confidential-client
            client-secret: ${CLIENT_SECRET}
            scope: openid,profile


// Keycloak realm config — enable PKCE for confidential client
"clientId": "my-confidential-client",
"publicClient": false,
"pkceCodeChallengeMethod": ""

"clientId": "my-confidential-client",
"publicClient": false,
"pkceCodeChallengeMethod": "S256"


// Disable PKCE enforcement if auth server can't support it (temporary)
// default PKCE behaviour
@Bean
public OAuth2AuthorizationRequestResolver pkceResolver(
        ClientRegistrationRepository repo) {
    DefaultOAuth2AuthorizationRequestResolver resolver =
        new DefaultOAuth2AuthorizationRequestResolver(
```

```
            repo, "/oauth2/authorization");
    // Temporary: remove when auth server supports PKCE
    resolver.setAuthorizationRequestCustomizer(
        OAuth2AuthorizationRequestCustomizers.withoutPkce());
    return resolver;
}
```

WHY THIS CHANGED

OAuth 2.1 and the latest security best practices mandate PKCE for all clients, not just public clients. PKCE prevents authorization code interception attacks regardless of client type. Spring Security aligned with this requirement.

HOW TO FIX

A **Enable PKCE on your authorization server.** Configure your authorization server (Keycloak, Auth0, Okta, Azure AD) to accept PKCE `code_challenge` parameters for confidential clients. Most modern providers already support this — it may just need to be enabled per client. Use `S256` as the challenge method.

B **Verify no auth server rejects extra parameters.** Some older or custom authorization servers reject unknown parameters. If yours rejects `code_challenge`, you need to update the auth server first, or temporarily disable PKCE enforcement on the Spring Security side using a custom resolver.

C **Disable PKCE temporarily (not recommended).** If you cannot update the authorization server immediately, register a custom `OAuth2AuthorizationRequestResolver` that strips PKCE parameters. Treat this as technical debt and plan the auth server update.

✓ **Verify:** OAuth2 login completes with PKCE challenge in the auth request

Scope check: Check every OAuth 2.0 client registration in your application. For each one, verify that the corresponding authorization server accepts PKCE parameters. If you use Spring Authorization Server, it already supports PKCE. If you use a third-party provider, check their documentation.

WATCH OUT

The failure often appears only in the browser redirect flow. Automated tests that mock the OAuth exchange may not trigger the error. Test with a real authorization server.

If you use Spring Authorization Server as your auth server and it is also being upgraded, both sides get PKCE support automatically. The risk is when the client (Spring Boot 4) and the auth server (third-party or older version) are upgraded independently.

Azure AD and some ADFS versions may need tenant-level configuration changes to allow PKCE for confidential clients. Check your identity provider's documentation.

FURTHER INFO

spring-break module: pkce-mandatory · PKCE enforcement issue · Spring Security 7 migration · Spring Boot 4.0 Migration Guide

Driven by Spring Security 7.0, aligning with OAuth 2.1 (RFC 9700) which mandates PKCE for all clients. PKCE enforcement was tracked in spring-security#16391. Affects spring-boot-starter-oauth2-client consumers using the authorization_code grant. See also: oauth-password-grant.

RestTemplate Auto-Configuration Removed

Spring Boot 4.0 removed RestTemplateAutoConfiguration. Tests and production code that relied on an auto-configured RestTemplateBuilder or RestTemplate bean get NoSuchBeanDefinitionException at startup.

EFFORT: S

WHAT YOU'LL SEE

```
org.springframework.beans.factory.NoSuchBeanDefinitionException:
  No qualifying bean of type
  'org.springframework.boot.web.client.RestTemplateBuilder' available:
  expected at least 1 bean which qualifies as autowire candidate.
Dependency annotations:
  {@org.springframework.beans.factory.annotation.Autowired(required=true)}
 at org.springframework.beans.factory.support
  .DefaultListableBeanFactory.raiseNoMatchingBeanException(...)
---
APPLICATION FAILED TO START
---
Description:
Parameter 0 of constructor in com.example.OrderClient required a bean
of type 'org.springframework.boot.web.client.RestTemplateBuilder' that
could not be found.
```

WHAT CHANGED

Spring Boot 4.0 removed `RestTemplateAutoConfiguration`, which previously provided a `RestTemplateBuilder` bean and customizers. The auto-configuration for `RestClient` (the modern replacement) remains. Code that injected `RestTemplateBuilder` or relied on auto-configured `RestTemplate` beans no longer finds them.

```
// Injecting the HTTP client — service class
@Service
public class OrderClient {
    private final RestTemplate restTemplate;

    public OrderClient(RestTemplateBuilder builder) {
        this.restTemplate = builder
            .rootUri("https://api.example.com")
            .build();
    }

    public Order getOrder(Long id) {
        return restTemplate.getForObject(
            "/orders/{id}", Order.class, id);
    }
}

@Service
public class OrderClient {
    private final RestClient restClient;

    public OrderClient(RestClient.Builder builder) {
        this.restClient = builder
            .baseUrl("https://api.example.com")
            .build();
    }

    public Order getOrder(Long id) {
        return restClient.get()
            .uri("/orders/{id}", id)
            .retrieve()
            .body(Order.class);
    }
}


// Test — using @RestClientTest
@RestClientTest(OrderClient.class)
class OrderClientTest {
    @Autowired
```

```
~~    private MockRestServiceServer server;~~
~~    @Autowired~~
~~    private OrderClient client;~~

@RestClientTest(OrderClient.class)
class OrderClientTest {
    @Autowired
    private MockRestServiceServer server;
    @Autowired
    private OrderClient client;
    // Works the same — @RestClientTest supports RestClient
```

WHY THIS CHANGED

`RestTemplate` has been in maintenance mode since Spring 5. Spring Framework 6.1 introduced `RestClient` as the modern, fluent replacement. Spring Boot 4.0 completed the transition by removing the auto-configuration, encouraging migration to `RestClient` or `WebClient`.

HOW TO FIX

A **Migrate to RestClient (recommended).** Replace `RestTemplate` usage with `RestClient`. Inject `RestClient.Builder` (auto-configured by Spring Boot 4) instead of `RestTemplateBuilder`. The API is fluent and supports the same customizers.

B **Define your own RestTemplateBuilder bean.** If migration is too large to do immediately, register your own `RestTemplateBuilder` bean in a `@Configuration` class. This restores the old behaviour while you plan the migration.

C **Use WebClient for reactive code.** If your application is reactive, use `WebClient` instead. It remains fully auto-configured in Spring Boot 4.0.

✓ **Verify:** App starts with explicit RestTemplate bean and HTTP calls succeed

Scope check: Search for `RestTemplate`, `RestTemplateBuilder`, and `RestTemplateCustomizer` across your codebase. Every injection point is a potential startup failure. Also check test classes that use `TestRestTemplate` — its auto-configuration may also be affected.

WATCH OUT

`TestRestTemplate` is a separate concern. Check whether it is still auto-configured in your test slices or if you need to provide it manually.

Third-party libraries that inject `RestTemplateBuilder` (e.g., Spring Cloud OpenFeign, some tracing libraries) may also break. Check transitive dependencies.

The `RestClient` API is different from `RestTemplate`. Methods like `getForObject()`, `postForEntity()` become chained `.get().uri().retrieve().body()` calls. Plan time for the API change, not just the bean swap.

FURTHER INFO

Spring-Break Demo · Testing changes in Spring Boot 4.0 · Spring Boot 4.0 Migration Guide

RestTemplate has been in maintenance mode since Spring Framework 5.0. RestClient was introduced in Spring Framework 6.1 as the modern replacement. Spring Boot 4.0 removed RestTemplateAutoConfiguration to encourage migration. RestClient.Builder is auto-configured in Boot 4.0. See also: okhttp3-removed.

Test-Slice Annotation Starters Relocated

Spring Boot 4.0 relocated test-slice annotations and their auto-configuration metadata. Tests using custom test slices or referencing old packages fail with ClassNotFoundException or silently load the wrong context.

EFFORT: S

⚙ OpenRewrite

WHAT YOU'LL SEE

```
java.lang.IllegalStateException: Failed to load ApplicationContext for
  [MergedContextConfiguration@6f2b5e...
  testClass = com.example.web.ProductControllerTest]
Caused by: java.lang.ClassNotFoundException:
  org.springframework.boot.test.autoconfigure.web.servlet
  .WebMvcTypeExcludeFilter
 at java.base/jdk.internal.loader.BuiltinClassLoader.loadClass(...)
---
Or silently:
---
com.example.web.ProductControllerTest > shouldReturnProduct() FAILED
  org.springframework.beans.factory.NoSuchBeanDefinitionException:
  No qualifying bean of type 'com.example.web.ProductController'
  (expected in thin web-slice context but full context was loaded instead)
```

WHAT CHANGED

Spring Boot 4.0 reorganised the test auto-configuration packages. Several `TypeExcludeFilter` classes and test-slice support classes moved or were renamed. Custom test-slice annotations that referenced the old filter classes by fully qualified name break at runtime. Standard annotations like `@WebMvcTest`, `@DataJpaTest`, and `@JsonTest` still work, but custom slices that mirrored their structure need updating.

```
// Custom test-slice annotation
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@BootstrapWith(SpringBootTestContextBootstrapper.class)
@OverrideAutoConfiguration(enabled = false)
@TypeExcludeFilters(CustomSliceTypeExcludeFilter.class)
@AutoConfigureCache
@AutoConfigureCustomService
@ImportAutoConfiguration
public @interface CustomServiceTest {
}

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@BootstrapWith(SpringBootTestContextBootstrapper.class)
@OverrideAutoConfiguration(enabled = false)
@TypeExcludeFilters(CustomSliceTypeExcludeFilter.class)
@AutoConfigureCache
@AutoConfigureCustomService
@ImportAutoConfiguration
public @interface CustomServiceTest {
    // Annotation structure unchanged, but verify your
    // TypeExcludeFilter extends the correct base class
    // and spring.factories / AutoConfiguration.imports are updated
}


// META-INF/spring.factories — test slice registration
org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureWebMvc=\
  com.example.test.CustomWebMvcAutoConfiguration

# Moved to META-INF/spring/AutoConfiguration.imports
# or updated key in spring.factories
org.springframework.boot.autoconfigure.AutoConfiguration.imports=\
  com.example.test.CustomWebMvcAutoConfiguration
```

WHY THIS CHANGED

The test auto-configuration was restructured to support the new bean override mechanism and to align the package structure with Spring Framework 7's test context changes. The reorganisation also improved modularity by moving test-slice support into dedicated modules.

HOW TO FIX

A **Update custom test-slice annotations.** If you wrote custom test-slice annotations (like `@CustomServiceTest`), verify that the `TypeExcludeFilter`, bootstrapper, and auto-configuration imports still resolve. Update fully qualified class references to match the new package locations.

B **Migrate spring.factories to AutoConfiguration.imports.** Move test auto-configuration entries from `META-INF/spring.factories` to `META-INF/spring/AutoConfiguration.imports`. This was deprecated in Boot 3.x and is now enforced.

C **Re-test standard slices.** Even if you only use standard annotations like `@WebMvcTest`, run your full test suite to verify. The context filtering may behave differently and load or exclude different beans.

✓ **Verify:** mvn test — all @DataJpaTest/@WebMvcTest slices load correctly

Scope check: Search for custom annotations that use `@TypeExcludeFilters`, `@OverrideAutoConfiguration`, or `@ImportAutoConfiguration`. Also search for `spring.factories` entries under test source sets. If you only use standard `@WebMvcTest`, `@DataJpaTest`, etc. without customisation, you are likely safe — but still run the tests.

WATCH OUT

This can be a silent failure: your test may load a full application context instead of a thin slice, making it pass but run much slower and test less precisely.

Libraries that provide custom test slices (Spring Cloud, Spring Modulith) need to be updated too. Check their Spring Boot 4.0 compatibility versions.

If your test suite uses `spring.factories` for test auto-configuration and you haven't migrated to `AutoConfiguration.imports`, those configurations are silently ignored.

FURTHER INFO

Spring-Break Demo · Testing changes in Spring Boot 4.0 · Spring Boot 4.0 Migration Guide

Introduced in Spring Boot 4.0 as part of the test auto-configuration restructuring. Standard annotations (@WebMvcTest, @DataJpaTest) still work, but custom test slices referencing old TypeExcludeFilter classes or spring.factories entries need updating. The spring.factories to AutoConfiguration.imports migration is now enforced.

TIER 3

Wrong Results

Subtle Behavioural Changes

Your code runs but produces wrong output. Subtle behavioural changes in serialization, queries, and observability.

SUBSYSTEM INDEX · 13 CARDS

Jackson

- Date Format
- Locale Format
- Property Inclusion
- Dates & Timestamps

Hibernate

- Hibernate Date Types
- LIKE Pattern Escaping

Testing

- Test Slice Relocated

Observability

- Controller Spans Disabled
- Dashboard Gaps
- OTLP HTTP Primary

Spring Framework

- Null-Marked Default
- Retryable Transaction Order
- AspectJ @Observed

13 cards in this tier

Authorisation Default: Implicit Access → Explicit Deny

Spring Security 7 denies access to all endpoints not explicitly listed in your security configuration. Endpoints that worked via implicit permit now silently return 403.

EFFORT: M

WHAT YOU'LL SEE

```
// SecurityFilterChain — unchanged from Boot 3.5
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/public/**").permitAll()
);
// Note: no .anyRequest() rule

// Before (Spring Boot 3.5) — unlisted paths implicitly allowed
GET /api/health → 200 OK
GET /api/users/me → 200 OK

// After (Spring Boot 4.0) — unlisted paths denied
GET /api/health → 403 Forbidden
GET /api/users/me → 403 Forbidden

// Integration test failures
Expected status: 200
  Actual status: 403
```

WHAT CHANGED

Spring Security 7 changed the default policy for endpoints not covered by any `requestMatchers` rule. Previously, unconfigured paths were implicitly accessible (depending on your filter chain setup). Now, any request that doesn't match an explicit rule is denied with 403 Forbidden. This is an "explicit deny by default" model.

```
// Add explicit anyRequest() rule
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/public/**").permitAll()
);

http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/public/**").permitAll()
    .anyRequest().authenticated()  // explicit catch-all
);


// Or if you had truly open endpoints
// /api/health was implicitly accessible
.requestMatchers("/api/health", "/api/info").permitAll()

// Static resources
// /css/**, /js/** implicitly served
.requestMatchers("/css/**", "/js/**", "/favicon.ico").permitAll()
```

WHY THIS CHANGED

Implicit access was a security risk — developers would add new endpoints and forget to add corresponding security rules, accidentally exposing them. Denying by default is the safer posture: every endpoint must be explicitly allowed, so new endpoints are locked down until deliberately opened.

HOW TO FIX

A **Add an explicit .anyRequest() rule (recommended).** End every `authorizeHttpRequests` chain with `.anyRequest().authenticated()` or `.anyRequest().denyAll()`. This makes your policy complete and explicit. Then add `permitAll()` rules for truly public endpoints.

B **Audit all endpoints.** List every `@RequestMapping` / `@GetMapping` / `@PostMapping` in your app and verify each has a matching security rule. Use Spring Boot Actuator's `/mappings` endpoint to get the full list.

✓ **Verify:** All endpoints return 403 unless explicitly permitted in security config

Scope check: Check every `SecurityFilterChain` bean in your app. If any `authorizeHttpRequests` block is missing an `anyRequest()` catch-all rule, all unlisted endpoints are now silently returning 403. Also check for actuator endpoints, static resources, and health checks.

WATCH OUT

Actuator endpoints like `/actuator/health` and `/actuator/info` may be silently blocked. If your load balancer health checks use these, your app will look unhealthy immediately after deployment.

This interacts with CORS preflight requests. `OPTIONS` requests to unlisted paths now get 403, which browsers interpret as a CORS failure. You may see "CORS error" in the browser console when the real issue is the missing security rule.

FURTHER INFO

Spring-Break Demo · Spring Security 7 migration guide

Driven by Spring Security 7.0 (upstream of Spring Boot 4.0). The implicit-permit to explicit-deny change was tracked in spring-security#14597. Deprecated the old behaviour in Security 6.2 with a multi-release migration path. See also: security-dsl-rewrite, websecurity-adapter-removed.

Hibernate Native Query Date Return Types Changed

Native SQL queries now return `java.time.LocalDate` instead of `java.sql.Date`. Code casting to the old types throws ClassCastException or silently loses time components.

EFFORT: M

WHAT YOU'LL SEE

```
// Native query — same code, different result type
Object result = em.createNativeQuery("SELECT created_date FROM orders WHERE id = 1")
                  .getSingleResult();

// Before (Spring Boot 3.5)
result.getClass() → java.sql.Date

// After (Spring Boot 4.0)
result.getClass() → java.time.LocalDate

// ClassCastException at runtime
java.lang.ClassCastException:
  java.time.LocalDate cannot be cast to java.sql.Date

// Or subtler: time component silently lost
java.sql.Timestamp ts = (java.sql.Timestamp) result;  // was Timestamp
// Now: java.time.LocalDateTime — no getTime() method
```

WHAT CHANGED

Hibernate 7 (used by Spring Boot 4.0) changed the default Java types returned for SQL `DATE`, `TIME`, and `TIMESTAMP` columns in native queries. `java.sql.Date` became `java.time.LocalDate`, `java.sql.Time` became `java.time.LocalTime`, and `java.sql.Timestamp` became `java.time.LocalDateTime`.

```
// Fix native query result handling
~~java.sql.Date date = (java.sql.Date) row[2];~~
~~long millis = date.getTime();~~

LocalDate date = (LocalDate) row[2];
long millis = date.atStartOfDay(ZoneOffset.UTC)
                  .toInstant().toEpochMilli();


// Typed native query
~~@Query(value = "SELECT created_date FROM orders", nativeQuery = true)~~
~~List<java.sql.Date> findAllDates();~~

@Query(value = "SELECT created_date FROM orders", nativeQuery = true)
List<java.time.LocalDate> findAllDates();


// Timestamp results
~~java.sql.Timestamp ts = (java.sql.Timestamp) row[3];~~
LocalDateTime ts = (LocalDateTime) row[3];
```

WHY THIS CHANGED

The `java.sql` date/time classes are legacy wrappers around `java.util.Date` with known bugs (timezone handling, nanosecond truncation). The `java.time` API is immutable, thread-safe, and the modern standard. Hibernate aligned its defaults with the JSR 310 types.

HOW TO FIX

A **Update result type casts to java.time (recommended).** Replace all `java.sql.Date` casts with `java.time.LocalDate`, `java.sql.Time` with `java.time.LocalTime`, and `java.sql.Timestamp` with `java.time.LocalDateTime`. This is the correct modern approach.

B **Force old types with explicit addScalar().** For legacy code that can't be updated, use Hibernate's `addScalar("col", StandardBasicTypes.DATE)` to force the old `java.sql` return types on specific queries.

✓ **Verify:** SELECT queries return correct date/time types (check ResultSet)

Scope check: Search for `nativeQuery = true`, `createNativeQuery`, and any cast to `java.sql.Date`, `java.sql.Time`, or `java.sql.Timestamp` in query result handling code. JPQL queries with typed results are unaffected — only native queries and untyped result sets change.

WATCH OUT

This only affects native queries that return raw `Object[]` rows. JPQL queries with entity mappings or `@ColumnResult` type mappings are unaffected because the type is explicit.

`java.time.LocalDate` has no time component. If your code relied on `java.sql.Date.getTime()` returning midnight epoch millis, the replacement requires an explicit timezone conversion.

FURTHER INFO

spring-break module: hibernate-native-datetime · Spring Boot 4.0 Migration Guide

Driven by Hibernate 7.0 (upstream of Spring Boot 4.0). The switch from java.sql date/time types to java.time (JSR 310) in native query results was announced in the Hibernate 7.0 migration guide. Only affects native queries returning raw Object[] rows; JPQL typed queries are unaffected. See also: hibernate-dialect-removal.

Jackson Date Serialisation Flip

Jackson 3.0 defaults to ISO-8601 date strings instead of numeric timestamps. Every API response containing a date silently changes shape.

EFFORT: M

WHAT YOU'LL SEE

```
// API response — before (Spring Boot 3.5)
{"created": 1714089600000}

// API response — after (Spring Boot 4.0)
{"created": "2025-04-26T00:00:00Z"}

// Front-end JavaScript blows up
TypeError: date.getTime is not a function
// or: contract test fails
Expected: <1714089600000> (Long)
  Actual: <"2025-04-26T00:00:00Z"> (String)
```

WHAT CHANGED

Jackson 3.0 changed the default value of `SerializationFeature.WRITE_DATES_AS_TIMESTAMPS` from `true` to `false`. Dates, times, and durations now serialise as ISO-8601 strings by default instead of numeric epoch milliseconds.

```
// application.properties — restore old behaviour
# (no explicit setting needed in Boot 3.5)
spring.jackson.serialization.write-dates-as-timestamps=true

// or adopt ISO-8601 and fix the client
const ts = new Date(response.created);  // was a number
const ts = new Date(response.created);  // now parses ISO string

// ObjectMapper explicit configuration
// Jackson 2 default: timestamps enabled
ObjectMapper mapper = new ObjectMapper();
mapper.registerModule(new JavaTimeModule());

// Jackson 3 default: ISO strings
ObjectMapper mapper = new ObjectMapper();
mapper.registerModule(new JavaTimeModule());
// only if you need the old numeric format:
mapper.enable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
```

WHY THIS CHANGED

ISO-8601 strings are human-readable, timezone-aware, and the de facto standard in JSON APIs. The old numeric default caused constant confusion about whether the value was seconds or milliseconds, and lost timezone information entirely.

HOW TO FIX

A **Opt in to the new ISO-8601 default (recommended).** Update your API clients and contract tests to expect ISO-8601 strings. This is the better long-term format. Coordinate the change with front-end teams so they parse the string, not a number.

B **Restore numeric timestamps.** Add `spring.jackson.serialization.write-dates-as-timestamps=true` to `application.properties`. This gives you the old behaviour while you plan the migration.

✓ **Verify:** JSON responses show dates in expected format (check API output)

Scope check: Search for any DTO field of type `java.util.Date`, `Instant`, `LocalDate`, `LocalDateTime`, `ZonedDateTime`, or `Duration`. Every one of those fields will serialise differently after the upgrade. Check API consumers, contract tests, and front-end parsing code.

WATCH OUT

This change is invisible at compile time and at startup. The app runs perfectly — until a client tries to parse the date field as a number. If you have no contract tests, you won't catch this until production.

The `@JsonFormat` annotation on individual fields still overrides the global default. If you've already annotated your dates, those fields are unaffected. It's the unannotated ones that silently flip.

FURTHER INFO

spring-break module: jackson-date-serialisation · Jackson 3 migration guide · Jackson 3 in Spring (blog)

Driven by Jackson 3.0 (upstream of Spring Boot 4.0). The default for WRITE_DATES_AS_TIMESTAMPS flipped from true to false. Announced in the Jackson 3.0 release notes. Affects all spring-boot-starter-json consumers with date/time fields in DTOs. See also: jackson-dates-timestamps, jackson-property-inclusion.

write-dates-as-timestamps Config Silently Not Applied

The `spring.jackson.serialization.write-dates-as-timestamps` property is silently ignored in Boot 4.0. Your explicit timestamp configuration stops working.

EFFORT: M

WHAT YOU'LL SEE

```
# application.properties (still present, no warning)
spring.jackson.serialization.write-dates-as-timestamps=true

// API response — before (Spring Boot 3.5, property honoured)
{"timestamp": 1714089600000, "expiry": 1714176000000}

// API response — after (Spring Boot 4.0, property ignored)
{"timestamp": "2025-04-26T00:00:00Z", "expiry": "2025-04-27T00:00:00Z"}

// Mobile app crash log
java.lang.NumberFormatException: For input string: "2025-04-26T00:00:00Z"
```

WHAT CHANGED

Spring Boot 4.0's Jackson auto-configuration no longer applies the `spring.jackson.serialization.*` feature toggles to the Jackson 3 ObjectMapper. The property remains in your config with no startup warning, but the serialisation feature is never enabled. Jackson 3's own default (ISO-8601 strings) takes effect instead.

```
// application.properties — old (silently ignored)
spring.jackson.serialization.write-dates-as-timestamps=true
# Removed — configure via ObjectMapper customiser instead

// ObjectMapper customiser replacement
# (relied on spring.jackson.serialization.write-dates-as-timestamps)
@Bean
public Jackson2ObjectMapperBuilderCustomizer timestampDates() {
    return builder -> builder.featuresToEnable(
        SerializationFeature.WRITE_DATES_AS_TIMESTAMPS
    );
}


// Alternative: per-field annotation
private Instant timestamp;
@JsonFormat(shape = JsonFormat.Shape.NUMBER)
private Instant timestamp;
```

WHY THIS CHANGED

Jackson 3 reorganised its feature flag system. The old `SerializationFeature` enum values don't map one-to-one to the new configuration API. Spring Boot's property-to-feature binding was removed to avoid incorrect mappings, but the removal wasn't flagged as a breaking change in early milestones.

HOW TO FIX

A **Register an ObjectMapper customiser bean.** Create a `Jackson2ObjectMapperBuilderCustomizer` that explicitly enables `WRITE_DATES_AS_TIMESTAMPS`. This replaces the dead properties-based config.

B **Migrate clients to ISO-8601 (recommended).** If you can coordinate with API consumers, drop the timestamp format entirely and adopt ISO-8601 strings. Remove the old property and let Jackson 3's new default take effect.

✓ **Verify:** Date fields serialize as ISO-8601 strings, not numeric timestamps

Scope check: Search your `application.properties` and `application.yml` files for any `spring.jackson.serialization` or `spring.jackson.deserialization` entries. Every one of those feature toggles is potentially dead in Boot 4.0.

WATCH OUT

This compounds with the jackson-date-format change. If you set `write-dates-as-timestamps=true` specifically to keep numeric timestamps, you now have two problems: the default flipped AND your override is ignored. You get ISO strings regardless.

Other `spring.jackson.serialization.*` and `spring.jackson.deserialization.*` properties may also be silently ignored. Audit all of them, not just the date-related ones.

FURTHER INFO

spring-break module: jackson-date-serialisation · Spring Boot 4.0 Migration Guide · Jackson 3 in Spring (blog)

Driven by the combination of Jackson 3.0 and Spring Boot 4.0's auto-configuration changes. The spring.jackson.serialization.* property binding was removed because Jackson 3.0 reorganised its feature flag system. Compounds with jackson-date-format. See also: jackson-date-format, jackson-property-inclusion.

Jackson Locale Format Change (BCP 47)

Jackson 3.0 serialises `Locale` using IETF BCP 47 format (zh-CN) instead of Java's underscore format (zh_CN). Locale-matching logic silently breaks.

EFFORT: S

WHAT YOU'LL SEE

```
// API response — before (Spring Boot 3.5)
{"userLocale": "zh_CN"}

// API response — after (Spring Boot 4.0)
{"userLocale": "zh-CN"}

// Downstream lookup fails silently
Map<String, String> labels = i18nMap.get(user.getLocale());
// returns null — map keys use "zh_CN", response now sends "zh-CN"

// Test failure
org.opentest4j.AssertionFailedError:
Expected: "zh_CN"
  Actual: "zh-CN"
```

WHAT CHANGED

Jackson 3.0 switched `Locale` serialisation from Java's `toString()` format (`zh_CN`) to the IETF BCP 47 language tag format (`zh-CN`). Underscores become hyphens and the structure follows RFC 5646.

```
// Custom serialiser to restore old format
// No custom serialiser needed in Boot 3.5

@JsonSerialize(using = JavaLocaleSerializer.class)
private Locale userLocale;


// Or update all consumers to use BCP 47
String localeKey = user.getLocale(); // "zh_CN"
String localeKey = Locale.forLanguageTag(user.getLocale()).toString(); // normalise

// Fix i18n lookup maps
i18nMap.put("zh_CN", chineseLabels);
i18nMap.put("zh-CN", chineseLabels);
```

WHY THIS CHANGED

BCP 47 is the web standard used by HTTP `Accept-Language` headers, HTML `lang` attributes, and JavaScript's `Intl` API. Aligning Jackson's output with the rest of the web ecosystem removes a constant source of mismatch bugs.

HOW TO FIX

A **Adopt BCP 47 throughout (recommended).** Update your i18n maps, database locale columns, and downstream consumers to use hyphenated BCP 47 tags. This aligns with HTTP headers and browser APIs. Run a data migration for persisted locale strings.

B **Write a custom Locale serialiser.** Register a custom Jackson serialiser that calls `locale.toString()` instead of `locale.toLanguageTag()` . This preserves the old underscore format while you plan the broader migration.

✓ **Verify:** Locale-sensitive fields render correctly in all target locales

Scope check: Search for any DTO or entity field of type `java.util.Locale` that gets serialised to JSON. Also check database columns, message queues, and i18n resource bundle keys that use the underscore format. Every one of those is a potential mismatch.

WATCH OUT

If you store locale strings in a database and compare them with equality checks, the format flip will cause silent mismatches. A `SELECT WHERE locale = 'zh_CN'` query won't match the new `zh-CN` value sent by the API.

Some locales have three components (e.g. `zh_Hant_TW` becomes `zh-Hant-TW`). The conversion isn't just replacing underscores with hyphens — BCP 47 has subtag rules. Use `Locale.forLanguageTag()` for correct round-tripping.

FURTHER INFO

spring-break module: jackson-locale-format · Jackson 3 migration guide · IETF BCP 47 (RFC 5646)

Driven by Jackson 3.0 (upstream of Spring Boot 4.0). Locale serialisation switched from Java's toString() format (zh_CN) to IETF BCP 47 / RFC 5646 language tags (zh-CN). Affects any DTO with java.util.Locale fields serialised via spring-boot-starter-json. See also: jackson-date-format, jackson-group-id.

spring.jackson.default-property-inclusion Silently Ignored

The `spring.jackson.default-property-inclusion` property no longer configures Jackson 3's ObjectMapper. Null fields reappear in your API responses without warning.

EFFORT: M

WHAT YOU'LL SEE

```
# application.properties (still present, no startup warning)
spring.jackson.default-property-inclusion=non_null

// API response — before (Spring Boot 3.5)
{"name": "Alice", "age": 30}

// API response — after (Spring Boot 4.0)
{"name": "Alice", "age": 30, "middleName": null, "nickname": null}

// Contract test failure
org.opentest4j.AssertionFailedError:
Expected JSON keys: [name, age]
  Actual JSON keys: [name, age, middleName, nickname]
```

WHAT CHANGED

Spring Boot 4.0's auto-configuration for Jackson 3 no longer reads the `spring.jackson.default-property-inclusion` property. The property sits in your config file doing nothing — no deprecation warning, no startup error. The ObjectMapper reverts to Jackson's default inclusion policy: `ALWAYS` (include everything, including nulls).

```
// application.properties — old (silently ignored)
spring.jackson.default-property-inclusion=non_null
# Removed — configure via ObjectMapper bean instead

// ObjectMapper customiser bean
# (relied on spring.jackson.default-property-inclusion)
@Bean
public Jackson2ObjectMapperBuilderCustomizer nonNullInclusion() {
    return builder -> builder
        .serializationInclusion(JsonInclude.Include.NON_NULL);
}


// Or per-class annotation
public class UserDto {
@JsonInclude(JsonInclude.Include.NON_NULL)
public class UserDto {
```

WHY THIS CHANGED

Jackson 3 restructured how serialisation inclusion is configured, moving from a single global enum to a more granular per-type system. The old Spring Boot property didn't map cleanly to the new API, so the auto-configuration binding was removed pending a redesign.

HOW TO FIX

A **Register an ObjectMapper customiser bean (recommended).** Create a `Jackson2ObjectMapperBuilderCustomizer` bean that calls `serializationInclusion(NON_NULL)`. This replaces the property-based configuration and works with Jackson 3.

B **Annotate DTOs individually.** Add `@JsonInclude(JsonInclude.Include.NON_NULL)` to each DTO class. More verbose, but gives you per-class control and makes the contract explicit in the code.

✓ **Verify:** JSON output excludes null fields where expected (compare before/after)

Scope check: Check whether your `application.properties` or `application.yml` sets `spring.jackson.default-property-inclusion`. If it does, that setting is now dead. Every API response from your app may now include null fields that were previously omitted.

WATCH OUT

There is no deprecation warning at startup. The property is silently ignored. The only signal is larger JSON payloads with unexpected null fields — easy to miss without contract tests.

If your front-end checks for key presence (e.g. `if ("middleName" in user)`) instead of checking for null values, the extra null fields will change branching logic silently.

FURTHER INFO

Spring-Break Demo · Spring Boot 4.0 Migration Guide · Jackson 3 in Spring (blog)

Driven by the combination of Jackson 3.0 and Spring Boot 4.0's auto-configuration changes. The spring.jackson.default-property-inclusion property binding was removed because Jackson 3.0 restructured its inclusion configuration API. Affects all spring-boot-starter-json consumers. See also: jackson-dates-timestamps, jackson-date-format.

LIKE Pattern Escaping for IsStartingWith / IsContaining

Spring Data now escapes `%` and `_` wildcards in derived LIKE queries. Search terms that relied on user-supplied wildcards silently stop matching.

EFFORT: S

WHAT YOU'LL SEE

```
// Repository method
List<Product> findByNameStartingWith(String prefix);

// Call: findByNameStartingWith("100%")

// Before (Spring Boot 3.5) — unescaped
SELECT * FROM product WHERE name LIKE '100%%'
// Matches: "100% Cotton", "1000 Islands", "100x Zoom"

// After (Spring Boot 4.0) — wildcards escaped
SELECT * FROM product WHERE name LIKE '100\%%' ESCAPE '\'
// Matches: "100% Cotton" only (correct!)

// Test failure (if tests relied on old wildcard behaviour)
Expected result size: 3
  Actual result size: 1
```

WHAT CHANGED

Spring Data JPA now automatically escapes SQL wildcard characters (`%` and `_`) in the parameter values for derived query methods like `IsStartingWith` , `IsEndingWith` , `IsContaining` , and `IsLike` . A search for `100%` now literally matches the percent sign instead of treating it as a wildcard.

```
// Generated SQL — the framework now adds ESCAPE
WHERE name LIKE ?1 || '%'
WHERE name LIKE ?1 || '%' ESCAPE '\'

// If you WANT wildcard passthrough, use @Query
List<Product> findByNameLike(String pattern);
@Query("SELECT p FROM Product p WHERE p.name LIKE :pattern")
List<Product> findByNameLike(@Param("pattern") String pattern);


// Fix tests that relied on unescaped wildcards
// Test: search for "100%" expected to match "1000 Islands"
List<Product> results = repo.findByNameStartingWith("100%");
assertThat(results).hasSize(3);

// Test: "100%" now matches only literal percent sign
List<Product> results = repo.findByNameStartingWith("100%");
assertThat(results).hasSize(1);  // only "100% Cotton"
```

WHY THIS CHANGED

The old behaviour was a subtle SQL injection vector and a correctness bug. If a user searched for `100%` , the unescaped `%` became a wildcard that matched any suffix — returning results the user didn't ask for. Escaping makes the queries correct by default.

HOW TO FIX

A **Embrace the fix (recommended).** The new escaping is correct behaviour. Update your tests to expect precise matching. If your app was returning extra results due to accidental wildcard expansion, those results were bugs.

B **Use @Query for intentional wildcard passthrough.** If you genuinely need users to pass `%` and `_` as wildcards, switch to an explicit `@Query` annotation with a hand-written LIKE clause. This opts out of the escaping.

✓ **Verify:** LIKE queries with % or _ in search terms return correct results

Scope check: Search for derived query methods containing `StartingWith` , `EndingWith` , `Containing` , `Like` , or `NotLike` . Any call that passes `%` or `_` in the parameter value will now behave differently. Also check any test data that includes these characters.

WATCH OUT

This is a correctness fix that may look like a regression. If your existing tests pass search terms containing `%` or `_`, those tests were unknowingly relying on wildcard expansion. The new behaviour is the right one — but your tests will still break.

The escaping applies to the parameter value, not to the `%` that Spring Data adds for `StartingWith` / `Containing` patterns. The framework's own wildcards are unaffected — only user-supplied characters are escaped.

FURTHER INFO

Spring-Break Demo · Spring Boot 4.0 Migration Guide

Introduced in Spring Data JPA 4.0 (upstream of Spring Boot 4.0). The escaping fix addresses a long-standing correctness bug and subtle SQL injection vector in derived LIKE queries. Affects spring-boot-starter-data-jpa consumers using IsStartingWith, IsEndingWith, IsContaining, or IsLike derived query methods.

@NullMarked Default — IDE/Compiler Null Safety

Spring Framework 7 applies `@NullMarked` to all packages. IDEs and null-checking tools now flag your existing null-passing code as warnings or errors — even though the runtime behaviour is unchanged.

EFFORT: M

WHAT YOU'LL SEE

```
// Your existing code — unchanged, worked fine in Boot 3.5
ApplicationContext ctx = new AnnotationConfigApplicationContext(AppConfig.class);
String name = ctx.getEnvironment().getProperty("app.name");
int length = name.length();  // ← IDE warning: name may be null

// IntelliJ inspection results after upgrade
WARNING: Method invocation 'length' may produce NullPointerException
  at MyService.java:42
WARNING: Passing 'null' argument to parameter annotated as @NonNull
  at MyConfig.java:18

// If using -Werror or NullAway/ErrorProne
$ mvn compile
[ERROR] MyService.java:[42,24] error: [NullAway] dereferencing expression
  name, which is @Nullable
[ERROR] MyConfig.java:[18,35] error: [NullAway] passing @Nullable parameter
  where @NonNull is required
```

WHAT CHANGED

Spring Framework 7 added `@NullMarked` (from JSpecify) at the package level across the entire framework. This means all method parameters and return types are assumed `@NonNull` by default unless explicitly annotated `@Nullable`. IDEs, static analysis tools, and compile-time null checkers now see Spring's null contracts and flag violations.

```
// Handle nullable return from getProperty()
~~String name = env.getProperty("app.name");~~
~~return name.toUpperCase();~~

String name = env.getProperty("app.name");
if (name == null) {
    throw new IllegalStateException("app.name not configured");
}
return name.toUpperCase();


// Or use the non-null variant
~~String name = env.getProperty("app.name");~~
String name = env.getRequiredProperty("app.name");

// Suppress warnings during migration
~~// (no null warnings in Boot 3.5)~~
// Temporarily suppress in build.gradle
tasks.withType(JavaCompile) {
    options.compilerArgs += ['-Xlint:-nullness']
}
```

WHY THIS CHANGED

Null safety has been a long-standing request. By declaring null contracts formally, the framework helps tools catch `NullPointerException` bugs at compile time instead of runtime. Kotlin interop also improves — Kotlin's type system can now correctly infer nullability from Spring APIs.

HOW TO FIX

A **Fix null handling in your code (recommended).** Follow the IDE warnings and add null checks, use `Optional`, or switch to non-null API variants like `getRequiredProperty()`. These are real potential NPE bugs that were always there — the framework is now telling you about them.

B **Suppress during migration.** If the volume of warnings is too high to fix at once, suppress null-checking warnings at the compiler level and create a backlog to address them incrementally. Don't leave the suppression in place permanently.

✓ **Verify:** No new NullPointerException at boundaries where nulls were OK before

Scope check: Open your project in IntelliJ or VS Code with Java support and look at the new inspection warnings. Any code that passes `null` to a Spring API parameter, or uses a Spring API return value without a null check, will be flagged. The volume depends on how much Spring API surface your code touches directly.

WATCH OUT

If your CI pipeline uses `-Werror`, NullAway, ErrorProne, or Checker Framework, the new null annotations will turn warnings into build failures. Your build breaks not because of code changes, but because the framework's null contracts are now visible to your tools.

Kotlin projects are affected differently. Kotlin's compiler uses these annotations to infer platform types. A Spring method that returned `String!` (platform type, nullable unknown) now returns `String` (non-null) or `String?` (nullable). Kotlin code that didn't handle nullability may now fail to compile.

FURTHER INFO

Spring Boot 4.0 Migration Guide · JSpecify @NullMarked

Introduced in Spring Framework 7.0 using JSpecify @NullMarked annotations at the package level. This is a compile-time / IDE-time change; runtime behaviour is unchanged. Kotlin interop improves because the compiler can now infer nullability from Spring APIs. Affects all Spring Framework API consumers.

Controller/View Spans Silently Disabled

Spring Boot 4.0 disables automatic controller and view rendering spans by default. Observability dashboards show gaps where traces used to appear — but the app itself works fine.

EFFORT: M

WHAT YOU'LL SEE

```
// Grafana / Jaeger trace — before (Spring Boot 3.5)
── HTTP GET /api/orders ──────────────────────────
  └── controller: OrderController.getOrders  [12ms]
      └── view: orders/list                  [3ms]
          └── jdbc: SELECT * FROM orders     [8ms]

// Grafana / Jaeger trace — after (Spring Boot 4.0)
── HTTP GET /api/orders ──────────────────────────
  └── jdbc: SELECT * FROM orders             [8ms]
// controller and view spans MISSING

// Dashboard alert
"No data" for panel "Controller Response Time p99"
// SLO breach: "99th percentile latency unknown"
```

WHAT CHANGED

Spring Boot 4.0 disabled the automatic instrumentation for Spring MVC controller method spans and view rendering spans by default. The properties `management.observations.http.server.requests.enabled` and related `spring.mvc.observation` settings changed their defaults. The traces still record HTTP and JDBC spans, but the controller-level granularity is gone.

```
// application.properties — re-enable controller spans
# (default in Boot 3.5: controller spans enabled)
management.tracing.enabled=true
spring.mvc.observation.auto-configuration.enabled=true


// Or enable via ObservationRegistry customiser
# (auto-instrumentation was on by default)
@Bean
public ObservationRegistryCustomizer<ObservationRegistry> serverSpans() {
    return registry -> registry.observationConfig()
        .observationHandler(new DefaultMeterObservationHandler(meterRegistry));
}


// Micrometer tracing config
# (no explicit config needed in Boot 3.5)
management.observations.http.server.requests.enabled=true
management.observations.http.client.requests.enabled=true
```

WHY THIS CHANGED

The automatic controller spans added overhead (an extra span per request) and duplicated information already available from the HTTP server span. For high-traffic services, the span volume was significant. The new default reduces overhead and storage costs, with opt-in for teams that need the detail.

HOW TO FIX

A **Re-enable controller observation spans.** Add `spring.mvc.observation.auto-configuration.enabled=true` and `management.observations.http.server.requests.enabled=true` to your `application.properties`. This restores the controller-level spans in your traces.

B **Update dashboards to use HTTP spans.** If the overhead matters, update your Grafana/Datadog dashboards to use the HTTP server span (`http.server.requests`) instead of the controller span. This span is still present and includes the handler information as attributes.

✓ **Verify:** All Grafana/Jaeger panels show data — no 'No data' gaps

Scope check: Check your observability dashboards, SLO definitions, and alerting rules. Any panel or alert that queries for controller-level span names (e.g. `controller.*`) or view rendering spans will show "No data" after the upgrade. Also check custom `SpanExporter` or `ObservationHandler` beans that filter by span name.

WATCH OUT

This is invisible in application logs and functionality. The app works perfectly. The only symptom is missing data in your observability platform — which you might not notice until an incident requires the traces that are no longer there.

If you have SLO alerting based on controller span latency percentiles, those alerts will fire as "no data" — which some alerting systems treat as "OK" (no data = no violations). You could silently lose SLO monitoring.

FURTHER INFO

Spring Boot 4.0 Migration Guide

Introduced in Spring Boot 4.0 as part of the Micrometer Tracing defaults overhaul. Controller and view rendering spans are disabled by default to reduce trace volume and overhead. This is the dashboard-impact perspective of the same change as controller-spans-disabled. See also: controller-spans-disabled, aspectj-observed.

Path Matching Engine: AntPathMatcher → PathPattern

Spring Security now uses `PathPattern` instead of `AntPathMatcher`. Some wildcard patterns match differently, causing silent authorisation gaps or false denials.

EFFORT: M

WHAT YOU'LL SEE

```
// Security configuration — unchanged
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/**/admin").hasRole("ADMIN")
);

// Before (AntPathMatcher — Spring Boot 3.5)
GET /api/v1/admin         → 403 (matched, requires ADMIN)
GET /api/v1/users/admin   → 403 (matched, ** crosses segments)

// After (PathPattern — Spring Boot 4.0)
GET /api/v1/admin         → 403 (still matched)
GET /api/v1/users/admin   → 200 (NOT matched — pattern behaves differently)

// Security test failure
Expected: request denied (403)
  Actual: request allowed (200)
// Potential security hole: endpoint exposed without authorisation
```

WHAT CHANGED

Spring Boot 4.0 switched the default URL path matching engine from `AntPathMatcher` to `PathPatternParser` for both MVC request mapping and Security request matchers. While most patterns work identically, edge cases around `**` in the middle of patterns, trailing slashes, and encoded path segments differ.

```
// Fix: use PathPattern syntax for multi-segment match
.requestMatchers("/api/**/admin").hasRole("ADMIN")
.requestMatchers("/api/{*path}/admin").hasRole("ADMIN")

// Trailing slash no longer matches by default
.requestMatchers("/api/users").authenticated()
.requestMatchers("/api/users", "/api/users/").authenticated()

// Fall back to AntPathMatcher if needed
// using default PathPatternParser
@Bean
public WebSecurityCustomizer legacyMatching() {
    return web -> web.httpFirewall(new StrictHttpFirewall());
}
// In security config:
.requestMatchers(new AntPathRequestMatcher("/api/**/admin"))
    .hasRole("ADMIN")
```

WHY THIS CHANGED

`PathPatternParser` is significantly faster (pre-parsed at startup, no runtime string manipulation), handles path variables and matrix parameters correctly, and is the default in Spring WebFlux. Unifying on one engine simplifies the framework.

HOW TO FIX

A **Audit all path patterns in security config.** Review every `requestMatchers()` pattern for uses of `**` in the middle (not at the end), trailing slashes, and URL-encoded segments. Test each pattern with PathPattern semantics. Use `{*varName}` for capturing multi-segment path variables.

B **Use AntPathRequestMatcher for legacy patterns.** For patterns that can't be rewritten, wrap them in `new AntPathRequestMatcher(pattern)` to use the old matching engine on a per-rule basis.

✓ **Verify:** All API endpoints respond (especially wildcard/regex patterns)

Scope check: Check every `.requestMatchers()` call in your security configuration and every `@RequestMapping` pattern in your controllers. Patterns with `**` in the middle, trailing slashes, or regex segments are the most likely to break. Simple patterns like `/api/users/**` (at the end) are generally safe.

WATCH OUT

A pattern that no longer matches is a potential security hole, not just a broken feature. If `/api/**/admin` stops matching certain paths, those paths become unprotected. This is a Tier 3 change with Tier 0 security implications.

`PathPattern` does not support `**` in the middle of a pattern (e.g. `/api/**/admin`). It only supports `**` at the end (e.g. `/api/**`). Use `{*path}` for variable-length segment capture in the middle.

FURTHER INFO

Spring-Break Demo · Spring Security 7 migration guide · Spring Boot 4.0 Migration Guide

Spring Boot 4.0 unified on PathPatternParser (already the default in WebFlux since 5.0) for both MVC and Security. AntPathMatcher is still available as a per-rule fallback. The key difference is that ** is only supported at the end of patterns in PathPattern. See also: security-dsl-rewrite, auth-default-deny.

maxAttempts Now Means Retries, Not Total Attempts

`@Retryable(maxAttempts = 3)` now performs 3 retries (4 total calls) instead of 3 total attempts. Your retry budget silently doubles.

EFFORT: M

WHAT YOU'LL SEE

```
// Before (Spring Boot 3.5): 3 total attempts
INFO  Calling payment service (attempt 1/3)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 2/3)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 3/3)
ERROR Payment failed after 3 attempts

// After (Spring Boot 4.0): 1 initial + 3 retries = 4 total
INFO  Calling payment service (attempt 1/4)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 2/4)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 3/4)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 4/4)
ERROR Payment failed after 4 attempts

// Integration test failure
Expected invocation count: 3
  Actual invocation count: 4
```

WHAT CHANGED

The `maxAttempts` parameter in `@Retryable` changed semantics. In Spring Retry 1.x (Boot 3.5), it meant total attempts including the initial call. In the new version (Boot 4.0), it means the number of retries after the initial call. So `maxAttempts = 3` now results in 4 total calls.

```
// Fix: reduce maxAttempts by 1 to preserve old behaviour
@Retryable(maxAttempts = 3)
@Retryable(maxAttempts = 2)  // 1 initial + 2 retries = 3 total

// Or use the new explicit parameter
@Retryable(maxAttempts = 3)
@Retryable(retries = 2)  // clearer: 2 retries after initial call

// RetryTemplate configuration
RetryTemplate template = RetryTemplate.builder()
    .maxAttempts(3)
    .build();

RetryTemplate template = RetryTemplate.builder()
    .maxAttempts(2)  // was 3 — now means retries, not total
    .build();
```

WHY THIS CHANGED

The old naming was misleading — "max attempts" of 3 producing only 2 retries was a constant source of off-by-one bugs. The new semantics align the parameter name with its actual meaning: the number of retry attempts.

HOW TO FIX

A **Subtract 1 from every maxAttempts value.** Search for all `@Retryable` annotations and `RetryTemplate` configurations. Reduce each `maxAttempts` value by 1 to preserve the original total attempt count.

B **Audit retry budgets.** The extra attempt may be acceptable or even desirable. Review each retry site and decide whether the new count is correct. Document the intended behaviour either way.

✓ **Verify:** Retried operations use correct backoff timing and max attempts

Scope check: Search for `@Retryable`, `RetryTemplate`, and any `maxAttempts` configuration in properties files. Every instance is now making one extra call than before. For services with tight SLAs or rate-limited downstream APIs, this can cause cascading failures.

WATCH OUT

The extra retry is invisible in logs unless you count carefully. Services with exponential backoff will take longer to fail — a `maxAttempts = 5` with 2-second backoff now takes 62 seconds total instead of 30.

If your downstream service has rate limiting, the extra call per failure scenario may push you over the limit. Multiply the extra attempt by your error rate to estimate the impact.

FURTHER INFO

spring-break module: retry-semantics-change · Spring Framework Resilience docs

Driven by Spring Retry 2.x (upstream of Spring Boot 4.0). The maxAttempts parameter changed from meaning total attempts to meaning retry count (excluding the initial call). Spring Retry was also removed from the Boot BOM in 4.0, requiring an explicit version. See also: spring-retry-bom, retryable-transaction-order.

@Retryable + @Transactional AOP Ordering Change

The default AOP advice ordering flipped: retry now wraps outside the transaction. A retried method gets a new transaction each attempt instead of retrying inside the same one.

EFFORT: M

WHAT YOU'LL SEE

```
// Before (Spring Boot 3.5) — retry inside transaction
DEBUG Opening transaction
DEBUG  Attempt 1: inserting order
WARN   Optimistic lock failure, retrying...
DEBUG  Attempt 2: inserting order
DEBUG  Attempt 2: success
DEBUG Committing transaction (1 commit total)

// After (Spring Boot 4.0) — retry outside transaction
DEBUG Opening transaction #1
DEBUG  Attempt 1: inserting order
WARN   Optimistic lock failure
DEBUG Rolling back transaction #1
DEBUG Opening transaction #2
DEBUG  Attempt 2: inserting order
DEBUG  Attempt 2: success
DEBUG Committing transaction #2

// Symptom: partial writes visible between retries
// Symptom: retry sees stale data from previous transaction
```

WHAT CHANGED

The default ordering of Spring AOP advice changed so that `@Retryable` has a higher precedence (runs first) than `@Transactional`. This means the retry logic wraps the transaction, not the other way around. Each retry attempt runs in its own transaction.

```
// Force retry inside transaction (old behaviour)
@Retryable(maxAttempts = 3)
@Transactional
public void placeOrder(Order order) {

@Retryable(maxAttempts = 3)
@Transactional
@Order(Ordered.HIGHEST_PRECEDENCE)  // force tx to wrap retry
public void placeOrder(Order order) {


// Or set global ordering in configuration
# (default ordering in Boot 3.5: tx outside, retry inside)
@EnableRetry(order = Ordered.LOWEST_PRECEDENCE)
// Ensures retry runs inside the transaction


// Best practice: separate the concerns
@Retryable @Transactional
public void placeOrder(Order order) { ... }

// Outer bean: handles retry
@Retryable(maxAttempts = 3)
public void placeOrderWithRetry(Order order) {
    orderService.placeOrder(order);
}
// Inner bean: handles transaction
@Transactional
public void placeOrder(Order order) { ... }
```

WHY THIS CHANGED

Retrying outside the transaction is generally safer — a failed transaction is fully rolled back before the next attempt starts, avoiding stale persistence context and dirty session state. The old default led to subtle Hibernate session corruption when retrying inside a failed transaction.

HOW TO FIX

A **Separate retry and transaction into different beans (recommended).** Move `@Retryable` to an outer service and `@Transactional` to an inner service. This makes the ordering explicit, avoids AOP precedence surprises, and works the same regardless of framework defaults.

B **Set explicit advice ordering.** Use `@EnableRetry(order = ...)` and `@EnableTransactionManagement(order = ...)` to explicitly control which advice runs first. Document the intended order.

✓ **Verify:** Retried transactional methods execute retries outside the transaction

Scope check: Search for any method annotated with both `@Retryable` and `@Transactional`. Each one has silently changed its transactional behaviour. Also check for `@Retryable` methods that call `@Transactional` methods on the same bean (self-invocation doesn't go through the proxy).

WATCH OUT

If your retry logic depends on reading data written earlier in the same transaction, that data is now rolled back before the retry. The second attempt starts with a clean slate, which may cause duplicate inserts or missing context.

Optimistic locking exceptions (`OptimisticLockException`) are now handled differently. Previously the retry happened inside the same persistence context. Now each retry gets a fresh EntityManager, which is usually better but changes the behaviour.

FURTHER INFO

Spring-Break Demo · Spring Boot 4.0 Migration Guide

Driven by a Spring Framework 7.0 change in default AOP advice ordering. @Retryable now has higher precedence than @Transactional, so retry wraps outside the transaction. The old default (retry inside transaction) led to Hibernate session corruption. See also: retry-semantics, spring-retry-bom.

Spring Data Default Query Generation Changed

Spring Data's derived query method resolution changed how it interprets compound property names. Queries that worked before now silently resolve to wrong columns or return wrong results.

EFFORT: M

WHAT YOU'LL SEE

```
// Repository method
List<Order> findByOrderStatusCode(String code);

// Before (Spring Boot 3.5) — resolved as:
// order.statusCode (single property traversal)
SELECT * FROM orders WHERE status_code = ?

// After (Spring Boot 4.0) — resolved as:
// order.status.code (nested property traversal)
SELECT * FROM orders o JOIN order_status s ON o.status_id = s.id
  WHERE s.code = ?

// Result: different rows returned, wrong data
// Test failure
Expected result size: 5
  Actual result size: 0  // no matching join path exists
```

WHAT CHANGED

Spring Data JPA changed its property resolution strategy for derived query methods. The parser now favours nested property traversal (splitting camelCase into entity associations) over flat property matching. A method like `findByOrderStatusCode` is now parsed as `order → status → code` instead of `order → statusCode`.

```
// Fix: use underscore separator for explicit splitting
List<Order> findByOrderStatusCode(String code);
List<Order> findByStatusCode(String code);  // flat property

// Or use underscore to force traversal boundary
List<Order> findByOrderStatusCode(String code);
List<Order> findByOrder_StatusCode(String code);  // explicit nesting

// Safest: use @Query annotation
List<Order> findByOrderStatusCode(String code);
@Query("SELECT o FROM Order o WHERE o.statusCode = :code")
List<Order> findByOrderStatusCode(@Param("code") String code);
```

WHY THIS CHANGED

The new resolution strategy is more consistent with JPA's entity graph model and handles deeply nested associations correctly. The old strategy had ambiguous cases where it was impossible to tell whether `StatusCode` was one property or two.

HOW TO FIX

A **Add explicit @Query annotations (recommended).** For any derived query method with compound property names, add an explicit `@Query` annotation. This removes all ambiguity and is immune to parser changes.

B **Use underscore separators.** Spring Data uses `_` as an explicit property boundary marker. `findByStatus_Code` unambiguously means `status.code` (nested), while `findByStatusCode` means the flat property.

✓ **Verify:** Repository methods return correct results (especially nulls and empties)

Scope check: Review every Spring Data repository interface. Any derived query method with two or more camelCase words after `findBy`, `countBy`, `deleteBy`, etc. is a candidate for changed resolution. Methods with simple single-word properties are safe.

WATCH OUT

The wrong resolution may still produce valid SQL — just against the wrong columns or join paths. The query runs, returns results, and your app looks fine. Only the data is wrong.

If your entity has both a flat property `statusCode` and a nested association `status.code`, the parser's choice between them has changed. The old behaviour preferred the flat property; the new behaviour prefers the association.

FURTHER INFO

spring-break module: hibernate-query-type-required · Spring Boot 4.0 Migration Guide

Introduced in Spring Data JPA 4.0 (upstream of Spring Boot 4.0). The property resolution strategy change favours nested entity traversal over flat property matching for derived query methods. Affects spring-boot-starter-data-jpa consumers with compound camelCase property names in repository method names.

What's At Stake

19	13	13
BUILD FAILURES	RUNTIME CRASHES	SILENT BUGS

Spring Boot 3.5 reaches end of support in November 2026. After that date, no security patches, no CVE fixes, no bug fixes. Every unpatched vulnerability is a compliance risk.

Option 1: Migrate Now

Start the migration while you still have time. Use this guide and OpenRewrite to move systematically.

Option 2: Migrate Incrementally

Use OpenRewrite recipes to automate what you can. Fix the rest module by module. Track progress against this checklist.

Option 3: Stay on 3.5 Safely

HeroDevs Never-Ending Support (NES) provides security patches, CVE fixes, and compliance certifications for Spring Boot after end-of-life. Same SLAs, same API — no migration required.

Talk to HeroDevs

herodevs.com/support/spring-nes
spring-nes@herodevs.com

Free migration assessment available — we'll estimate your specific migration effort.

© 2026 HeroDevs · herodevs.com